Mulligan: Performance-Guided Data Collection for Efficient On-Robot Learning

Lars Ankile **Rohan Bhowmik** **Perry Dong** **Aneesh Muppidi** **David D. Yuan**
Shuran Song **Chelsea Finn**

Stanford University

https://mulligan.page/ • ankile@stanford.edu

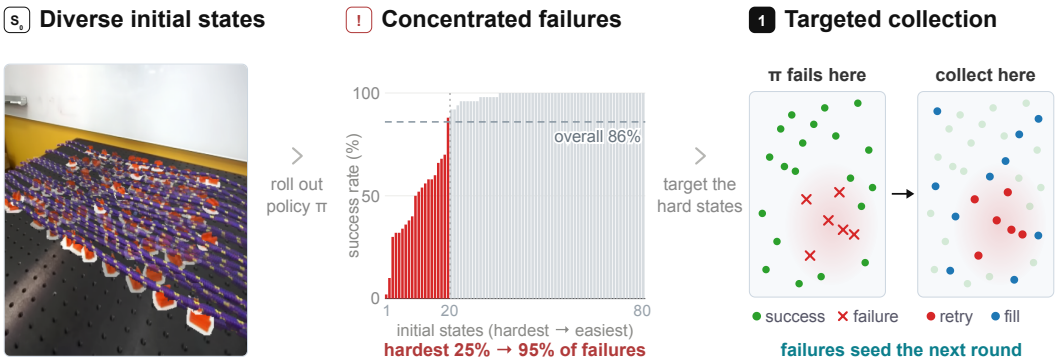


Figure 1: *Diverse initial states:* We study learning in deployment with wide initial-state distributions. *Concentrated failures:* Failures are not spread evenly across the initial-state space. *Targeted collection:* Retry observed failures (red) and fill the rest with the least-explored states (blue), to spend operator time where needed.

Abstract: Learning from human demonstrations is a reliable way to teach robots new tasks, but the gains from each additional demonstration shrink as the policy improves. Continued improvement can instead come from supervised deployment, where an operator places the objects and intervenes when the policy fails. We ask how to maximize improvement from a fixed budget of supervised episodes on high-precision manipulation tasks with wide ranges of object placements. We observe that failures can concentrate in a small subset of initial states, so uniform collection spends much of the operator's time on states the policy already handles. **Mulligan** makes the initial-state distribution a decision, starting each round's episodes at observed failures and untried states. To further improve data efficiency, we augment interactive imitation learning with a value function trained on all data, including failures that imitation discards. Across three real-world tasks evaluated on 2,550 held-out, blinded episodes and two simulated tasks, **Mulligan** outperforms uniform initial-state sampling at matched collection budgets, and combined with value-based action selection, HiL-IDQL+**Mulligan**, improves final real-task success by 10–34 percentage points. With operator interventions, the human-robot team completes 98% of collection episodes, remaining productive while the policy learns. Videos, code, and data are available at https://mulligan.page/.

Keywords: Human-in-the-loop learning, Data collection, Manipulation

Accepted to the R2RL Workshop at the Conference on Robot Learning (CoRL 2026).

1 Introduction

Human demonstrations are a popular starting point for teaching a robot a new manipulation task [1, 2, 3, 4, 5], but the returns on additional demonstrations often diminish as success rate improves, especially for tasks requiring high precision [6, 7, 8, 9]. A common next step is to collect corrections with a human in the loop (HiL), where an operator places the objects for each episode, takes control when the policy goes off track, and the corrections join the training set [10, 11, 12, 13]. Recent systems show this reliably achieves high success rates on physical hardware [13, 14, 15], but every episode still costs operator time. The cost is easier to bear in supervised deployment, where an operator already oversees the robot on real tasks and its rollouts arrive as a byproduct. We study the batch-online form of this loop [16], which retrains the policy only between rounds, and ask how to get the most improvement out of each supervised episode.

Several lines of work address parts of this question. Interactive imitation learning trains any policy class by behavior cloning (BC) on the corrections but cannot learn from the failed rollouts that deployment yields as a byproduct [12, 13]. Real-world RL learns from every rollout, failures included, through a value function, but gathers those rollouts by exploring online on the hardware [17, 18, 19, 20, 21, 22, 23, 24] and has shown its best results under narrow initial-state distributions [17, 18, 19] (ranges in App. K.3). In simulation, its learner [25] fails when the range widens, and human interventions [17] do not recover it (Fig. 27). Batch-online systems [16] combine corrections with a value function trained on all deployed rollouts, with a policy updated only between rounds and on the wide initial-state distributions of real deployment [14, 15]. The lines differ in policy class and learning update. Recent work scales corrections and RL to vision-language-action (VLA) policies rather than asking how much improvement each supervised episode yields.

Our key observation is that failures are often unevenly distributed across the initial-state space. E.g., in Fig. 1 (middle), 95% of a policy's failures fall in 25% of the initial states even though its overall success rate is 86%. Raising the success rate means correcting those failures where they happen. Under uniform collection, the policy makes fewer mistakes as it improves, and the operator becomes increasingly passive. Yet the operator who decides when to intervene also places the objects, so the initial-state distribution is a decision rather than a fixed part of the task, and can be made deliberately. The choice is independent of the policy class and learning update, so it combines with any of the methods above. The question is how to find those states without exhaustive search.

Mulligan finds them with the outcomes each round already produces, deciding which initial states the next round visits and what the operator records there (Fig. 1, right, and Sec. 3.1). The first round covers the initial-state space evenly and yields the first evidence of where the policy fails. Each later round balances retrying the initial states where the previous one failed with exploring the rest of the space for new failures. At a retried initial state, a correction begins where the policy went off track, so when the mistake is big, it teaches only recovery. Instead, the operator can record a full demonstration from the same initial state, a *counterfactual demonstration* that places the missing behavior in the policy's support. **Mulligan** only decides what each round collects, so we evaluate it with two learners, imitation alone and imitation with a value function trained on all rollouts, failures included, that selects among the policy's actions (Sec. 3.2).

Our main contribution is **Mulligan**, a data-collection strategy for batch-online imitation and reinforcement learning that selects each round's initial states from the policy's observed performance and yields more improvement from the same supervised budget. Across three real-world tasks (Fig. 18), evaluated on 2,550 held-out, blinded episodes, and two simulated tasks, **Mulligan** outperforms uniform initial-state sampling at matched supervised collection budgets. Combined with value-based action selection, HiL-IDQL+**Mulligan** improves final-round success on the real tasks by 10–34 percentage points over HG-DAgger [11] and completes more tasks per robot minute (Sec. 4). With an operator intervening, the human-robot team completes 98% of collection episodes with no round below 90%, so learning does not come at the cost of task completion.

2 Related Work

Human-in-the-loop robot learning. We build on interactive imitation learning, which collects human corrections under the policy's own state distribution and provides recovery supervision where the policy drifts [10, 11]. This line has studied when the human should intervene, by expert judgment [11] or estimated uncertainty [26, 27], and has scaled collection to long-horizon bimanual manipulation with expressive policies [7, 12, 13]. Interventions also carry implicit reward signal [28], and a second family combines human-in-the-loop collection with RL: RLIF [20] derives rewards from intervention signals, HiL-SERL [17] learns from corrections via RLPD [25], and ConRFT [19] and EXPO-FT [29, 21] extend this to finetuning VLAs. We similarly learn from the robot's own rollouts as well as the corrections, but update the policy between rounds rather than online. Closest to our setting are RECAP [14] and LWD [15], which improve VLAs over rounds of supervised deployment at scale. These recipes focus on the learning update, e.g., advantage-conditioned training on accumulated experience in RECAP. **Mulligan** instead decides where each round's episodes begin and leaves the learning update unchanged. We evaluate it with imitation alone and with a value function trained on all rollouts, and it could be combined with either recipe.

Choosing where episodes begin. Where episodes begin has been a design choice in RL since restart distributions were used to direct policy improvement [30]. Curricula move initial states toward intermediate difficulty, expanding outward from the goal [31], generating goals of suitable difficulty [32], resetting to states along demonstrations [33, 34, 35, 36, 37] or a guide policy's rollouts [38], or returning to stored states and replaying the levels with the highest learning potential alongside fresh ones [39, 40]. Narvekar et al. [41] and Portelas et al. [42] survey the area. Hard-example mining concentrates training on what a model gets wrong within a fixed dataset [43]. For real robots, recent work chooses what to collect by which environmental factors to vary [44], which failure conditions a model predicts [45], where the policy is uncertain [46], which failure states its executions reveal [47, 48], which perturbations to apply during a demonstration [49], or in what order a human demonstrates [50]. **Mulligan** moves the initial-state choice to a supervised deployment, where the operator places each initial state, and mixes retried and fresh states as prioritized level replay does. Because the operator can complete the task from any state, collection is not limited to a band of intermediate difficulty and can go to the hardest states found so far, where a counterfactual demonstration records the missing behavior within the same round. Unlike the easy-to-hard curriculum of Hou et al. [50], **Mulligan** retries the exact initial states where the policy failed.

3 Mulligan: Performance-Guided Collection for Supervised Deployment

Mulligan is a collection strategy that decides where each round's episodes start and what the operator records there (Sec. 3.1). We pair it with two learners. HG-DAgger [11] behavior-clones the human data, and HiL-IDQL, our human-in-the-loop variant of IDQL [51], keeps the same actor and adds a critic trained on all data, failures included. Actor and critic learning are decoupled, and at deployment the critic reranks the actor's samples (Sec. 3.2). Alg. 2 gives the loop both learners run.

Problem setup. We model each task as a Markov decision process $(\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0)$ with states $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, transition dynamics P, a sparse reward r that is 1 when the task is completed and 0 otherwise, discount γ, and an initial-state distribution ρ_0, uniform over a bounded range of object poses $\mathcal{S}_0$. The goal is a policy π_θ that maximizes autonomous success rate on initial states drawn from ρ_0. We study a *human-in-the-loop* (HiL) *batch-online RL* setting that proceeds in rounds under the supervision of an *operator*. The operator places the objects at the start of each episode, watches the policy act, and may take over to correct it. Before the first round, the operator records a base set $\mathcal{D}_0$ of demonstrations. Each round then deploys the policy for K episodes without intervention to measure performance, and for a budget of M supervised episodes, in which the operator intervenes when the policy fails. Every transition from both is stored. Between rounds, we retrain the models on all accumulated data and redeploy. The policy's parameters stay fixed within a round, since physical deployment favors a stable and performant policy over one that changes

Algorithm 1 SELECTINITIALSTATES: performance-guided initial-state selection

Require: budget M; collected initial states $\mathcal{P}$; initial states $\mathcal{F}$ of the round's evaluation failures; promotion cap m_f; nearest-neighbor distance $d(s, \mathcal{Y}) = \min_{y \in \mathcal{Y}} \|s - y\|$

$\mathcal{G} \leftarrow$ up to m_f states from $\mathcal{F}$, earliest failure stage first ▷ failure promotion

while $|\mathcal{G}| < M$ **do**

  $\mathcal{G} \leftarrow \mathcal{G} \cup \{\arg\max_{s \in \mathcal{S}_0} d(s, \mathcal{P} \cup \mathcal{G})\}$ ▷ coverage fill

end while

return $\mathcal{G}$

continually through online exploration. We analyze the accumulated data at each round boundary to plan the next round. Because the operator places the objects, the supervised episodes can start from any initial state in $\mathcal{S}_0$ rather than from states sampled from ρ_0, and **Mulligan** chooses which ones.

Data and notation. Supervised deployment produces four kinds of data. The operator provides *demonstrations*, teleoperated completions of the whole task, which include $\mathcal{D}_0$ and the counterfactual demonstrations of Sec. 3.1, and *corrections*, in which the operator takes over mid-episode. The robot contributes its own *autonomous successes* and *autonomous failures*. Besides the reward, each frame carries only a tag saying whether the human or the policy controlled it. We write $\mathcal{D}_h$ for the human-controlled frames of successful episodes (demonstrations, counterfactuals, and the human segments of corrections) and $\mathcal{D}_{\text{all}}$ for every collected frame, failures included.

Learning algorithms. We train an actor and a critic separately and combine them only at deployment. The actor $\pi_\theta(\mathbf{a} \mid s)$ is a Diffusion Policy [1], a generative model over *action chunks* $\mathbf{a} = (a_t, \ldots, a_{t+H-1})$, of which the first $h \leq H$ actions, the *executed prefix* $\mathbf{a}_{1:h}$, are executed before the policy is queried again. BC fits π_θ to a dataset of state and chunk pairs with the diffusion denoising objective, a reweighted variational bound on the log-likelihood. The critic is trained by implicit Q-learning (IQL) [52], which fits a value function V_ξ to Q_ψ by expectile regression with level $\tau > 0.5$, so that V_ξ tracks the better actions in the data, and regresses Q_ψ onto $r + \gamma V_\xi(s')$, all without sampling actions from the policy. The critic scores only the executed prefix of a chunk, so $\mathbf{a}$ inside Q_ψ denotes $\mathbf{a}_{1:h}$ throughout. At deployment, implicit diffusion Q-learning (IDQL) [51] uses the critic to select among the actor's samples, improving the actor without gradients. At each state it draws N chunks $\{\mathbf{a}_1, \ldots, \mathbf{a}_N\} \sim \pi_\theta(\cdot \mid s)$ and executes

$$\mathbf{a}_{i^\star}, \qquad i^\star = \arg\max_i Q_\psi(s, \mathbf{a}_i), \tag{1}$$

which we refer to as best-of-N (BoN) sampling and which later work has adopted as a stable improvement operator for generative policies [16, 53, 29].

3.1 Performance-guided collection

As Fig. 1 (middle) shows, not all of $\mathcal{S}_0$ is equally difficult. The policy is near-perfect over most of $\mathcal{S}_0$, and a small region accounts for almost all of its failures, even though the policy was trained on episodes with evenly-spread initial states (per-region success rates in App. F.1). When failures cluster this way, collecting uniformly across the initial states spends most of the operator's time on states where the policy is already proficient. The states worth visiting are those where the policy can learn the most. Autonomous improvement with RL is often most effective from states with *intermediate difficulty*, where the policy sometimes succeeds and sometimes fails, and curricula are designed to stay in that regime [32, 41, 54]. A human-in-the-loop obviates the need for a curriculum, as the operator can complete the task from any state and directly supply the missing behavior in the hardest states, where it stands to help the most. The question becomes how to find these hard states, since hardness is observed only where the policy has already been tried. Our collection strategy balances returning to past failures with improving the coverage of $\mathcal{S}_0$, so that we also discover new failures as known ones are fixed. Alg. 1 selects each round's initial states. We describe its two steps, *failure promotion* and *coverage fill*, and then *counterfactual demonstrations*.

Failure promotion. An observed failure is the best available evidence of where the policy tends to fail. Each round initializes part of its episodes at the exact initial states where the deployed policy failed in the evaluation just completed. Failures are taken in order of the task stage at which the policy failed, earliest and most severe first (stage labels in App. C.3), up to m_f promoted failures, which reserves the rest of the budget for the coverage fill (guardrails in App. F.7).

Coverage fill. Coverage begins before any failure has been observed. With a budget of tens of episodes, i.i.d. uniform samples leave gaps and clusters by chance, so the round-0 base demonstrations are recorded at initial states drawn from a scrambled Sobol sequence [55, 56, 57], a *low-discrepancy* sequence whose points cover $\mathcal{S}_0$ more evenly (comparison in App. F.2). In later rounds, the episodes left after promotion keep supervision reaching unvisited parts of $\mathcal{S}_0$. These depend on everything collected so far, so the coverage fill uses farthest-point selection [58, 59]. Each coverage-fill state is the point of $\mathcal{S}_0$ farthest from every collected initial state, including this round's promoted failures. The distance d in Alg. 1 is normalized Euclidean, computed after scaling each pose coordinate to the unit interval and treating yaw as periodic, with per-task axis weights in App. H.4.

Counterfactual demonstrations. A correction begins from a state the policy reached on its own, so it demonstrates recovery from a mistake rather than what the policy should have done. For example, when the policy drops an object in transit, the correction picks it back up, which is useful, but the more important lesson is to carry the object without dropping it. In such cases, the operator completes the correction, resets to the same initial state, and demonstrates the whole task, a *counterfactual demonstration* (CF) that places the missing behavior in the actor's support. When corrections only need to nudge a stuck or slightly misaligned robot to demonstrate the intended behavior, no CF is recorded. We count each CF toward the M-episode budget so rounds with and without CFs cost the same number of episodes.

Instantiation. Round 0 records 100 demonstrations (200 on SQUARE-BROAD), and each later round collects $M = 50$ episodes (100 on ROUTE CABLE and in simulation). In the real world, the operator places objects from the reset schematic in Fig. 22. Per-task selection settings, including an optional coverage-fill bias toward harder states, are in Apps. F.6 and H.4.

3.2 Policy learning: decoupled actor and critic

Collection produces two pools of data: human data from successful episodes only and autonomous rollouts that include outright failures. The actor π_θ is trained by BC on $\mathcal{D}_h$ alone, whereas prior work folds autonomous successes into the actor's mixture [16, 13], which in our ablation lowers success (Sec. 4.2). The critic (Q_ψ, V_ξ) is trained on all of $\mathcal{D}_{\text{all}}$, so failures that the actor's data excludes still inform action selection. No gradients pass between the two, so the actor can be any policy class, and each component can be evaluated on its own. Beyond the choice of data, the critic departs from IQL in its transition length, target, value head, and action-input scaling (App. G.2).

4 Experiments

Our experiments address three questions. **Q1.** How does **Mulligan** compare to existing methods (Sec. 4.1)? **Q2.** What does each part of **Mulligan** contribute (Sec. 4.2)? **Q3.** Does learning come at the cost of completing tasks during collection (Sec. 4.3)?

4.0 Experiment Setup

Task suite. For real-world experiments, we use the DROID [5] Franka Panda platform and evaluate the three tasks in Fig. 18: INSERT MARKER (grasp a pen and insert it into a holder), THREAD NUT (grasp a nut and place it onto a peg), and ROUTE CABLE (seat a rope into two clips in succession). All tasks end in a tight-clearance insertion (e.g., $\sim$1 mm for INSERT MARKER) or seating

stage where small pose errors cause failure, and objects are randomized over wide initial-state ranges (e.g., 15–46 cm). Fig. 22 and Fig. 1 (left) show the ranges, and App. K.3 lists them.

In simulation, we use two Robosuite [60] versions of the same square-nut insertion task. SQUARE-NARROW is the Robomimic [6] Square task, which randomizes the nut pose with the peg fixed. SQUARE-BROAD is the MimicGen [9] variant that expands the nut-position range and also randomizes the peg position (implementation names and versions in App. C.1).

Compared methods. The human-in-the-loop methods vary two design choices. Initial states are either sampled uniformly or chosen by **Mulligan**, which selects them from the policy's observed performance and adds counterfactual demonstrations (Sec. 3.1, App. B.10). The learner is either HG-DAgger [11], which behavior-clones the human data, or HiL-IDQL, which keeps the same actor, trained on human data, and adds an IQL critic, trained on all data, that reranks N sampled action chunks at deployment (BoN, Sec. 3.2; N per task in App. H.3). We name each method by the learning method and append "+**Mulligan**" when collection is performance-guided, giving HG-DAgger, HG-DAgger+**Mulligan**, HiL-IDQL, and HiL-IDQL+**Mulligan**. All four share a Diffusion Policy U-Net actor [1], the critic formulation, and the per-round collection budget (architectures and hyperparameters in App. G). The budget counts supervised episodes, each of which occupies the operator from start to finish, and not human-controlled frames. Because **Mulligan** fills part of each round's budget with counterfactual demonstrations, its datasets hold 19%, 28%, and 8% more human frames on the three real tasks (App. B.10). We run all 4 combinations in simulation. In the real world, we run one collection campaign per strategy, so we deploy HG-DAgger against the full method, HiL-IDQL+**Mulligan**. Reranking helps only once on-policy data has accumulated (Sec. 4.2), so we deploy the actor alone in rounds 0–2 and the reranked actor from round 3 on (teal and purple points in Fig. 2). In simulation, we also compare against autonomous self-improvement methods, branching from the shared round-0 checkpoints: self-imitation retrains on its own rollouts [61, 1], filtered BC on only the successful ones [62], and batch-online IDQL trains a critic on all rollouts for BoN reranking [51, 16]. RLPD [25, 29] and HiL-SERL [17] start from the same demonstrations.

Evaluation metrics. We report autonomous success rate (SR) on held-out initial states and, on SQUARE-NARROW, its complement, failure rate ($100 - \text{SR}$). On ROUTE CABLE, the primary outcome is full-task success (both clips seated), and Fig. 2 plots the finer clip success rate, the mean 0–2 clip score divided by two. Execution speed (time to complete successful episodes), throughput (completed tasks per robot minute, counting time spent on failures), and the intervention rate during collection are in Apps. B.3 and B.9.

Evaluation protocol. We keep evaluation separate from collection and run it without interventions. Each round scores every method on one fresh block of 50 Sobol-sampled initial states that no policy has trained from (Sec. 3.1), with the methods interleaved on a shuffled list under operator-blind labels, and collection is blinded and interleaved the same way. After scoring, the block's rollouts enter $\mathcal{D}_{\text{all}}$, so success rates are comparable within a round but, with a new block each round and possible day-to-day drift, not across rounds. In simulation, every policy trains for a preset number of gradient steps and is scored on one fixed set of initial states over five seeds. Blinding, paired per-state tests, and the Wilson [63] and Student-t interval constructions are in App. D.

4.1 Q1: How does Mulligan compare to existing methods?

Held-out performance. On all three real tasks in Fig. 2, **Mulligan** ends above uniform sampling, and pooled over all rounds the improvement is significant on each task (paired tests over initial states, $p < 0.005$ on INSERT MARKER and THREAD NUT, and $p = 0.04$ on ROUTE CABLE). At the final checkpoint, HiL-IDQL+**Mulligan** finishes $+34$, $+10$, and $+16$ percentage points (pp) above HG-DAgger in full-task success on INSERT MARKER, THREAD NUT, and ROUTE CABLE (tests in App. B.2). Its actor alone, HG-DAgger+**Mulligan**, already finishes $+22$, $+8$, and $+10$ pp above HG-DAgger, and reranking adds $+12$, $+2$, and $+6$ pp (Tab. 1). The plotted ROUTE CABLE

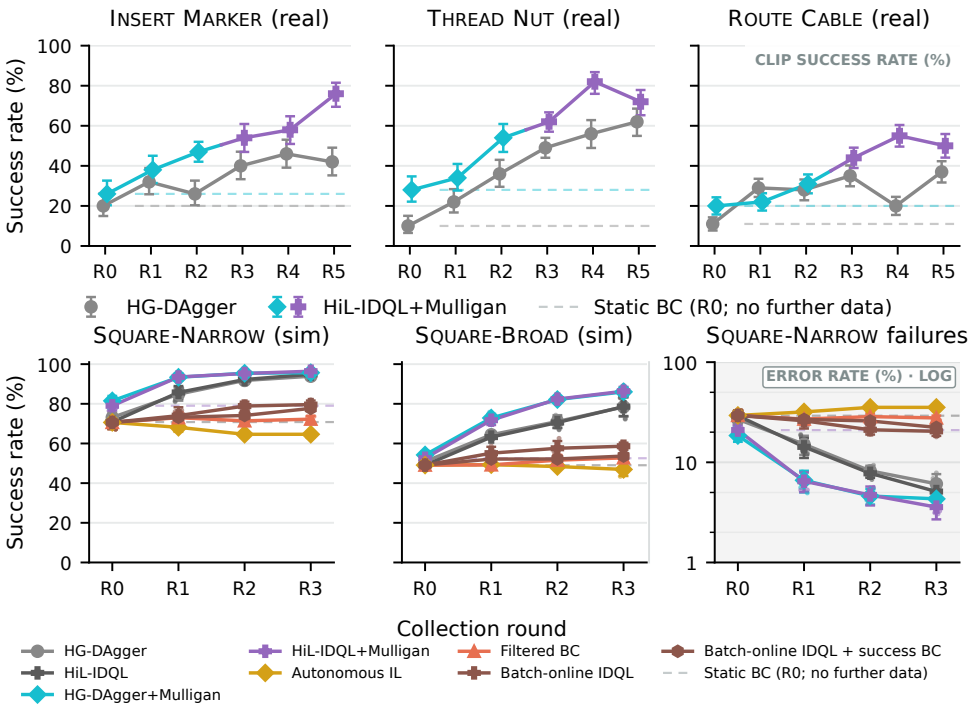


Figure 2: **Held-out performance over DAgger rounds, real world (top) and simulation (bottom).** Real-world panels show full-task success on INSERT MARKER and THREAD NUT and clip success rate on ROUTE CABLE, the fraction of two clips seated (per-method full-task success in App. B.1). Real-world whiskers are ± 1 SE over 50 episodes, or 100 for the pooled INSERT MARKER R2 HiL-IDQL+**Mulligan** point and both THREAD NUT R3 points (App. D). SQUARE-NARROW failures ($100 - \text{SR}$) use a log scale. Dashed lines mark each campaign's static round-0 policy. For real-world HiL-IDQL+**Mulligan**, teal points use the actor alone and purple points add critic reranking.

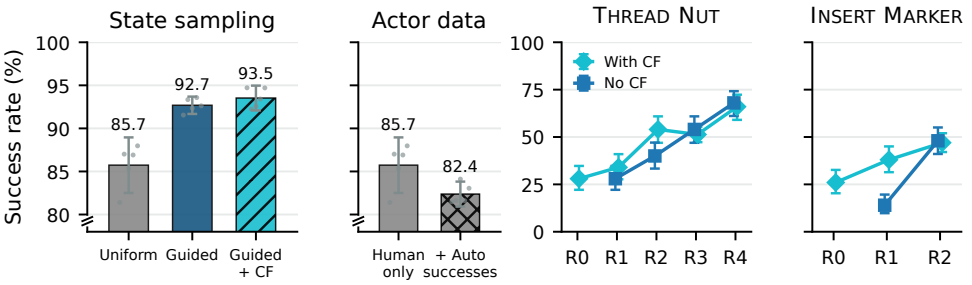


Figure 3: **Component ablations in simulation (left two) and the real world (right two). State sampling** and **actor data** on SQUARE-NARROW round 1, with five-seed means, seed dots, and 95% t-intervals (App. B.5). **Counterfactual demos** on THREAD NUT and INSERT MARKER through the last measured no-CF round, with Wilson ± 1 SE whiskers. R0 is the shared source actor.

clip success rate is 37% for HG-DAgger and 50% for HiL-IDQL+**Mulligan**. HiL-IDQL+**Mulligan** also completes more tasks per robot minute than HG-DAgger on every real task (App. B.3).

In simulation, the HiL methods improve far more than any autonomous method. Without the human in the loop, only the BoN-reranked methods improve, so a value-based objective appears necessary. On SQUARE-NARROW, HiL-IDQL+**Mulligan** continues to drive failures down through the final round (Fig. 2, log scale). There, HiL-SERL needs over ten times the frames of HiL-IDQL+**Mulligan** to pass 90%, and RLPD does not pass it (Fig. 27, App. I.4).

4.2 Q2: What does each component contribute?

Data-collection strategies. *Initial-state sampling strategy:* Holding learning fixed, we compare collection strategies. Sobol sampling of the round-0 initial states yields equal or better demo-only point estimates on all five tasks, largest on THREAD NUT and SQUARE-NARROW ($+8.2$ pp). On SQUARE-NARROW round 1 (Fig. 3, first panel), mean success rises from uniform to performance-guided collection, with a smaller further increase with CF demos. Full comparisons and sampler definitions are in Apps. B.5 and F.

Counterfactual demonstrations: Fig. 3 (right two panels) compares performance-guided collection with and without CF demos on THREAD NUT and INSERT MARKER. CF demos raise round-1 success from 14% to 38% on INSERT MARKER and from 28% to 34% on THREAD NUT, and the arms converge by round 2 on INSERT MARKER and by rounds 3–4 on THREAD NUT. CF demos thus appear most useful early, before corrections have accumulated (larger panels in App. B.6).

Decoupled actor- and critic-learning. *Value-augmented sampling:* Holding collection fixed, we compare the actor alone with the critic-reranked actor (all methods and rounds in App. B.1). Reranking raises success only in later rounds, once on-policy data has accumulated. The two components help at different stages, with performance-guided collection mainly improving the grasp stage of INSERT MARKER and THREAD NUT and the critic the precise insertion stage (App. B.7).

Actor-critic data curation: In Fig. 3 (second panel), adding successful autonomous segments to the actor's human data lowers mean success by 3.4 pp, with a 95% Welch interval of $[-6.5, -0.2]$ (critic and data-mixture ablations in App. J).

4.3 Q3: Does learning impede productivity?

Deployment-time collection is productive. We count a collection episode as completed when the task ends in success, autonomously or after a correction, and exclude counterfactual replays. Despite low initial success rates and performance-guided sampling, the human-robot team completes 98% of the 1,009 policy-initiated collection episodes across the three real tasks, with no round below 90%, even though a mistake the operator cannot recover from, such as an object knocked off the table, ends the episode as a failure (App. B.8). Each round is thus a batch of tasks completed at near-perfect reliability that also improves the policy, and the share of human-controlled frames generally declines across rounds (App. B.9).

5 Discussion

In supervised deployment, choosing each round's initial states from observed failures and untried states, with counterfactual demonstrations where a correction would only teach recovery, improves the policy faster than uniform collection at equal budget on three real and two simulated tasks, and a critic trained on all rollouts, failures included, adds later-round gains by reranking actor samples.

We identify three limitations. First, **Mulligan** assumes initial states can be parameterized and placed at collection time, which supervised collection provides but a fully autonomous deployment may not. Without it, the robot would have to choose among arriving initial states or move objects to informative ones itself, which we did not test. Second, we evaluated **Mulligan** with one imitation learner and one value-based learner (critic variants in Tab. 25), and how it performs with other learners and base policies, including generalist ones updated between rounds, is open. Third, the critic's gains stay outside the actor. Adding the actor's successful autonomous rollouts to its data degrades it (Sec. 4.2), but training it on the reranked actions could distill these gains. More broadly, our results suggest that how fast a deployed policy improves depends on which experience it collects.

Acknowledgments

We thank Andy Tang for help with running evaluations, Ajay Sridhar for help setting up the physical robot, and Colin Li for helpful discussions early in the project. We also thank Marcel Torne, Anthony Simeonov, and Luke Bailey for feedback on the paper draft. We thank John Yao for help with the tasks, including props and 3D printing, Zhanyi Sun for useful input in early project discussions, and the members of the REAL Lab for input during presentations. This work was supported by an NSF CAREER award.

Compute for this work was provided by Yotta Labs Inc. through its Research Credit Program. This work used Anvil at Purdue University and Delta at the National Center for Supercomputing Applications through allocation CIS251374 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296. Delta is supported by the National Science Foundation (award OAC 2005572) and the State of Illinois. Some of the computing for this project was performed on the Sherlock cluster. We would like to thank Stanford University and the Stanford Research Computing Center for providing computational resources and support that contributed to these research results.

References

[1] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.

[2] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

[3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[4] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[5] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, P. D. Fagan, J. Hejna, C. Finn, and D. Sadigh. DROID: A large-scale in-the-wild robot manipulation dataset. In *Robotics: Science and Systems (RSS)*, 2024.

[6] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kacholia, Y. Zhu, L. Fan, A. Garg, S. Savarese, L. Fei-Fei, R. Martín-Martín, and F. Xia. What matters in learning from offline human demonstrations for robot manipulation. In *Conference on Robot Learning (CoRL)*, 2021.

[7] T. Z. Zhao, J. Tompson, D. Driess, P. Florence, K. Ghasemipour, C. Finn, and A. Wahid. ALOHA Unleashed: A simple recipe for robot dexterity. *arXiv preprint arXiv:2410.13126*, 2024.

[8] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal. From imitation to refinement – residual RL for precise assembly. *arXiv preprint arXiv:2407.16677*, 2024.

[9] A. Mandlekar, Y. Zhu, L. Fan, D. Xu, R. Martín-Martín, L. Fei-Fei, S. Savarese, A. Garg, and F. Xia. MimicGen: A data generation system for scalable robot learning using human demonstrations. In *Conference on Robot Learning (CoRL)*, 2023.

[10] S. Ross, G. J. Gordon, and J. A. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[11] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer. HG-DAgger: Interactive imitation learning with human experts. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2019.

[12] P. Wu, Y. Shentu, Q. Liao, D. Jin, M. Guo, K. Sreenath, X. Lin, and P. Abbeel. RoboCopilot: Human-in-the-loop interactive imitation learning for robot manipulation. *arXiv preprint arXiv:2503.07771*, 2025.

[13] Z. Hu, R. Wu, N. Enock, J. Li, R. Kadakia, Z. Erickson, and A. Kumar. RaC: Robot learning for long-horizon tasks by scaling recovery and correction. *arXiv preprint arXiv:2509.07953*, 2025.

[14] Physical Intelligence. $\pi_0.6$: A VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.

[15] Y. Wang, X. Li, P. Xie, P. Yang, B. Nie, Y. Cai, Q. Zhang, C. Qu, J. Wu, J. Song, X. Ren, J. Huang, M. Pan, S. Feng, Z. Chen, and J. Luo. Learning while deploying: Fleet-scale reinforcement learning for generalist robot policies. *arXiv preprint arXiv:2605.00416*, 2026.

[16] P. Dong, S. Mirchandani, D. Sadigh, and C. Finn. What matters for batch online reinforcement learning in robotics? *arXiv preprint arXiv:2505.08078*, 2025.

[17] J. Luo, C. Xu, J. Wu, and S. Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *arXiv preprint arXiv:2410.21845*, 2024.

[18] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine. SERL: A software suite for sample-efficient robotic reinforcement learning. *arXiv preprint arXiv:2401.16013*, 2024.

[19] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao. ConRFT: A reinforced fine-tuning method for VLA models via consistency policy. *arXiv preprint arXiv:2502.05450*, 2025.

[20] J. Luo, F. Dong, Y. Zhai, Y. Ma, and S. Levine. RLIF: Interactive imitation learning as reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024.

[21] P. Dong, K.-H. Hung, T. Gao, D. Sadigh, and C. Finn. EXPO-FT: Sample-efficient reinforcement learning finetuning for vision-language-action models. *arXiv preprint arXiv:2605.25477*, 2026.

[22] K. Lei, H. Li, D. Yu, Z. Wei, L. Guo, Z. Jiang, Z. Wang, S. Liang, and H. Xu. RL-100: Performant robotic manipulation with real-world reinforcement learning. *arXiv preprint arXiv:2510.14830*, 2025.

[23] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi. Residual off-policy RL for finetuning behavior cloning policies. *arXiv preprint arXiv:2509.19301*, 2025.

[24] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, G. Shi, and Y. Zhu. Self-improving vision-language-action models with data generation via residual RL. *arXiv preprint arXiv:2511.00091*, 2025.

[25] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning (ICML)*, 2023.

[26] R. Hoque, A. Balakrishna, E. Novoseller, A. Wilcox, D. S. Brown, and K. Goldberg. ThriftyDAgger: Budget-aware novelty and risk gating for interactive imitation learning. In *Conference on Robot Learning (CoRL)*, 2021.

[27] R. Hoque, L. Y. Chen, S. Sharma, K. Dharmarajan, B. Thananjeyan, P. Abbeel, and K. Goldberg. Fleet-DAgger: Interactive robot fleet learning with scalable human supervision. In *Conference on Robot Learning (CoRL)*, 2022.

[28] J. Spencer, S. Choudhury, A. Venkatraman, B. Zhan, and J. A. Bagnell. Learning from interventions: Human-robot interaction as both explicit and implicit feedback. In *Robotics: Science and Systems (RSS)*, 2020.

[29] P. Dong, Q. Li, D. Sadigh, and C. Finn. EXPO: Stable reinforcement learning with expressive policies. *arXiv preprint arXiv:2507.07986*, 2025.

[30] S. Kakade and J. Langford. Approximately optimal approximate reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2002.

[31] C. Florensa, D. Held, M. Wulfmeier, M. Zhang, and P. Abbeel. Reverse curriculum generation for reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2017.

[32] C. Florensa, D. Held, X. Geng, and P. Abbeel. Automatic goal generation for reinforcement learning agents. In *International Conference on Machine Learning (ICML)*, 2018.

[33] T. Salimans and R. Chen. Learning Montezuma's Revenge from a single demonstration. *arXiv preprint arXiv:1812.03381*, 2018.

[34] C. Resnick, R. Raileanu, S. Kapoor, A. Peysakhovich, K. Cho, and J. Bruna. Backplay: "Man muss immer umkehren". *arXiv preprint arXiv:1807.06919*, 2018.

[35] S. Dai, A. Hofmann, and B. Williams. Automatic curricula via expert demonstrations. In *NeurIPS Deep Reinforcement Learning Workshop*, 2021.

[36] S. Tao, A. Shukla, T.-K. Chan, and H. Su. Reverse forward curriculum learning for extreme sample and demonstration efficiency in reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024.

[37] M. Bauza, J. E. Chen, V. Dalibard, N. Gileadi, R. Hafner, M. F. Martins, J. Moore, R. Pevceviciute, A. Laurens, D. Rao, M. Zambelli, M. Riedmiller, J. Scholz, K. Bousmalis, F. Nori, and N. Heess. DemoStart: Demonstration-led auto-curriculum applied to sim-to-real with multi-fingered robots. *arXiv preprint arXiv:2409.06613*, 2024.

[38] I. Uchendu, T. Xiao, Y. Lu, B. Zhu, M. Yan, J. Simon, M. Bennice, C. Fu, C. Ma, J. Jiao, S. Levine, and K. Hausman. Jump-start reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2023.

[39] A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley, and J. Clune. First return, then explore. *Nature*, 590:580–586, 2021.

[40] M. Jiang, E. Grefenstette, and T. Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning (ICML)*, 2021.

[41] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181):1–50, 2020.

[42] R. Portelas, C. Colas, L. Weng, K. Hofmann, and P.-Y. Oudeyer. Automatic curriculum learning for deep RL: A short survey. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2020.

[43] A. Shrivastava, A. Gupta, and R. Girshick. Training region-based object detectors with online hard example mining. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[44] J. Gao, A. Xie, T. Xiao, C. Finn, and D. Sadigh. Efficient data collection for robotic manipulation via compositional generalization. In *Robotics: Science and Systems (RSS)*, 2024.

[45] S. Sagar, J. Duan, S. Vasudevan, Y. Zhou, H. B. Amor, D. Fox, and R. Senanayake. RoboMD: Uncovering robot vulnerabilities through semantic potential fields. *arXiv preprint arXiv:2412.02818*, 2024.

[46] U. B. Karli and T. Fitzgerald. RECALL: Recovery experience collection for active lifelong learning in vision-language-action models. *arXiv preprint arXiv:2606.23617*, 2026.

[47] G. Zhao, Z. Tang, X. Chen, Z. Kuang, Y. Tian, and G. Li. FLARE: A failure-aware framework for autonomous correction and recovery in visual-language robotic manipulation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[48] H. Hao, S. N. Syed, J. Ichnowski, and J. Schneider. FAR: Failure-aware retry for test-time recovery and continual policy improvement. *arXiv preprint arXiv:2607.01111*, 2026.

[49] S. Huang, Y. Liao, S. Feng, S. Jiang, S. Liu, H. Li, M. Yao, and G. Ren. Adversarial data collection: Human-collaborative perturbations for efficient and robust robotic imitation learning. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025.

[50] M. Hou, K. Hindriks, A. E. Eiben, and K. Baraka. Active robot curriculum learning from online human demonstrations. *arXiv preprint arXiv:2503.02277*, 2025.

[51] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine. IDQL: Implicit Q-learning as an actor-critic method with diffusion policies. *arXiv preprint arXiv:2304.10573*, 2023.

[52] I. Kostrikov, A. Nair, and S. Levine. Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations (ICLR)*, 2022.

[53] Q. Li, Z. Zhou, and S. Levine. Reinforcement learning with action chunking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[54] L. Bailey, K. Wen, K. Dong, T. Hashimoto, and T. Ma. Scaling self-play with self-guidance. *arXiv preprint arXiv:2604.20209*, 2026.

[55] I. M. Sobol'. On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics*, 7(4):86–112, 1967. Russian original: Zh. Vychisl. Mat. i Mat. Phys., 7:784–802.

[56] A. B. Owen. Scrambling Sobol and Niederreiter-Xing points. *Journal of Complexity*, 14(4): 466–489, 1998.

[57] S. Joe and F. Y. Kuo. Constructing Sobol sequences with better two-dimensional projections. *SIAM Journal on Scientific Computing*, 30(5):2635–2654, 2008.

[58] T. F. Gonzalez. Clustering to minimize the maximum intercluster distance. *Theoretical Computer Science*, 38:293–306, 1985.

[59] Y. Eldar, M. Lindenbaum, M. Porat, and Y. Y. Zeevi. The farthest point strategy for progressive image sampling. *IEEE Transactions on Image Processing*, 6(9):1305–1315, 1997.

[60] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, S. Nasiriany, and Y. Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020.

[61] D. A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1988.

[62] S. Mirchandani, S. Belkhale, J. Hejna, E. Choi, M. S. Islam, and D. Sadigh. So you think you can scale up autonomous robot data collection? In *Conference on Robot Learning (CoRL)*, 2024.

[63] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[64] H. Kress-Gazit, K. Hashimoto, N. Kuppuswamy, P. Shah, P. Horgan, G. Richardson, S. Feng, and B. Burchfiel. Robot learning as an empirical science: Best practices for policy evaluation. *arXiv preprint arXiv:2409.09491*, 2024.

[65] A. Wald. Tests of statistical hypotheses concerning several parameters when the number of observations is large. *Transactions of the American Mathematical Society*, 54(3):426–482, 1943.

[66] L. D. Brown, T. T. Cai, and A. DasGupta. Interval estimation for a binomial proportion. *Statistical Science*, 16(2):101–133, 2001.

[67] Q. McNemar. Note on the sampling error of the difference between correlated proportions. *Psychometrika*, 12(2):153–157, 1947.

[68] B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1): 1–26, 1979.

[69] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

[70] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[71] X. Chen, C. Wang, Z. Zhou, and K. Ross. Randomized ensembled double Q-learning: Learning fast without a model. *arXiv preprint arXiv:2101.05982*, 2021.

[72] C. Parés-Morlans, N. Kuhn, I. Liu, A. Longhini, and J. Bohg. Demystifying when and why VLAs fail in contact-rich tasks and how to fix them. *arXiv preprint arXiv:2608.01402*, 2026.

[73] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[74] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[75] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009.

[76] J. Peters and S. Schaal. Reinforcement learning of motor skills with policy gradients. *Neural Networks*, 21(4):682–697, 2008.

[77] J. Kober and J. Peters. Policy search for motor primitives in robotics. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 21, 2009.

[78] S. Levine, C. Finn, T. Darrell, and P. Abbeel. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39):1–40, 2016.

[79] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, and S. Levine. QT-Opt: Scalable deep reinforcement learning for vision-based robotic manipulation. In *Conference on Robot Learning (CoRL)*, 2018.

[80] Y. Seo, J. Uruç, and S. James. Continuous control with coarse-to-fine reinforcement learning. *arXiv preprint arXiv:2407.07787*, 2024.

[81] H. Hu, S. Mirchandani, and D. Sadigh. Imitation bootstrapped reinforcement learning. *arXiv preprint arXiv:2311.02198*, 2023.

[82] T. Lampe, A. Abdolmaleki, S. Bechtle, S. H. Huang, J. T. Springenberg, M. Bloesch, O. Groth, R. Hafner, T. Hertweck, M. Neunert, M. Wulfmeier, J. Zhang, F. Nori, N. Heess, and M. Riedmiller. Mastering stacking of diverse shapes with large-scale iterative reinforcement learning on real robots. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 7772–7779, 2024.

[83] J. Yang, M. Sobol Mark, B. Vu, A. Sharma, J. Bohg, and C. Finn. Robot fine-tuning made easy: Pre-training rewards and policies for autonomous real-world reinforcement learning. *arXiv preprint arXiv:2310.15145*, 2023.

[84] S. K. S. Ghasemipour, A. Wahid, J. Tompson, P. Sanketi, and I. Mordatch. Self-improving embodied foundation models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[85] J. Kober, J. A. Bagnell, and J. Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013.

[86] M. P. Deisenroth and C. E. Rasmussen. PILCO: A model-based and data-efficient approach to policy search. In *International Conference on Machine Learning (ICML)*, pages 465–472, 2011.

[87] M. P. Deisenroth, C. E. Rasmussen, and D. Fox. Learning to control a low-cost manipulator using data-efficient reinforcement learning. In *Robotics: Science and Systems (RSS)*, 2011.

[88] S. Gu, E. Holly, T. Lillicrap, and S. Levine. Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3389–3396, 2017.

[89] S. Levine, P. Pastor, A. Krizhevsky, J. Ibarz, and D. Quillen. Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International Journal of Robotics Research*, 37(4–5):421–436, 2018.

[90] D. Kalashnikov, J. Varley, Y. Chebotar, B. Swanson, R. Jonschkowski, C. Finn, S. Levine, and K. Hausman. MT-Opt: Continuous multi-task robotic reinforcement learning at scale. *arXiv preprint arXiv:2104.08212*, 2021.

[91] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.

[92] H. Zhu, A. Gupta, A. Rajeswaran, S. Levine, and V. Kumar. Dexterous manipulation with deep reinforcement learning: Efficient, general, and low-cost. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3651–3657, 2019.

[93] A. Nagabandi, K. Konolige, S. Levine, and V. Kumar. Deep dynamics models for learning dexterous manipulation. In *Conference on Robot Learning (CoRL)*, pages 1101–1112, 2020.

[94] H. Zhu, J. Yu, A. Gupta, D. Shah, K. Hartikainen, A. Singh, V. Kumar, and S. Levine. The ingredients of real-world robotic reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2020.

[95] P. Wu, A. Escontrela, D. Hafner, P. Abbeel, and K. Goldberg. DayDreamer: World models for physical robot learning. In *Conference on Robot Learning (CoRL)*, pages 2226–2240, 2023.

[96] A. Gupta, J. Yu, T. Z. Zhao, V. Kumar, A. Rovinsky, K. Xu, T. Devlin, and S. Levine. Reset-free reinforcement learning via multi-task learning: Learning dexterous manipulation behaviors without human intervention. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[97] Z. Hu, A. Rovinsky, J. Luo, V. Kumar, A. Gupta, and S. Levine. REBOOT: Reuse data for bootstrapping efficient real-world dexterous manipulation. In *Conference on Robot Learning (CoRL)*, 2023.

[98] A. Sharma, A. M. Ahmed, R. Ahmad, and C. Finn. Self-improving robots: End-to-end autonomous visuomotor reinforcement learning. In *Conference on Robot Learning (CoRL)*, pages 3292–3308, 2023.

[99] R. Mendonca, E. Panov, B. Bucher, J. Wang, and D. Pathak. Continuously improving mobile manipulation with autonomous real-world RL. In *Conference on Robot Learning (CoRL)*, 2024.

[100] K. Bousmalis, G. Vezzani, D. Rao, C. Devin, A. X. Lee, M. Bauza, T. Davchev, Y. Zhou, A. Gupta, A. Raju, A. Laurens, C. Fantacci, V. Dalibard, M. Zambelli, M. Martins, R. Pevceviciute, M. Blokzijl, M. Denil, N. Batchelor, T. Lampe, E. Parisotto, K. Żołna, S. Reed, S. G. Colmenarejo, J. Scholz, A. Abdolmaleki, O. Groth, J.-B. Regli, O. Sushkov, T. Rothörl, J. E. Chen, Y. Aytar, D. Barker, J. Ortiz, M. Riedmiller, J. T. Springenberg, R. Hadsell, F. Nori, and N. Heess. RoboCat: A self-improving generalist agent for robotic manipulation. *Transactions on Machine Learning Research*, 2023.

[101] S. Zhai, Q. Zhang, T. Zhang, F. Huang, H. Zhang, M. Zhou, S. Zhang, L. Liu, S. Lin, and J. Pang. A vision-language-action-critic model for robotic real-world reinforcement learning. *arXiv preprint arXiv:2509.15937*, 2025.

[102] Q. Li and S. Levine. Q-learning with adjoint matching. *arXiv preprint arXiv:2601.14234*, 2026.

[103] A. Z. Ren, J. Lidard, L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz. Diffusion policy policy optimization. *arXiv preprint arXiv:2409.00588*, 2024.

[104] D. McAllister, S. Ge, B. Yi, C. M. Kim, E. Weber, H. Choi, H. Feng, and A. Kanazawa. Flow matching policy gradients. *arXiv preprint arXiv:2507.21053*, 2025.

[105] S. Park, Q. Li, and S. Levine. Flow Q-learning. *arXiv preprint arXiv:2502.02538*, 2025.

[106] X. Yuan, T. Mu, S. Tao, Y. Fang, M. Zhang, and H. Su. Policy decorator: Model-agnostic online refinement for large policy model. *arXiv preprint arXiv:2412.13630*, 2024.

[107] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. *arXiv preprint arXiv:2506.15799*, 2025.

[108] M. Nakamoto, O. Mees, A. Kumar, and S. Levine. Steering your generalists: Improving robotic foundation models via value guidance. *arXiv preprint arXiv:2410.13816*, 2024.

[109] M. Nakamoto, S. Zhai, A. Singh, M. Sobol Mark, Y. Ma, C. Finn, A. Kumar, and S. Levine. Cal-QL: Calibrated offline RL pre-training for efficient online fine-tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[110] M. Sobol Mark, T. Gao, G. G. Sampaio, M. K. Srirama, A. Sharma, C. Finn, and A. Kumar. Policy agnostic RL: Offline RL and online RL fine-tuning of any class and backbone. *arXiv preprint arXiv:2412.06685*, 2024.

Appendix contents

A Author Contributions **18**

B Extended Results **19**

B.1 Full per-arm success rates across rounds . . . 19

B.2 Headline comparisons and statistical evidence . . . 19

B.3 Real-world execution speed and throughput . . . 21

B.4 Simulation execution speed and throughput . . . 23

B.5 Sampling-strategy and actor-data ablations . . . 24

B.6 Counterfactual-demonstration ablation on INSERT MARKER and THREAD NUT . . . 24

B.7 Per-stage decomposition of the real-world tasks . . . 25

B.8 Collection success: unassisted comparison and all real-world tasks . . . 27

B.9 Collection burden on all real-world tasks . . . 29

B.10 Training-data composition by round . . . 29

C Task and Environment Details **32**

C.1 Platforms and tasks . . . 32

C.2 Initial-state ranges and difficulty . . . 33

C.3 Success criteria and stage ladders . . . 33

D Evaluation Protocol Details **36**

E Data and Collection Protocol **38**

E.1 The four data streams and how frames are routed . . . 38

E.2 Round budgets and dataset ledger . . . 38

E.3 Counterfactual-demonstration protocol . . . 38

E.4 Reset-helper schematics for real-world collection . . . 40

F Initial-State Sampling Details **41**

F.1 Difficulty varies sharply across the initial-state space . . . 41

F.2 Sobol vs. uniform coverage . . . 41

F.3 Sobol sequence configuration . . . 41

F.4 The shared sampler . . . 42

F.5 Hardness signal: simulation . . . 42

F.6 Hardness signal: real world . . . 44

F.7 Budget split and coverage guardrails . . . 44

G Actor and Critic Implementation **46**

G.1 Actor . . . 46

G.2 Critic . . . 46

G.3 Action-magnitude modulation in the critic . . . 47

G.4 Best-of-N reranking and inference cost . . . 47

G.5 Simulation . . . 48

G.6 Real world . . . 48

G.7 Compute and training budgets . . . 48

H Hyperparameters **49**

H.1 Actor . . . 49
H.2 Critic . . . 49
H.3 Best-of-N reranking . . . 49
H.4 Initial-state sampler . . . 49
H.5 Environment, control, and the improvement loop . . . 49

I Baseline Details **55**

I.1 HG-DAgger . . . 55
I.2 Autonomous self-improvement arms . . . 55
I.3 RECAP learner comparison . . . 55
I.4 HiL-SERL in simulation . . . 55

J Value-Learning Diagnostics and Ablations **58**

J.1 Real-world value learning . . . 58
J.2 Critic training-data composition . . . 59
J.3 Critic-objective variations . . . 60
J.4 Distributional value head (DIVL) vs. scalar IQL critic . . . 62
J.5 Critic ensemble size (REDQ) . . . 63
J.6 Reranking budget N . . . 63

K Extended Related Work **64**

K.1 Real-world RL for robot manipulation . . . 64
K.2 RL finetuning of expressive policies . . . 64
K.3 Initial-state randomization: ours and prior real-world RL . . . 65
K.4 Data and interaction budgets: ours and prior real-world RL . . . 66

A Author Contributions

LA conceived the project, analyzed how the success rate varies across initial states, designed **Mulligan** and HiL-IDQL, and built the simulation, teleoperation, real-robot, training, and evaluation infrastructure. LA collected most of the demonstrations and corrections, ran real-world evaluations, implemented, ran, and analyzed the experiments and baselines, and wrote the paper. RB helped collect real-world data, ran real-robot evaluations, helped conceive the state-conditioned action scaling in the critic, contributed to the abstract and introduction, provided writing feedback, and suggested the name **Mulligan**. PD provided input on the algorithm, experimental design, and early paper structure and framing. AM took part in technical discussions, helped run evaluations, advised on the HiL-SERL baseline, contributed to the related-work framing, and provided writing feedback. DY built the stage-labeling and human-review tools behind the stage-wise analysis, helped run real-robot evaluations, and provided writing feedback. SS advised the project's early development, suggested batch rather than online data collection, proposed training the actor on successful human data and the critic on all data, and provided writing feedback. CF supervised the project and helped develop the idea of focusing data collection on the initial states where the policy fails. CF also suggested IDQL for value learning and guided the choice of tasks, baselines, and experiments. CF helped set the paper's framing, including by drafting an early version of the abstract, and provided extensive writing feedback.

B Extended Results

We expand the main results on held-out performance and throughput, component contributions, and collection success and human burden, in Q1–Q3 order. Binary real-world curves use Wilson one-standard-error intervals (z=1). Paired comparisons report 95% intervals, and other metrics specify their uncertainty in the captions. Task definitions and evaluation details are in Apps. C and D.

B.1 Full per-arm success rates across rounds

Referenced from Sec. 4.1.

Fig. 4 splits the main headline (Fig. 2) into its three arms. Here we use full-episode success for ROUTE CABLE, which requires both clips, while the main headline reports the mean clip score divided by two.

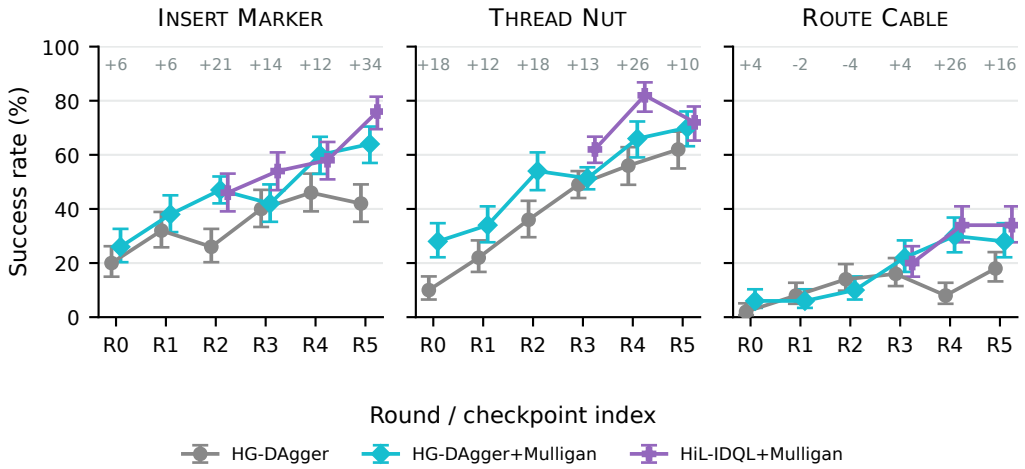


Figure 4: Per-arm full-success results for the policies in Fig. 2, with HG-DAgger (grey), HG-DAgger+**Mulligan** (teal), and HiL-IDQL+**Mulligan** with critic reranking (purple) as separate series. The top row gives the per-round delta of the policy deployed in the **Mulligan** campaign over the uniform-sampling HG-DAgger baseline in percentage points. Intervals are Wilson-score one-standard-error (z=1). The ROUTE CABLE panel shows full-episode success for rounds R0–R5, scored on one shared block of 50 initial states.

B.2 Headline comparisons and statistical evidence

Referenced from Sec. 4.1.

The real-world tests behind Sec. 4.1 pair arms by initial state. In each block, we run that round's own actors on the same held-out Sobol states, blind and interleaved in one session (App. D).

Final-round comparisons. Tab. 1 gives the per-task final-round tests behind the headline claim of 10–34 percentage points above HG-DAgger. The final-checkpoint HG-DAgger+**Mulligan** to HiL-IDQL+**Mulligan** increment is positive on all three tasks but individually significant on none. Across the ROUTE CABLE rounds, the full-success reranking increment changes sign.

Round-pooled treatment effect. In Tab. 2, we pool the matched blocks of all rounds for the uniform HG-DAgger baseline and the actor-only HG-DAgger+**Mulligan** arm, sum within-block discordant pairs, and bootstrap the paired delta stratified within block. Each block compares contemporaneous actors, which makes the result a treatment effect pooled over rounds and not a single-checkpoint comparison. Performance-guided collection adds +11.7 percentage points on INSERT MARKER, +10.9 on THREAD NUT, and +6.0 on ROUTE CABLE. Rescues outnumber regressions

Table 1: Final-round paired comparisons. We evaluated INSERT MARKER R5 in one contemporaneous interleaved session over a single Sobol-50 block, and THREAD NUT R5 in the final three-arm session on 50 fresh Sobol initial states (n=50 per arm) with the selected FiLM critic. On ROUTE CABLE, R5 is the 600-episode round, and we score both-clip success on all 50 initial states of the shared block. We report paired-bootstrap intervals and exact McNemar p-values. "vs. base" contrasts an arm with that task's uniform HG-DAgger arm, and "vs. actor" with HG-DAgger+**Mulligan**. Dashes mark contrasts we do not report for that round.

Task	Arm	Success	Contrast	Δ [CI], p
INSERT MARKER R5	HG-DAgger	21/50 (42%)	—	—
	HG-DAgger+ Mulligan	32/50 (64%)	vs. base	+22 pp [+4, +40], $\mathbf{p = 0.0347}$
	HiL-IDQL+ Mulligan	38/50 (76%)	vs. base	+34 pp [+16, +52], $\mathbf{p = 0.0015}$
			vs. actor	+12 pp [0, +26], $p = 0.146$
THREAD NUT R5	HG-DAgger	31/50 (62%)	—	—
	HG-DAgger+ Mulligan	35/50 (70%)	vs. base	+8 pp [−8, +24], $p = 0.454$
	HiL-IDQL+ Mulligan	36/50 (72%)	vs. base	+10 pp [−8, +26], $p = 0.359$
			vs. actor	+2 pp [−12, +14], $p = 1.000$
ROUTE CABLE R5	HG-DAgger	9/50 (18%)	—	—
	HG-DAgger+ Mulligan	14/50 (28%)	vs. base	+10 pp [−4, +24], $p = 0.302$
	HiL-IDQL+ Mulligan	17/50 (34%)	vs. base	+16 pp [+2, +30], $p = 0.0768$
			vs. actor	+6 pp [−10, +22], $p = 0.629$

on every task (78 to 43 on INSERT MARKER, 80 to 42 on THREAD NUT, 42 to 24 on ROUTE CABLE). On THREAD NUT, we count the final three-arm R5 session once per initial state. Superseded R5 evaluations enter neither the headline nor the pooled comparison.

Table 2: Round-pooled paired comparison of uniform HG-DAgger against HG-DAgger+**Mulligan**. We compute exact McNemar p-values on the summed discordant pairs and bootstrap the intervals stratified within block. All six ROUTE CABLE blocks use the same 50 initial states.

Task	Paired n	HG-DAgger	HG-DAgger+ Mulligan	Paired Δ [95% CI]	Exact McNemar p
INSERT MARKER	300	103 (34.3%)	138 (46.0%)	+11.7 pp [+5, +19]	**0.0019**
THREAD NUT	350	142 (40.6%)	180 (51.4%)	+10.9 pp [+5, +17]	**0.0007**
ROUTE CABLE	300	33 (11.0%)	51 (17.0%)	+6.0 pp [+1, +11]	**0.0356**

ROUTE CABLE round comparisons. On ROUTE CABLE, we evaluated 15 policies on the same 50 initial states, 750 rollouts in all (App. D). Tab. 3 reports every round, R0–R5 with 100–600 training episodes per arm, in both full-episode success and the unnormalized clip score.

Table 3: ROUTE CABLE results per round on the shared block of 50 initial states. A success count requires both clips seated, and progress is the mean 0–2 clip score. Triplets list HG-DAgger / HG-DAgger+**Mulligan** / HiL-IDQL+**Mulligan**, with a dash where we did not evaluate the reranked arm.

Round	Training episodes / arm	Full successes	Mean clip score
R0	100	1/3/—	0.22/0.40/—
R1	200	4/3/—	0.58/0.44/—
R2	300	7/5/—	0.56/0.62/—
R3	400	8/11/10	0.70/0.82/0.88
R4	500	4/15/17	0.40/1.12/1.10
R5	600	9/14/17	0.74/0.98/1.00

HG-DAgger+**Mulligan** outperforms HG-DAgger in mean clip score at every round except R1.

Simulation. Tab. 4 gives the per-round contrasts behind the simulation panels of Fig. 2. We train five seeds per arm and score each on the same fixed held-out grid (8,000 episodes on SQUARE-NARROW, 30,000 on SQUARE-BROAD). Treating the seed as the unit of analysis, we report Welch-t differences of seed means with 95% intervals. Performance-guided collection beats uniform collection at every round on both tasks, with or without critic reranking. On SQUARE-NARROW the margin shrinks as both arms near the ceiling. On SQUARE-BROAD it peaks at R2. The full recipe beats HG-DAgger at every round except SQUARE-BROAD R0. There the two arms differ only in Sobol versus uniform demonstrations, and reranking has no on-policy data yet.

Table 4: Simulation per-round contrasts in percentage points of held-out success: Welch-t difference of five-seed means with 95% intervals and two-sided p-values. Actor-only arms use the headline evaluation of each seed's final checkpoint. Critic-reranked arms (N=32) use the DIVL critics of App. J.4, which are the N=32 numbers plotted in the main paper.

Contrast	R0	R1	R2	R3
SQUARE-NARROW				
HG-DAgger+**Mulligan**	+8.4 [+5.5, +11.3]	+8.5 [+5.3, +11.8]	+3.6 [+2.5, +4.7]	+1.8 [+0.3, +3.3]
vs. HG-DAgger	$\mathbf{p < 0.001}$	$\mathbf{p < 0.001}$	$\mathbf{p < 0.001}$	$\mathbf{p = 0.028}$
HiL-IDQL+**Mulligan**	+8.2 [+5.2, +11.2]	+7.8 [+4.6, +11.0]	+3.0 [+2.0, +4.1]	+1.5 [+0.5, +2.5]
vs. HiL-IDQL	$\mathbf{p < 0.001}$	$\mathbf{p = 0.001}$	$\mathbf{p < 0.001}$	$\mathbf{p = 0.007}$
HiL-IDQL+**Mulligan**	+5.9 [+3.1, +8.7]	+8.7 [+5.4, +11.9]	+3.5 [+2.3, +4.7]	+2.5 [+1.0, +4.1]
vs. HG-DAgger	$\mathbf{p = 0.001}$	$\mathbf{p < 0.001}$	$\mathbf{p < 0.001}$	$\mathbf{p = 0.006}$
SQUARE-BROAD				
HG-DAgger+**Mulligan**	+3.3 [+1.0, +5.6]	+8.5 [+6.4, +10.5]	+11.1 [+7.9, +14.3]	+7.6 [+2.7, +12.4]
vs. HG-DAgger	$\mathbf{p = 0.011}$	$\mathbf{p < 0.001}$	$\mathbf{p < 0.001}$	$\mathbf{p = 0.009}$
HiL-IDQL+**Mulligan**	+3.5 [+1.1, +6.0]	+8.5 [+6.1, +10.9]	+11.8 [+9.0, +14.5]	+7.8 [+3.1, +12.5]
vs. HiL-IDQL	$\mathbf{p = 0.011}$	$\mathbf{p < 0.001}$	$\mathbf{p < 0.001}$	$\mathbf{p = 0.007}$
HiL-IDQL+**Mulligan**	+1.7 [−0.9, +4.2]	+7.4 [+5.0, +9.8]	+11.3 [+8.1, +14.5]	+7.9 [+3.0, +12.7]
vs. HG-DAgger	$p = 0.17$	$\mathbf{p < 0.001}$	$\mathbf{p < 0.001}$	$\mathbf{p = 0.008}$

B.3 Real-world execution speed and throughput

Referenced from Sec. 4.1.

These gains do not come at the expense of execution speed. HG-DAgger has been reported to trade speed for success [13], whereas Fig. 5 shows HiL-IDQL+**Mulligan** completing more tasks per robot minute than HG-DAgger on all three real tasks (time-to-completion in Fig. 6, simulation throughput in App. B.4).

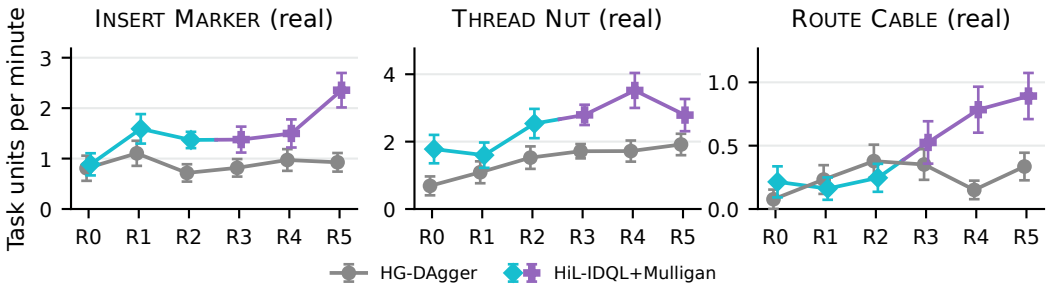


Figure 5: Completed tasks per robot minute, counting time on failed episodes. A completed ROUTE CABLE task requires both clips seated. Colors follow Fig. 2, with ±1 bootstrap SE whiskers.

Fig. 6 reports execution speed on the real-world held-out evaluations behind Fig. 5. Fig. 7 shows all three policy arms behind Fig. 5. For ROUTE CABLE, we use full-episode completion times at 15 Hz. On INSERT MARKER, the reranked arm completes successful episodes faster than the actor-only arm in the final round. Unlike completion time, throughput also charges time spent on failures.

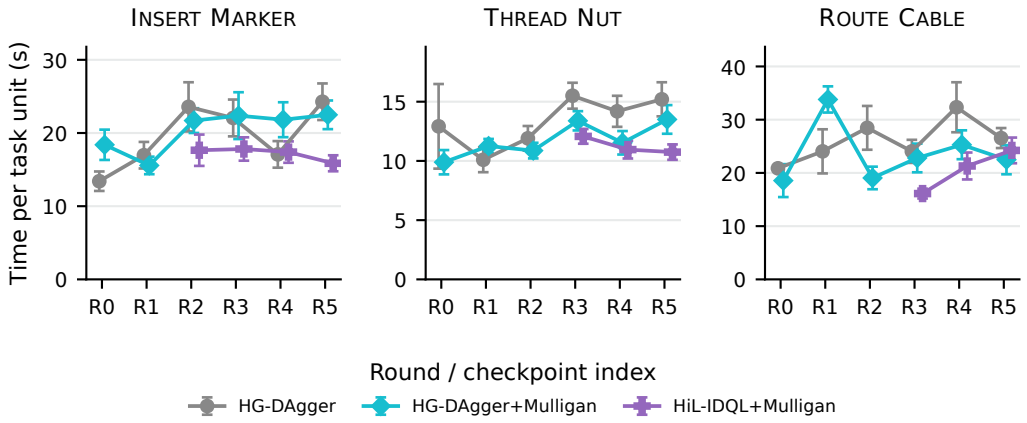


Figure 6: Execution speed on the held-out evaluations: mean robot time per completed task unit (one standard error over completed units), where a unit is a full task completion on every task, requiring both clips on ROUTE CABLE. ROUTE CABLE uses all six rounds on its shared block of 50 initial states. Failed episodes are excluded from this duration metric.

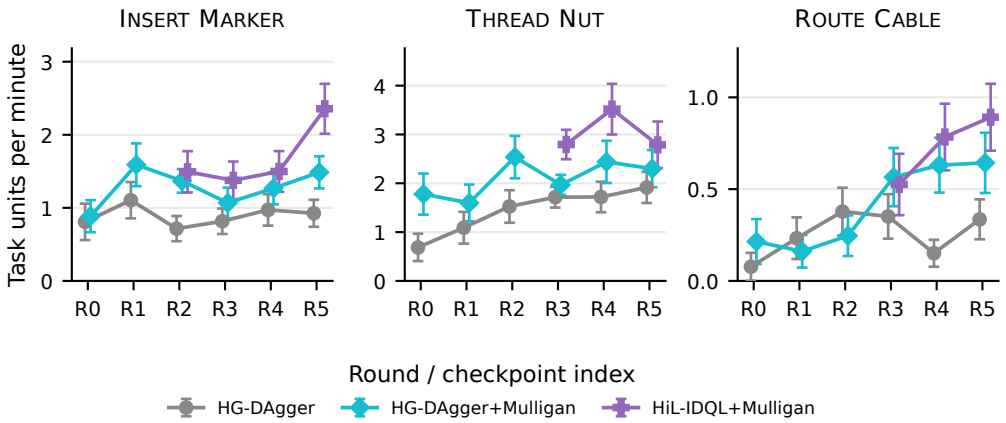


Figure 7: Throughput on the held-out evaluations: completed task units per robot minute over *all* evaluation episode time (one bootstrap standard error over evaluation episodes). A task unit is a full episode on all three tasks, and ROUTE CABLE uses all six rounds on its shared block of 50 initial states. Unlike the summary in Fig. 5, this view shows HG-DAgger+**Mulligan** and HiL-IDQL+**Mulligan** together wherever both were evaluated. Failed and timed-out episodes spend robot time but produce no units.

B.4 Simulation execution speed and throughput

Referenced from App. B.3.

Figs. 8 and 9 extend the real-world execution-speed and throughput analysis (Fig. 6 and Fig. 5) to the simulation campaigns behind Fig. 2. The reranked supervised arms use DIVL critics, and the autonomous arms use scalar-IQL critics (App. G.5). As Fig. 8 shows, successful-rollout durations move by less than a second across rounds while success rates climb by tens of points, so the throughput gains in Fig. 9 track reliability. The two **Mulligan** collection arms have the highest units per minute at every round on both tasks, except that HiL-IDQL passes HG-DAgger+**Mulligan** at SQUARE-NARROW R3. The autonomous-deployment arms improve more slowly, and the best of them (batch-online IDQL) remains below every supervised arm at R3.

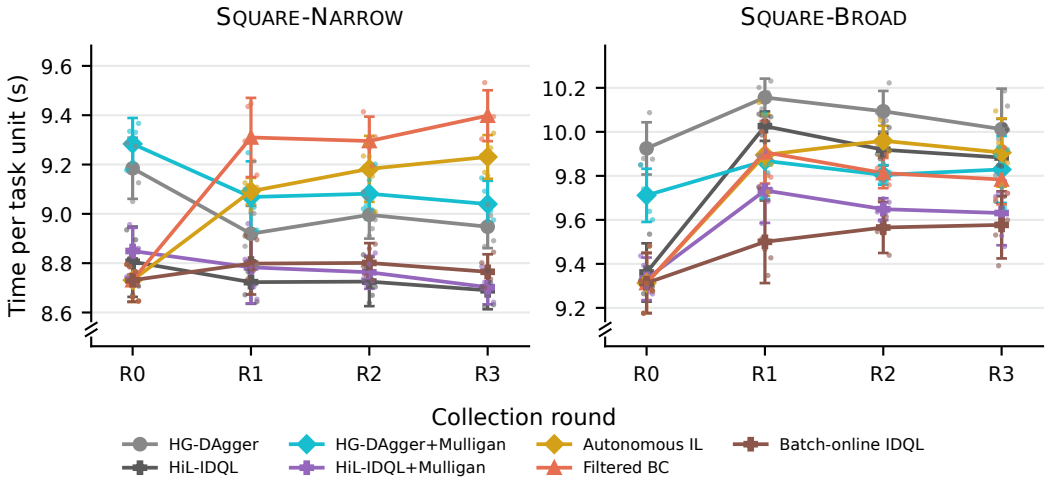


Figure 8: Simulation execution speed on the fixed-grid held-out evaluations for SQUARE-NARROW (left) and SQUARE-BROAD (right): mean successful-rollout duration in seconds (env control steps at 20 Hz), averaged over the five training seeds (two-sided Student-t 95% CIs, dots are per-seed means, y-axes are zoomed at the axis break). Series colors and markers follow Fig. 2.

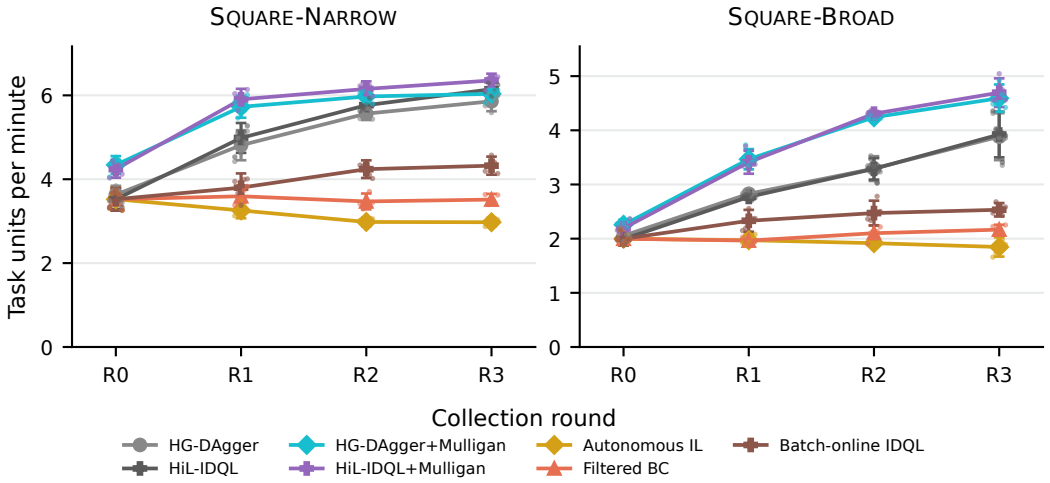


Figure 9: Throughput on the simulation fixed-grid held-out evaluations for SQUARE-NARROW (left) and SQUARE-BROAD (right): successful rollouts per robot-minute over *all* evaluation episode time (two-sided Student-t 95% CIs over five seeds, dots are per-seed values, series and marker conventions as in Fig. 8). Failed episodes spend robot time but produce no completions, so this metric combines reliability and speed, as in Fig. 5.

B.5 Sampling-strategy and actor-data ablations

Referenced from Sec. 4.2.

Tab. 5 gives the numbers behind the two simulation panels of Fig. 3. All six cells share the round-0 dataset, the round-1 collection budget, and the training recipe, so each row differs from the reference row only in what its label lists. The human-only rows move the sampler from uniform to Sobol to performance-guided and then add counterfactuals, and the second row ablates the actor's training set. When we add successful autonomous rollouts to the baseline actor's training mixture, the actor degrades.

Table 5: Sampling-strategy and actor-data ablation on SQUARE-NARROW round 1. Entries are fixed-grid held-out success rates with $N{=}32$ DIVL reranking, mean $\pm$ standard error over five seeds, and Δ compares each row with the uniform reference (Welch-t 95% intervals). "HO" is a human-only actor, and "+auto successes" adds successful autonomous rollouts.

Collection strategy / actor data	Grid SR (%)	Δ [95% CI]
Uniform, no-CF, HO (reference)	85.73 ± 1.16	—
Uniform, no-CF, +auto successes	82.37 ± 0.52	$-3.36\ [-6.53, -0.18]$
Sobol coverage, no-CF, HO	89.59 ± 0.45	$+3.86\ [+0.69, +7.03]$
Sobol coverage, with-CF, HO	91.49 ± 0.61	$+5.76\ [+2.55, +8.96]$
Performance-guided ($\beta{=}0.5$), no-CF, HO	92.69 ± 0.36	$+6.96\ [+3.78, +10.13]$
Performance-guided ($\beta{=}0.5$), with-CF, HO	93.52 ± 0.52	$+7.80\ [+4.62, +10.97]$

At round 0, we collect each task's demonstration budget once with uniform and once with Sobol sampling. Sobol initial states raise DIVL-reranked success on SQUARE-NARROW from 70.80 ± 0.97 to 79.03 ± 0.86 ($+8.23$ pp $[+5.24, +11.22]$) and on SQUARE-BROAD from 49.03 ± 0.74 to 52.56 ± 0.77 ($+3.53$ pp $[+1.07, +5.98]$). We evaluate with $N{=}32$ on the grid over five seeds (mean $\pm$ SE, Welch-t 95% intervals). On SQUARE-NARROW, the headline HiL-IDQL+**Mulligan** sequence then runs $79.03 \rightarrow 93.52 \rightarrow 95.28 \rightarrow 96.43$ across R0–R3.

B.6 Counterfactual-demonstration ablation on INSERT MARKER and THREAD NUT

Referenced from Sec. 4.2.

Fig. 10 enlarges the INSERT MARKER and THREAD NUT counterfactual-demonstration comparisons shown in Fig. 3. Most points use 50 held-out episodes. The with-CF THREAD NUT R3 point pools 150 episodes of the same checkpoint, and the with-CF INSERT MARKER R2 point pools 100 episodes.

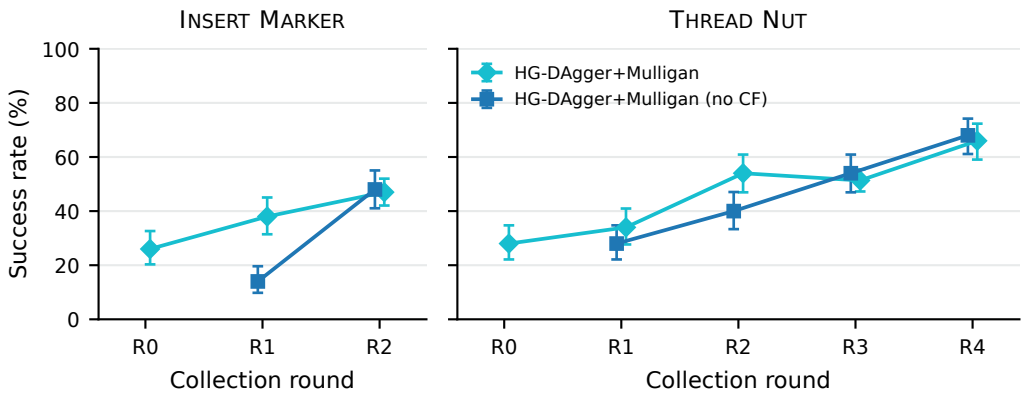


Figure 10: Counterfactual-demonstration ablation on INSERT MARKER (left) and THREAD NUT (right): held-out success rate over DAgger rounds for HG-DAgger+**Mulligan** with and without CF demos, with Wilson-score one-standard-error intervals (z=1). Each panel ends at the last round with a no-CF evaluation (INSERT MARKER R2, THREAD NUT R4). R0 is the shared source actor, shown once. Most points use 50 episodes, and the with-CF INSERT MARKER R2 and THREAD NUT R3 points pool 100 and 150 episodes. Removing counterfactuals lowers early-round success more on INSERT MARKER than on THREAD NUT, and on both tasks the arms converge as corrections accumulate.

B.7 Per-stage decomposition of the real-world tasks

Referenced from Sec. 4.2.

We decompose each INSERT MARKER and THREAD NUT episode into grasp, insertion, and overall-success stages (Figs. 11 and 12). On THREAD NUT, performance-guided collection improves early grasp success and the gap narrows in later rounds. On INSERT MARKER, a grasp-stage advantage remains at R5. The critic improves conditional insertion success at several checkpoints, most clearly on INSERT MARKER R5 and THREAD NUT R3–R4. The THREAD NUT R5 stage-labeled blocks show no reranking gain. Stage labels come from the labeler described in App. C.3. We use each arm's available stage-labeled blocks, which can be a subset of the binary-outcome evaluations used in the headline.

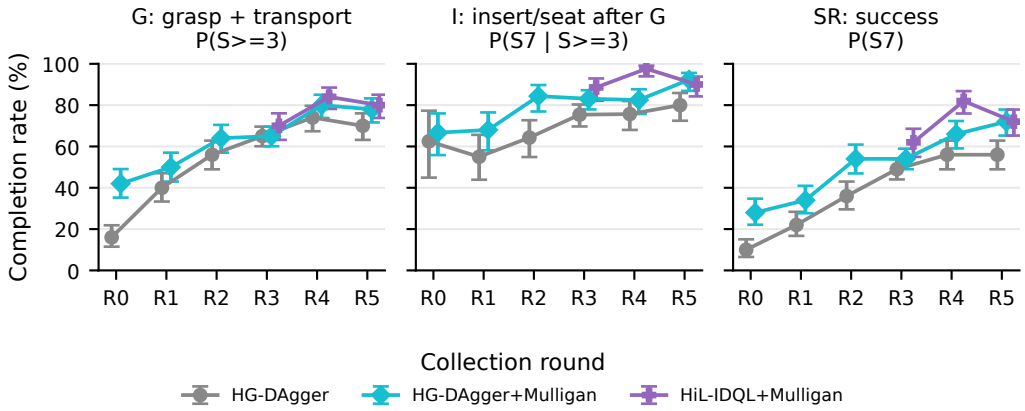


Figure 11: Per-stage decomposition on THREAD NUT across rounds: grasp+transport (left), insertion/seat conditioned on grasp (middle), and overall success (right), with Wilson-score one-standard-error intervals (z=1). The grasp-stage gap narrows in later rounds, and the critic improves conditional insertion success at R3–R4. At R5, HG-DAgger+**Mulligan** and HiL-IDQL+**Mulligan** each have $36/50$ stage successes. The R5 HG-DAgger+**Mulligan** labels come from an earlier three-arm session and the HiL-IDQL+**Mulligan** labels from a candidate-critic evaluation, as the final R5 evaluation has no stage labels.

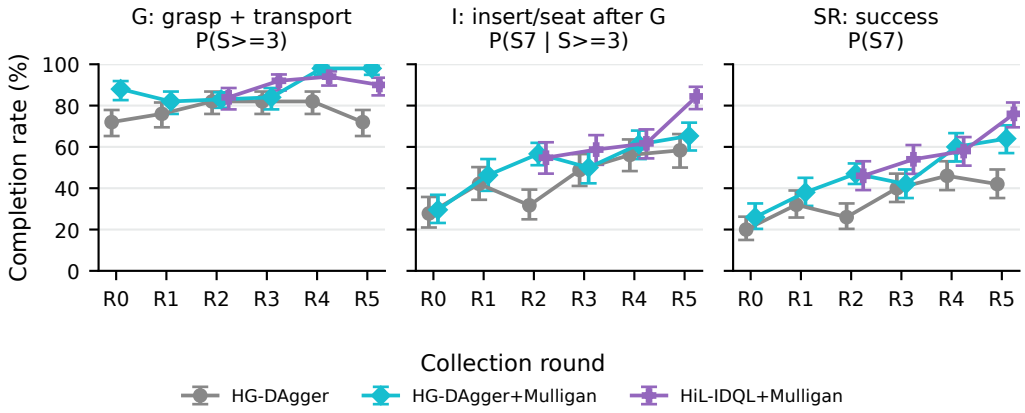


Figure 12: Per-stage decomposition on INSERT MARKER across rounds: grasp+transport (left), insertion/seat conditioned on grasp (middle), and overall success (right), with Wilson-score one-standard-error intervals (z=1). The collection arm retains a grasp-stage advantage at R5, while reranking improves conditional insertion success. R5 uses the stage labels of the headline three-arm session.

B.8 Collection success: unassisted comparison and all real-world tasks

Referenced from Sec. 4.3.

Fig. 13 shows collection success on all three real tasks. The solid series counts successful fresh collection episodes, excluding CF replay. The remaining failures include episodes in which the operator's own intervention failed. The dashed series is held-out episode success of the policy that collected that round: collection round k deploys the round $k-1$ checkpoint (on ROUTE CABLE R1–R4, the earlier Cartesian-velocity checkpoints), including the reranked collectors when they were used. Whiskers are Wilson one-standard-error intervals ($z=1$).

Collection and held-out evaluation use different initial states, so these curves are an observational comparison. On ROUTE CABLE the rounds are the paper's 100-episode increments (App. E), and the collector series pools the contemporaneous held-out evaluations of each increment's sessions. Tab. 6 lists the counts behind that panel, where collection success measures full-episode completion rather than the headline clip score. The R4 dip on ROUTE CABLE comes from 11 fresh attempts beyond the credited quota that ended in unrecoverable failures.

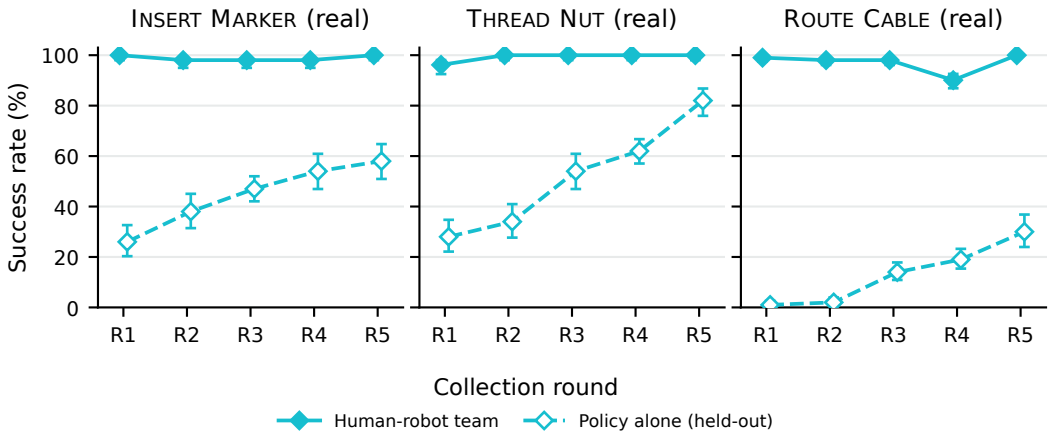


Figure 13: Collection success on INSERT MARKER, THREAD NUT, and ROUTE CABLE. The human-robot team completes 98% of fresh collection episodes, with no round below 90% (solid), while the collector policy's preceding-round held-out success is lower (dashed). Whiskers are Wilson-score one-standard-error intervals ($z=1$). CF replay episodes are excluded, and ROUTE CABLE rounds are the 100-episode increments of Tab. 6.

Fig. 14 compares the uniform-sampling and **Mulligan** collection campaigns in the team-only view.

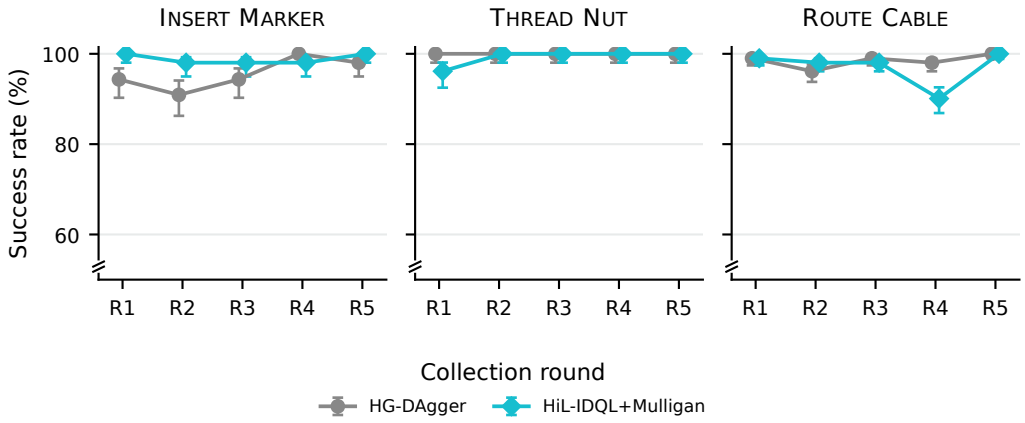


Figure 14: Human-robot team success rate during DAgger collection, per round (Wilson-score one-standard-error intervals with $z{=}1$, y-axis truncated at 50%), on INSERT MARKER (left), THREAD NUT (middle), and ROUTE CABLE (right, 100-episode increments), for the uniform-sampling (grey, HG-DAgger) and **Mulligan** (teal, HiL-IDQL+**Mulligan**) collection campaigns. Over all rounds, the **Mulligan** campaign completes 98% of collection episodes with no round below 90%, and the uniform-sampling campaign completes 98% with no round below 91%.

Table 6: ROUTE CABLE collection on the paper's 100-episode increments, both HG-DAgger arms. "Fresh / CF" splits each increment's credited 100 episodes into episodes that start under policy control and counterfactual replays that the operator demonstrates from the first frame. Collection success and human share count only the policy-initiated episodes. Success is successful fresh episodes over fresh attempts, and human share is the fraction of their valid frames under human control. The comparison is descriptive, since collector policies and sampled initial states change across increments. At R5, the HG-DAgger+**Mulligan** operator stopped fresh collection at the 88 episodes the increment credits, rather than the 100 of earlier rounds, because the R5 dataset without counterfactuals would not be used. R5 collection success is out of 88 attempts.

Round	Arm	Fresh / CF	Collection success	Human share
R1	HG-DAgger	100 / 0	100/101	43.6%
	HG-DAgger+**Mulligan**	87 / 13	100/101	44.2%
R2	HG-DAgger	100 / 0	100/104	44.0%
	HG-DAgger+**Mulligan**	76 / 24	100/102	41.6%
R3	HG-DAgger	100 / 0	100/101	38.9%
	HG-DAgger+**Mulligan**	89 / 11	100/102	38.0%
R4	HG-DAgger	100 / 0	100/102	32.3%
	HG-DAgger+**Mulligan**	90 / 10	100/111	35.7%
R5	HG-DAgger	100 / 0	100/100	35.1%
	HG-DAgger+**Mulligan**	88 / 12	88/88	39.3%

B.9 Collection burden on all real-world tasks

Referenced from Sec. 4.3.

In Fig. 15, the raw share of human-controlled frames (solid) is similar for both methods and generally declines across rounds, despite their different held-out success rates. Dividing this share by the collector's held-out failure rate $(1 - \mathrm{SR})$ gives a higher ratio for **Mulligan** on THREAD NUT (dashed). In simulation, guided sampling likewise shows lower raw but higher failure-adjusted intervention incidence. This is consistent with directing supervision toward difficult states, but the ratio also depends on intervention duration and on the failure-rate divisor, so it is not a direct measure of the sampled states' failure probability.

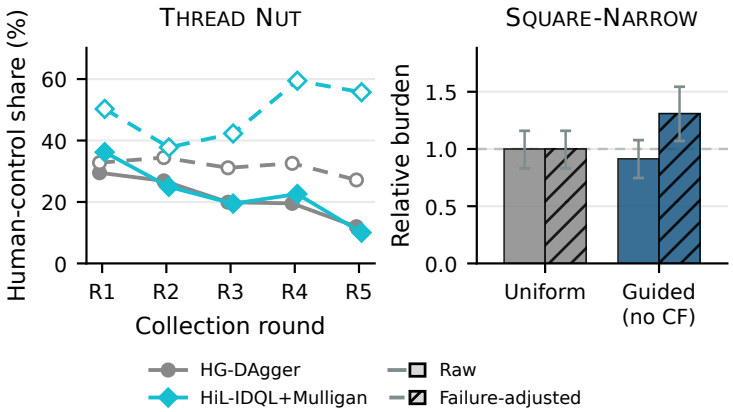


Figure 15: Human burden on THREAD NUT (left, human-control frame share) and SQUARE-NARROW (right, intervention-episode share relative to uniform), raw and divided by the collector's failure rate. Both panels exclude CF demos. Wilson 95% bounds treat the normalization as fixed (all real-world tasks in Fig. 16).

Fig. 16 shows the collection burden (Sec. 4.3) for all three real-world tasks on shared per-column scales. We count burden only on policy-initiated episodes and exclude counterfactual replays. On ROUTE CABLE, the collector success rate that normalizes the right column is the held-out full-task success of the checkpoint that collected each source round, weighted by fresh episodes over the two source rounds of an increment. On THREAD NUT, where the **Mulligan** campaign starts from harder states, the normalized burden is above HG-DAgger's from R3 on, as described above.

B.10 Training-data composition by round

Referenced from Sec. 4.0.

The uniform-sampling and **Mulligan** collection campaigns both start from 100 initial demonstrations and collect the same number of supervised episodes per round: 50 on INSERT MARKER and THREAD NUT and 100 on ROUTE CABLE. Fig. 17 and Tab. 7 split each round's training frames by source. Demonstrations are the human frames of the initial and counterfactual demonstrations, corrections are the human frames of episodes that the policy started, and autonomous frames are all policy-controlled frames, including those inside mixed-control counterfactual demonstrations. We count the valid frames, after outcome trimming, of the episodes that each campaign trains on. At the same episode budget, the **Mulligan** campaign's training data contain 19%, 28%, and 8% more human frames than the uniform-sampling campaign's on INSERT MARKER, THREAD NUT, and ROUTE CABLE. The difference comes from the counterfactual demonstrations, which fill part of each round's episode quota with full demonstrations.

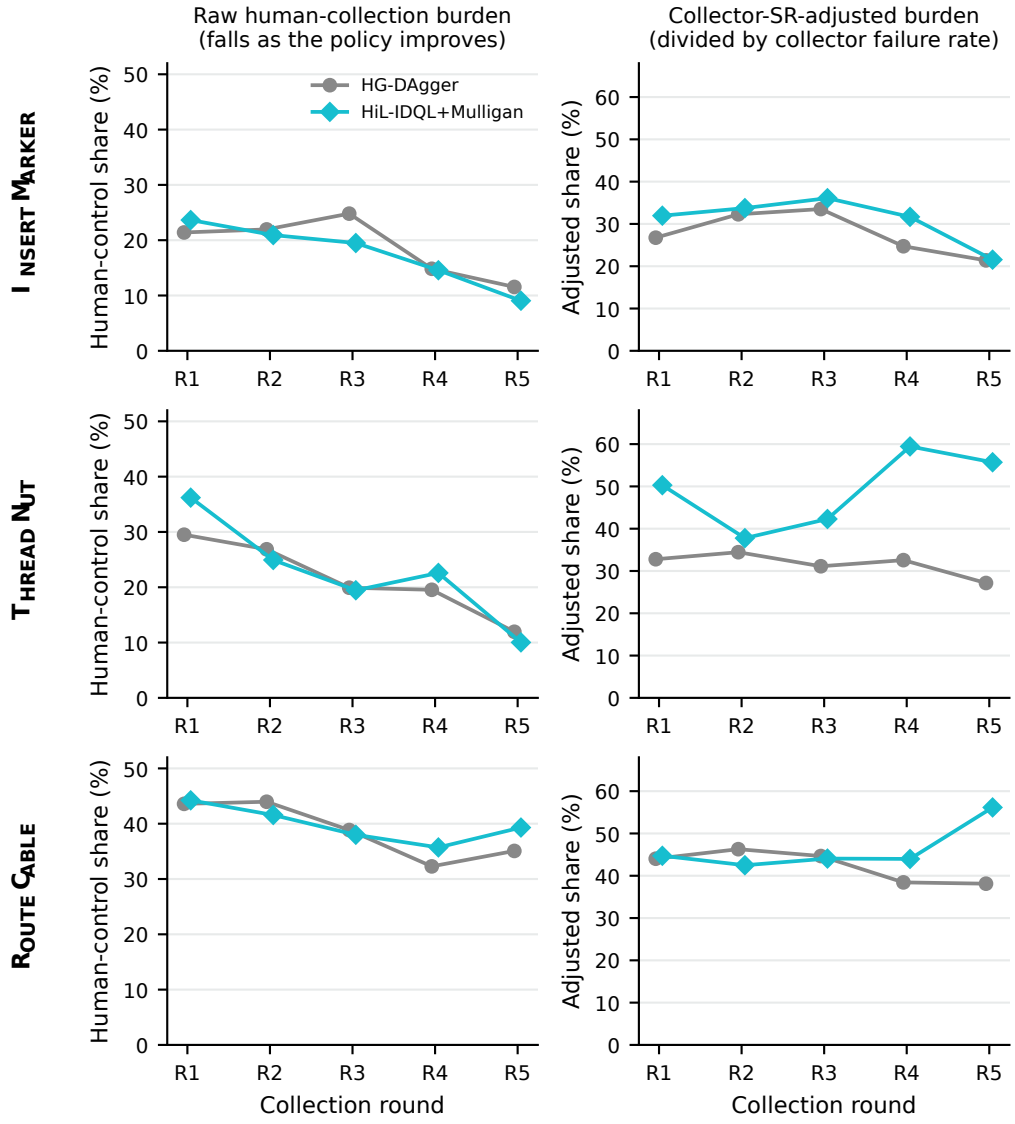


Figure 16: Human collection burden over DAgger rounds on INSERT MARKER (top), THREAD NUT (middle), and ROUTE CABLE (bottom, 100-episode increments, each normalized by the fresh-episode-weighted held-out success of the policies that collected its source rounds), for the uniform-sampling (grey, HG-DAgger) and **Mulligan** (teal, HiL-IDQL+**Mulligan**) collection campaigns. Left: raw burden (share of frames under human control in fresh collection episodes). Right: the same share normalized by the collection policy's failure rate $(1 - \mathrm{SR})$. On ROUTE CABLE, where complete success is still near the floor, the normalization is close to the identity.

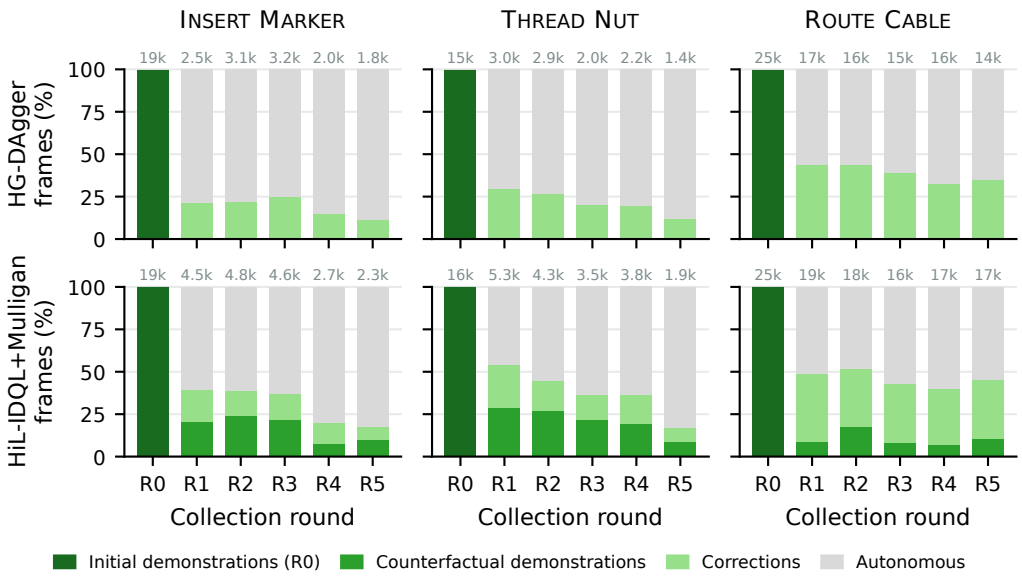


Figure 17: Composition of each round's training data for the uniform-sampling (top) and **Mulligan** (bottom) collection campaigns, as a share of valid frames. The number above each bar is the round's count of human-controlled frames. Both campaigns collect the same number of episodes per round. The uniform-sampling campaign's data train HG-DAgger, and the **Mulligan** campaign's data train both HG-DAgger+**Mulligan** and HiL-IDQL+**Mulligan**.

Table 7: Training frames per round and source for the uniform-sampling and **Mulligan** collection campaigns (Fig. 17). A dash marks a source that the round does not contain. Demonstrations plus corrections give the human frames, and the **Mulligan** totals equal $\mathcal{D}_h$ and $\mathcal{D}_{\text{all}}$ in Tab. 13.

		HG-DAgger			HiL-IDQL+**Mulligan**		
Task	Round	Demos	Corrections	Autonomous	Demos	Corrections	Autonomous
INSERT MARKER	R0	19,382	—	—	19,096	—	—
	R1	—	2,496	9,168	2,380	2,157	6,966
	R2	—	3,057	10,871	2,920	1,883	7,507
	R3	—	3,208	9,726	2,759	1,873	7,898
	R4	—	2,017	11,580	1,010	1,667	10,741
	R5	—	1,809	13,857	1,298	974	10,397
	Total	19,382	12,587	55,202	29,463	8,554	43,509
THREAD NUT	R0	15,429	—	—	15,728	—	—
	R1	—	3,042	7,269	2,787	2,495	4,460
	R2	—	2,915	7,935	2,586	1,705	5,310
	R3	—	2,000	8,042	2,080	1,407	6,124
	R4	—	2,170	8,935	2,026	1,786	6,698
	R5	—	1,350	9,946	998	938	9,226
	Total	15,429	11,477	42,127	26,205	8,331	31,818
ROUTE CABLE	R0	25,081	—	—	25,494	—	—
	R1	—	16,951	21,937	3,371	15,389	19,501
	R2	—	16,453	20,966	6,045	11,736	16,530
	R3	—	15,461	24,329	3,055	12,898	21,049
	R4	—	16,252	34,091	2,930	14,250	25,654
	R5	—	14,231	26,341	3,880	13,244	20,449
	Total	25,081	79,348	127,664	44,775	67,517	103,183

C Task and Environment Details

This section describes the five tasks (App. C.1), the initial states each task is reset to (App. C.2), and how episodes are scored (App. C.3). Platform, control-loop, and episode-cap settings for both domains are in Tab. 21.

C.1 Platforms and tasks

All real-world experiments run on a Franka Panda arm on the DROID platform [5], with ZED stereo cameras and a SpaceMouse for teleoperation at 15 fps. An operator watches every collection episode from the teleoperation station and may take over at any point to correct the policy, rescue the episode, or record a counterfactual demonstration. All three real-world tasks end in an insertion or seating stage with tight clearance, where a small pose error is enough to fail.

In simulation we use Robosuite [60] with Robomimic [6] and MimicGen [9] assets (MimicGen 1.0.0 on Robosuite 1.5.2), on a 7-DoF Franka arm at a 20 Hz environment control rate. Both simulated tasks are square-nut insertion and differ only in the initial-state distribution.

Tab. 8 lists the tasks, and Fig. 18 shows one successful final-round episode of each real-world task, frame by stage.

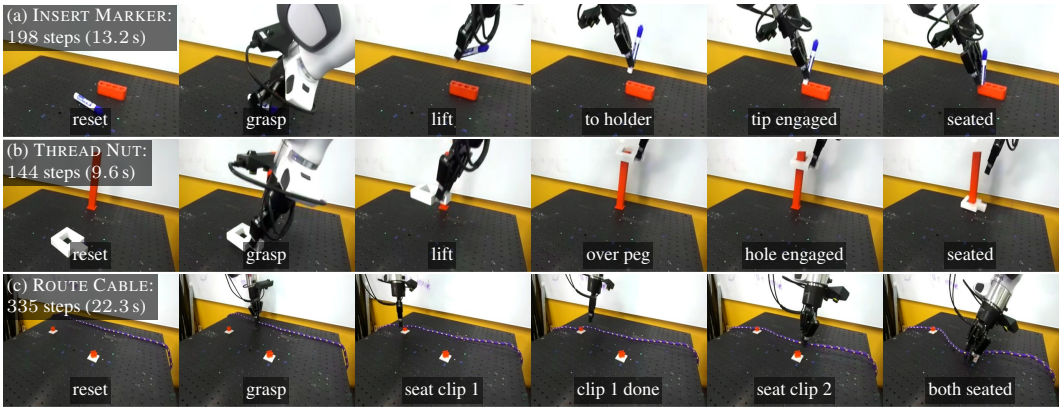


Figure 18: The three real-world tasks. Each row is the median-length successful held-out episode of the final-round HiL-IDQL+**Mulligan** policy from the side camera, with frames at the rungs of each task's stage ladder (App. C.3), not at even intervals. The median round-0 demonstration takes 12.7, 10.1, and 18.6 s. In (b), the policy carries the nut above the camera's field of view between frames.

Table 8: The five tasks. The last column is the median length of the round-0 teleoperated demonstrations over all arms (200 on SQUARE-NARROW and 400 on SQUARE-BROAD), and in the real world also the held-out round-0 block (250 per task).

Task	Objective	Median demo
Real world		
INSERT MARKER	Grasp a pen from the table and insert it into a holder whose mouth leaves $\sim$ 1 mm of clearance.	12.7 s
THREAD NUT	Grasp a square nut and thread it onto a fixed-diameter peg.	10.1 s
ROUTE CABLE	Seat a rope into two clips in succession.	18.6 s
Simulation		
SQUARE-NARROW	Insert a square nut onto a fixed peg. This is the Robomimic Square task (MimicGen `Square_D0`).	8.3 s
SQUARE-BROAD	The same insertion over a wider nut range with a randomized peg (MimicGen `Square_D1`). Nut and peg placements that collide or are infeasible are rejected.	8.6 s

C.2 Initial-state ranges and difficulty

Tab. 9 gives the initial-state ranges. Position ranges refer to object or target centers, and rotations are in-plane yaw. In the real world, the operator places the objects from a per-episode reset-helper schematic (App. E.4). We compare these ranges with those of prior systems in App. K.3.

Table 9: Initial-state ranges. Position ranges are the $x \times y$ extents, in centimeters, over which each object or target center is placed. Yaw is the in-plane rotation range of the movable object. Each ROUTE CABLE clip takes one of three discrete yaws, $0°$ or $\pm 45°$ from its nominal orientation.

Task	DoF	Position range ($x \times y$, cm)	Yaw
INSERT MARKER	5	Pen 15.2×30.5, holder 5.1×10.2	$360°$
THREAD NUT	5	Nut 25.4×45.7, peg 5.1×10.2	$360°$
ROUTE CABLE	7	Rope free end 40.6 in x only, each clip 40.6×22.9	clips $\pm 45°$
SQUARE-NARROW	3	Nut 0.5×11.5, peg fixed	$360°$
SQUARE-BROAD	5	Nut 23.0×51.0, peg 40.0×40.0	$360°$

Initial states differ widely in difficulty. On INSERT MARKER and THREAD NUT, the hardest states place the object at the minimum of its x range, closest to the robot base, with the pen or the nut's handle pointing toward the robot. On THREAD NUT these states also require a non-prehensile maneuver that rotates the nut before it can be grasped. Fig. 19 shows a typical and a hardest initial state of each task and the maneuver.

C.3 Success criteria and stage ladders

Tab. 10 summarizes how each task is scored. We pre-specify the real-world success criteria per task and score them from the evaluation video. The binary outcome in the headline numbers is the operator's label, recorded at the end of every episode and reconciled in an outcome-editor pass before ingest. An operator stop before the cap counts as a failure only when it is recorded as an unrecoverable state. ROUTE CABLE success plots report the binary both-clips outcome, and Tab. 3 reports both ROUTE CABLE metrics.

We freeze each real-world evaluation cap per task at roughly three times the median successful policy episode length and never recompute it between rounds. On THREAD NUT, round 0 ran with a 400-step cap and rounds 1–3 were operator-terminated. The frozen cap applies from round 4, so steps-to-success on THREAD NUT are comparable across rounds only from then on. Collection episodes are operator-terminated and uncapped.

Table 10: Scoring and evaluation caps. Clip success rate is the mean ROUTE CABLE clip score (0–2) divided by two. Caps are in environment steps, at 15 Hz in the real world and 20 Hz in simulation.

Task	Success	Headline metric	Cap
INSERT MARKER	Marker released and fully seated (S7)	success rate	800 (53 s)
THREAD NUT	Nut released and fully seated (S7)	success rate	600 (40 s)
ROUTE CABLE	Both clips seated (S10)	clip success rate	850 (57 s)
SQUARE-NARROW	Nut on the peg (Robosuite success check)	success rate	400 (20 s)
SQUARE-BROAD	Nut on the peg (Robosuite success check)	success rate	400 (20 s)

Episodes in the reviewed collection campaigns also carry an ordinal *stage* label recording how far along the task's ladder the rollout got before it broke down. These labels drive the earliest-breakdown-first hardness ordering of the real-world sampler (App. F.6) and the per-stage decompositions of App. B.7. Tab. 11 lists the S0–S7 ladders of INSERT MARKER and THREAD NUT, and Tab. 12 the S0–S10 ladder of ROUTE CABLE.

A vision-language model labels these stages from the episode video, blind to the policy and arm. On INSERT MARKER and THREAD NUT, we give Gemini 3.5 Flash the side and wrist-camera videos, the gripper-aperture trace, and crops of the grasp, release, and final frames. We label each episode three times and keep the majority label. The model cannot override the gripper caps of Tab. 11, and the S7 rung always comes from the operator's outcome label. We audit flagged and

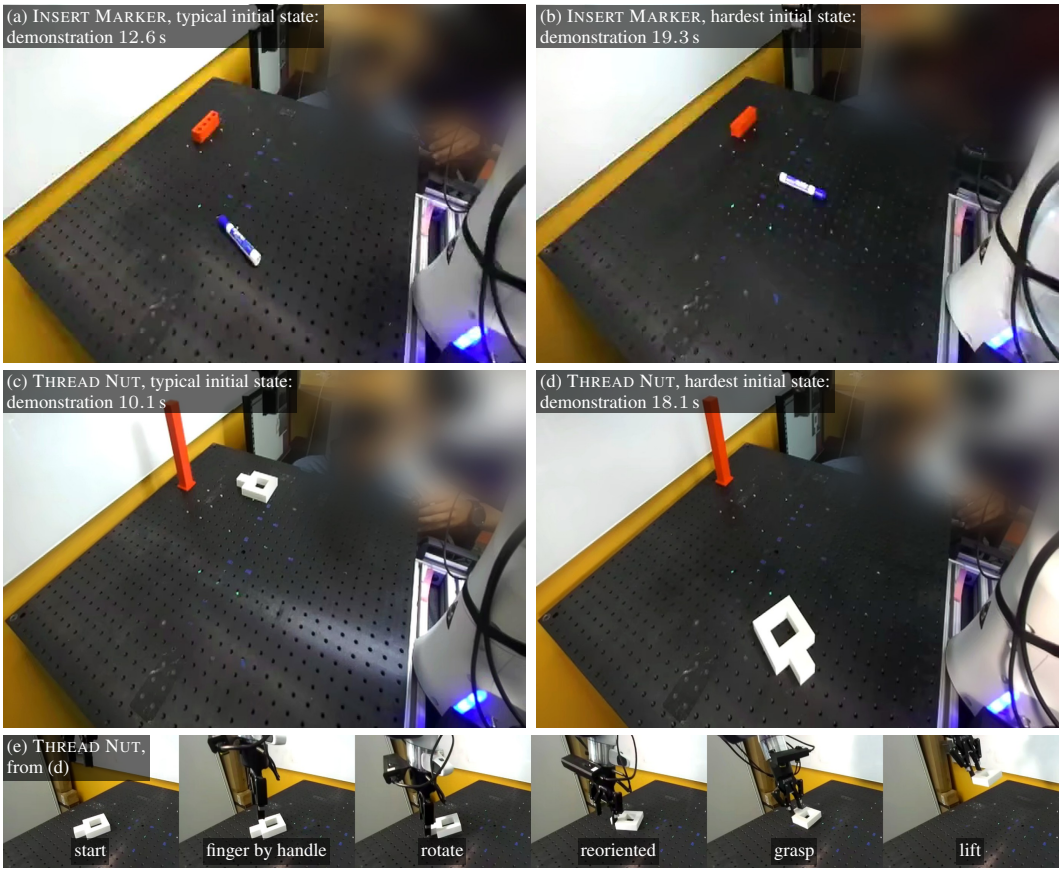


Figure 19: Typical and hardest initial states of INSERT MARKER and THREAD NUT in round-0 teleoperation. (a)–(d) are the first frames from the side camera next to the robot base, which is on the right (operator blurred). Each hardest initial state is the first state of the task's longest round-0 demonstration, with the object near the minimum of its x range and pointing toward the robot. Each typical initial state comes from a median-length demonstration whose object points away from the robot. The typical INSERT MARKER initial state lies at nearly the same x as the hardest one. (e) From the hardest THREAD NUT initial state, the robot places a finger next to the handle with the gripper yawed $180°$ from the grasp orientation, rotates the nut, and then grasps it, seen from the opposite side camera. The three longest round-0 demonstrations of each task all start with the object within $21°$ of pointing toward the robot.

uncertain episodes by hand in a video review tool. Wherever a human label exists, it replaces the model's. Scored before any correction, the labeler agrees with the human rung on 12 of the 19 INSERT MARKER round-1 audit episodes and is within one rung on 18. For ROUTE CABLE, Gemini 3.7 Flash labels the 750 held-out rollouts with no vote and no human adjudication. No reported result uses these labels.

Table 11: Stage ladders of the two binary real-world tasks. The labeler reports the furthest rung reached. S7 is the success predicate. An episode whose jaws never close is capped at S1, and one whose jaws never reopen at S3. On INSERT MARKER, an episode that ends with the marker back on the table is capped at S2.

Rung	INSERT MARKER	THREAD NUT
S0	No useful approach toward the marker.	No useful approach toward the nut.
S1	Approach or grasp attempt without a usable grasp (missed close, or an end pinch that cannot be inserted).	Reach or grasp attempt without a grasp that can lift and carry the nut.
S2	Marker acquired with a usable grasp and lifted clear of the table.	Nut grasped and lifted clear of the table, even if later lost.
S3	Marker brought to the holder area; pressing the holder body without the tip engaging the hole stays S3.	Held nut reaches the peg region in an insertion-directed pose; hole not verified on the peg.
S4	Marker tip engages the hole opening but is not fully seated.	Nut's hole visibly engages or aligns over the peg top; not seated at the base.
S5	Gripper released a partial insertion that the holder retains.	Gripper released a partial insertion; the nut hangs on the peg above the base.
S6	Marker fully seated while still held at episode end.	Nut fully seated at the peg base but still held.
S7	Marker released and remains fully seated (success).	Nut released and remains fully seated (success).

Table 12: Stage ladder of ROUTE CABLE. S10 is the both-clips success predicate.

Rung	ROUTE CABLE
S0	No useful approach.
S1	Approach or contact without a usable grasp.
S2	A grasp sufficient to route.
S3–S5	Reach, align, and engage the first clip.
S6	One clip seated.
S7–S9	Reach, align, and engage the remaining clip.
S10	Both clips seated (success).

D Evaluation Protocol Details

Referenced from Sec. 4.0.

Task-specific success criteria and stage labeling are in App. C.3.

We follow recommended practice for physical-robot policy evaluation [64], with matched initial conditions across compared policies, pre-specified success criteria, metrics that complement success rate, and intervals in addition to point estimates.

Separating evaluation from collection. Interventions during DAgger collection could double as failure labels, but we keep evaluation and collection in separate phases, for four reasons:

- Every policy in a round is scored on the same initial states, so comparisons are paired.
- Evaluating on Sobol initial states gives a lower-variance estimate of the average success rate. The initial states that performance-guided collection picks skew toward hard ones, and uniform random draws have high variance at these sample sizes.
- Evaluation rollouts feed the next round's choice of initial states and enter value learning directly.
- Operators correct before an irrecoverable mistake, so some interventions fall on episodes that would have succeeded, which inflates the failure rate measured during collection.

In simulation, we score each policy on a fixed held-out evaluation set, reused across rounds, using the final checkpoint at a pre-committed training-step count (no post-hoc checkpoint selection). In the real world, each round scores all of its policies together on a shared held-out block of 50 scrambled-Sobol initial states, disjoint from every initial state used for collection. On INSERT MARKER and THREAD NUT, each round uses a fresh block, and we retire used evaluation seeds into the sampler's design pool instead of reusing them. Because we score each round on a new block, and the environment can drift from day to day, success rates are comparable across methods within a round but not directly across rounds. On ROUTE CABLE, we instead scored the 15 policies of all rounds together on one block, drawn after the last collection round and disjoint from every collection and earlier evaluation state, so round-to-round comparisons on that task are also paired on the same initial states and less exposed to drift between sessions.

Rounds scored over several sessions. In two rounds, we scored the reranked policy in later sessions on new blocks, after the round's actor block was complete. Each plotted success rate pools all blocks on which that checkpoint was scored in its round, since every block is an independent Sobol draw from the same held-out distribution. Paired tests use only initial states on which both compared methods ran within one session. HG-DAgger+**Mulligan**, the one method scored in every session, succeeds on 23 and 24 of 50 initial states in the two INSERT MARKER R2 sessions and on 29, 23, and 25 of 50 in the three THREAD NUT R3 sessions. When we restrict THREAD NUT R3 to its last session, the only one that scores all three methods, HG-DAgger succeeds on $20/50$ and HiL-IDQL+**Mulligan** on $31/50$, compared with $49/100$ and $62/100$ pooled.

Blinded protocols. All teleop collection, DAgger corrections, and evaluations are blinded. We combine the initial states of all arms into one scrambled list and run the compared policies under anonymous labels on this shared set. This hides the arm from the operator and spreads environmental drift, such as lighting and motor temperature, across arms. Fig. 20 illustrates one evaluation session. Kress-Gazit et al. [64] report a policy dropping from $13/20$ to $0/8$ on *identical* initial conditions after an unrelated change to the lab, so we interleave compared policies within one session. Sobol evaluation states cover the space more evenly than uniform samples (Fig. 25), and on INSERT MARKER and THREAD NUT a fresh block every round keeps iterative development from overfitting to a fixed set of initial states. To place objects quickly and blindly, the operator follows a per-episode reset-helper schematic (Fig. 23). We release video of every evaluation episode behind the compared methods' reported results, alongside per-episode outcomes and the initial-state list each session was

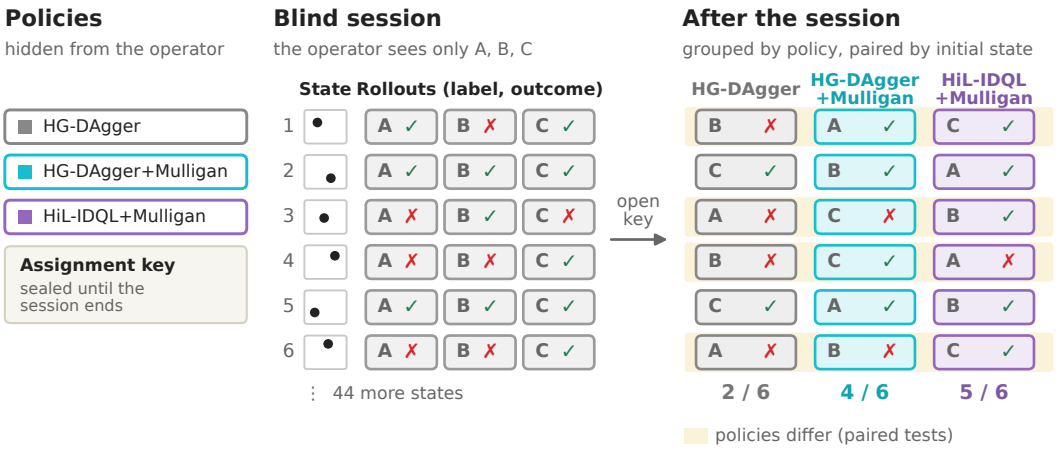


Figure 20: **Blinded, paired evaluation** (schematic, shuffles and outcomes are illustrative). The operator places the objects at each initial state of the held-out Sobol block, and every compared policy runs from that placement under an anonymous label, with the labels reshuffled for every initial state. Outcomes are recorded under the labels only. After the session, the sealed assignment is opened, and the outcomes are regrouped by policy, one row per shared initial state. Success rates come from the column totals, and the paired tests compare two policies on the initial states where their outcomes differ (shaded rows).

scored on, so readers can check the pairing across arms, the blinding, and the initial states realized at reset.

Uncertainty quantification. For simulation policy-performance outcomes, we report two-sided Student-t 95% confidence intervals across five training seeds, each estimated from a large number of trials. In the real world, each plotted policy is one training run, so per-arm whiskers describe evaluation-sampling variability rather than training-run variability. The Wald binomial standard error [65, 66] degenerates near 0% and 100%, so for binary success rates we use the Wilson score interval [63] with $z{=}1$ as a one-standard-error interval. The ROUTE CABLE clip success rate is not binary. Each episode scores 0, $1/2$, or 1, so its whiskers are the standard error of the mean score over episodes. Real-world comparisons use paired per-state statistics, with exact McNemar tests [67] on discordant pairs and 95% bootstrap confidence intervals [68] on the paired success-rate delta (App. B.2).

Released evaluations. The Hugging Face release (`https://huggingface.co/mulligan`) contains 2,550 evaluation episodes of the headline methods: 850 on INSERT MARKER, 950 on THREAD NUT, and 750 on ROUTE CABLE. With the no-CF ablation, it contains 2,850. Two further datasets hold the superseded INSERT MARKER and THREAD NUT R5 screening evaluations (400 episodes). All three THREAD NUT R5 arms are scored on a later fresh Sobol block, because the critic trained on rollouts from the earlier R5 initial states. The Policy Arena at `https://arena.mulligan.page` lets readers browse these episodes and the per-session comparisons.

E Data and Collection Protocol

Alg. 2 summarizes the supervised improvement loop of Sec. 3.

Algorithm 2 The supervised improvement loop, with **Mulligan** choosing the initial states in the Analyze step. HG-DAgger skips the critic and deploys π_θ alone.

1: $\mathcal{D}_h, \mathcal{D}_{\text{all}} \leftarrow \mathcal{D}_0$, base demonstrations at scrambled Sobol initial states in $\mathcal{S}_0$
2: **for** round $r = 1, \ldots, R$ **do**
3: **Train:** actor π_θ by BC on $\mathcal{D}_h$; critic (Q_ψ, V_ξ) on $\mathcal{D}_{\text{all}}$ (Sec. 3.2)
4: **Evaluate:** deploy π_θ with Q_ψ-reranking (eq. (1)) for K episodes; add data to $\mathcal{D}_{\text{all}}$
5: **Analyze:** choose the next M initial states with SELECTINITIALSTATES (Alg. 1)
6: **Collect:** M supervised episodes, each counterfactual demonstration replacing the last coverage-fill initial state; add successful episodes' human frames to $\mathcal{D}_h$ and all frames to $\mathcal{D}_{\text{all}}$
7: **end for**

E.1 The four data streams and how frames are routed

Supervised deployment produces four streams: base demonstrations, human corrections during a rollout, autonomous successes, and autonomous failures. Every frame carries a *source* tag $\in \{\text{policy}, \text{human}\}$ and a binary *intervention* flag, and every completed episode carries an *outcome* $\in \{\text{success}, \text{fail}\}$. These three tags decide which frames reach the actor and which reach the critic. We never select frames by hand.

Fig. 21 shows one supervised deployment round. The actor trains only on $\mathcal{D}_h$, the human frames from *successful* trajectories (base demonstrations, corrections that rescued an episode, and counterfactual demonstrations). The critic trains on $\mathcal{D}_{\text{all}}$, every frame of every evaluation rollout and collection episode, whether human or policy and whether the episode succeeded or not. Autonomous successes and failures, which interactive imitation alone discards, reach only the critic.

E.2 Round budgets and dataset ledger

Round 0 records 100 teleoperated demonstrations per arm on every task except SQUARE-BROAD, which records 200. Each later real-world round collects $M = 50$ HG-DAgger episodes per arm on INSERT MARKER and THREAD NUT and $M = 100$ on ROUTE CABLE, and is evaluated on a held-out block of 50 initial states shared across all arms in that round. On ROUTE CABLE, each of R1–R4 pools two 50-episode collection sessions and R5 is a single 100-episode session. We sum frame counts over a round's sessions. Each later simulation round collects $M = 100$ episodes per arm. Tab. 13 gives the per-task budgets and final-round training-set sizes. $\mathcal{D}_h$, the actor's set, contains every episode with human-controlled frames (demonstrations, corrected episodes, and counterfactual replays). Its frame count includes only the human-controlled frames, since those are what the actor trains on. $\mathcal{D}_{\text{all}}$ adds the autonomous frames of every collected episode. The critic also trains on autonomous rollouts of earlier policies (held-out evaluations in the real world, separate diagnostic rollouts in simulation), so we list its final-round set separately. Frame counts of $\mathcal{D}_h$ and $\mathcal{D}_{\text{all}}$ are valid frames after the outcome-edited prefix. Critic-set frames are the recorded lengths of the training datasets. None of these counts measure operator wall-clock time.

$|\mathcal{D}_h|$ leaves out the collection episodes that the policy completed without a correction: 56 on INSERT MARKER, 49 on THREAD NUT, 15 on ROUTE CABLE, 171 on SQUARE-NARROW, and 57 on SQUARE-BROAD.

E.3 Counterfactual-demonstration protocol

A counterfactual demonstration is a full human demonstration from the *same* initial state at which the policy just failed, with no policy frames (Sec. 3.1). An HG-DAgger correction picks up from whatever state the policy reached on its own, which limits it to teaching recovery. For a counterfactual, the operator resets the scene to the recorded initial state and teleoperates the whole task.

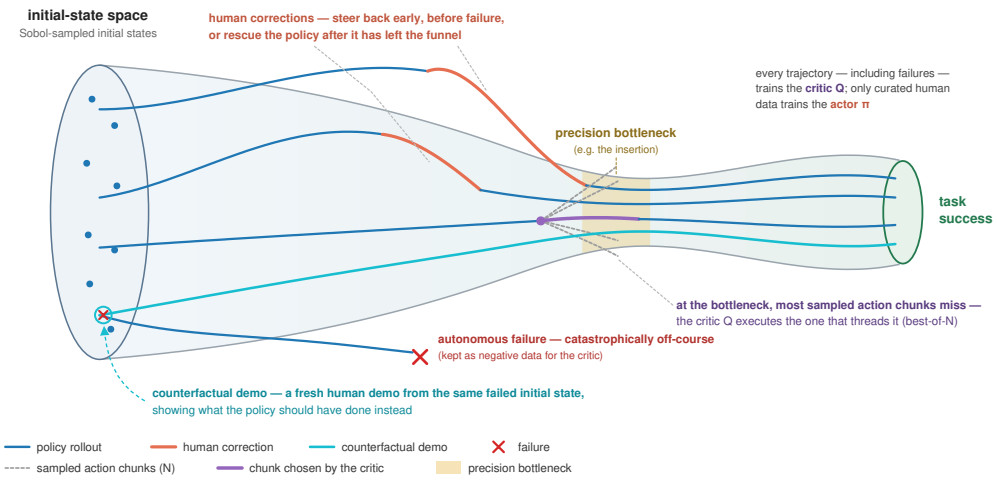


Figure 21: **One supervised deployment round as a trajectory funnel.** Each rollout starts from a Sobol-sampled initial state and succeeds only if it passes a precision bottleneck such as the insertion. The operator can correct *early*, steering the policy back before it fails, or *rescue* it after it has left the funnel. When a rollout fails without intervention, we keep it as negative data for the critic, and the operator may record a counterfactual demonstration from its initial state. Most action chunks sampled at the bottleneck miss. Best-of-N reranking with the IQL critic executes one that passes.

Table 13: Collection budgets and final-round training-set sizes for the HG-DAgger+**Mulligan** arm. M counts episodes per round per arm after round 0, which records 100 demonstrations per arm (200 on SQUARE-BROAD). "$\mathcal{D}_h$" is the actor's training set (episodes with human frames, with the human-controlled frame count), "$\mathcal{D}_{\text{all}}$ frames" counts all valid frames of the collected episodes, "critic set" is the final-round critic's training set including rollouts of earlier policies, and "collected" counts every recorded round-0 and collection episode across both the uniform-sampling and **Mulligan** arms.

Task	Rounds	M	$\lvert\mathcal{D}_h\rvert$ episodes	$\mathcal{D}_h$ frames	$\mathcal{D}_{\text{all}}$ frames	Critic set (episodes / frames)	Collected episodes
INSERT MARKER	R0–R5	50	294	38,017	81,526	825 / 262,164	824
THREAD NUT	R0–R5	50	301	34,536	66,354	900 / 209,953	824
ROUTE CABLE	R0–R5	100	585	112,292	215,475	1,200 / 548,588	1,334
SQUARE-NARROW	R0–R3	100	229	25,216	70,563	700 / 130,983	800
SQUARE-BROAD	R0–R3	100	443	57,983	100,781	900 / 208,219	1,000

A counterfactual costs a full demonstration of operator time, so the operator records one by choice, not after every failure. The collection software offers the option after every episode the operator drove to completion. The operator takes it when the correction did not itself show the intended behavior (Sec. 3.1). Each counterfactual counts toward the with-CF arm only and displaces the last coverage-fill initial state from that arm's M-episode budget, so we can compare the arms with and without CFs at the same total number of episodes. Tab. 14 lists the counterfactuals per round. The operator recorded most of them at promoted failure states, and about a quarter at coverage-fill states where the policy also failed.

Table 14: Counterfactual demonstrations per collection round in the HG-DAgger+**Mulligan** arm, out of 50 episodes on INSERT MARKER and THREAD NUT and 100 on ROUTE CABLE.

Task	R1	R2	R3	R4	R5
INSERT MARKER	12	15	14	9	9
THREAD NUT	17	16	15	15	9
ROUTE CABLE	13	24	11	10	12

E.4 Reset-helper schematics for real-world collection

Referenced from Sec. 3.1.

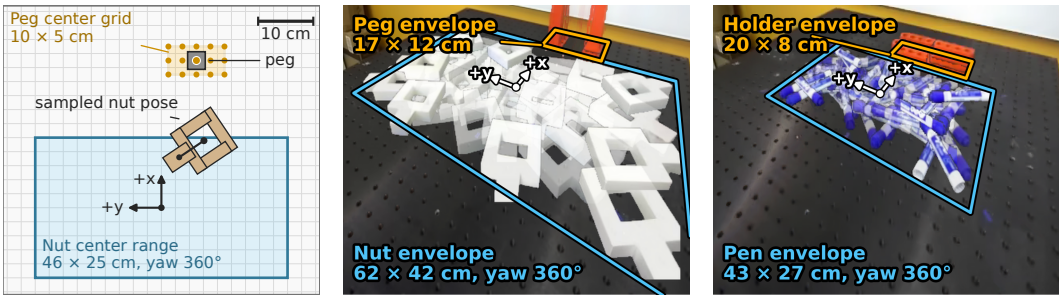


Figure 22: Left: the THREAD NUT reset-helper card from which the operator places each episode's objects. The nut center is sampled uniformly over the blue range at any yaw, and the peg is snapped to the orange grid (all three tasks in Fig. 23). Middle and right: side-camera overlays of boundary and interior reset poses for THREAD NUT and INSERT MARKER, with each object's envelope, its center range grown by its reach, outlined and labeled. The ROUTE CABLE overlay is in Fig. 1, left.

Fig. 23 shows the reset-helper schematic of each real-world task.

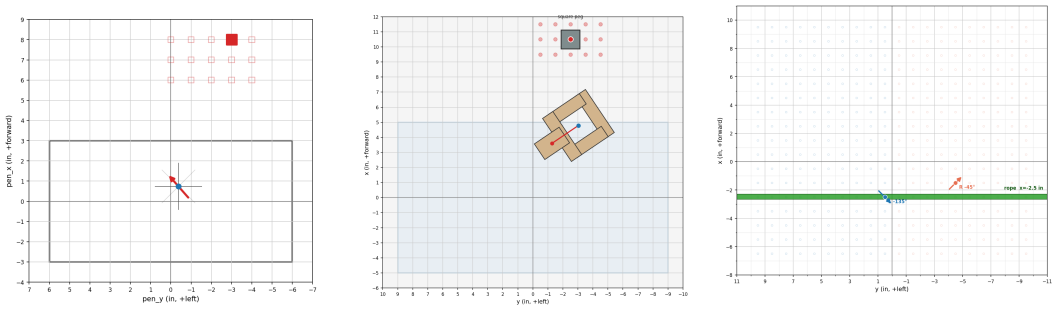


Figure 23: Per-episode reset-helper schematics for placing objects at each sampled initial state. Each card shows the allowed object-center grid, the chosen locations, and the sampled object poses (position and yaw) in table coordinates. Left: INSERT MARKER (pen + holder). Middle: THREAD NUT (nut + peg). Right: ROUTE CABLE (cable + two clips).

F Initial-State Sampling Details

Referenced from Sec. 3.1.

F.1 Difficulty varies sharply across the initial-state space

As Fig. 1 and Fig. 24 show, not all of $\mathcal{S}_0$ is equally difficult, which is the premise of Sec. 3.1. We evaluate a round-2 SQUARE-NARROW policy from 80 fixed initial states on a lattice spanning the full reset range (2 x-values $\times$ 5 y-values $\times$ 8 yaws), with 50 rollouts from each. Overall the policy succeeds 86% of the time. Most initial states sit at or near 100%, and a small tail of position/yaw combinations fails most of the time.

This policy comes from the coverage-only arm of that campaign, so where its data were collected cannot explain the structure. It was trained on 100 base demonstrations and two rounds of 100 supervised episodes each, and all 300 initial states were drawn from the same scrambled Sobol sequence over the full reset range, with no failure promotion and no hardness weighting. In both collection rounds the operator followed the standard protocol and chose to record 20 and 22 counterfactual demonstrations. Even with the same collection density everywhere in $\mathcal{S}_0$, the failures concentrate. Uniform sampling spends effort in proportion to volume and puts most of it on states the policy has already mastered. SELECTINITIALSTATES (Alg. 1) moves that budget to the failing tail.

Only the two endpoints of the 5 mm x range enter the lattice. On the same lattice, the other two training seeds reach 77.9% and 81.9%.

F.2 Sobol vs. uniform coverage

Referenced from Secs. 3.1 and 4.2.

Low-discrepancy sequences. A low-discrepancy sequence spreads every prefix evenly over $[0, 1]^d$. Its *star discrepancy*, the largest gap between the fraction of points in an anchored box and the volume of that box, falls as $\mathcal{O}((\log n)^d/n)$. I.i.d. uniform samples fluctuate at $\mathcal{O}(n^{-1/2})$ and, at our budget sizes ($n = 50$–100), leave clumps and holes. The Sobol sequence [55, 57] balances each block of 2^m consecutive points exactly across recursive halvings of the domain. We materialize draws in base-2 blocks to keep that balance and slice them by index range (App. F.3). Owen scrambling [56] randomly permutes the binary digits of each coordinate without breaking this balance. Each scrambled point is marginally uniform, which makes a scrambled draw an unbiased sample of $\mathcal{S}_0$, and independent scramblings give independent replicates.

Fig. 25 plots the SQUARE-NARROW R0 initial states of the uniform and Sobol arms compared in Sec. 4.2. The Sobol initial states cover nut y and yaw more evenly. All three statistics at the bottom of the figure agree, with lower centered discrepancy, a larger minimum nearest-neighbor distance, and flatter marginal bin counts for Sobol.

F.3 Sobol sequence configuration

All coverage draws use a scrambled Sobol sequence [55, 56, 57] over the per-task initial-state bounds, materialized in base-2 blocks and sliced by index range. The dimensions each sequence spans are:

- **SQUARE-NARROW (sim):** the initial-state space is 3-D (nut x, y, yaw), but the deployed sequence is 2-D over nut y and yaw, from a fixed 1024-point pool at scramble seed 42. Nut x is drawn independently uniform from its own seed, and each round consumes the next contiguous block of stream indices.
- **SQUARE-BROAD (sim):** 5-D, adding peg x and y, with rejection of nut/peg placements that collide or are infeasible.
- **Real (THREAD NUT, INSERT MARKER, ROUTE CABLE):** a Sobol draw over each task's placement dimensions (e.g. nut x, y, and yaw for THREAD NUT, and the rope's free x-

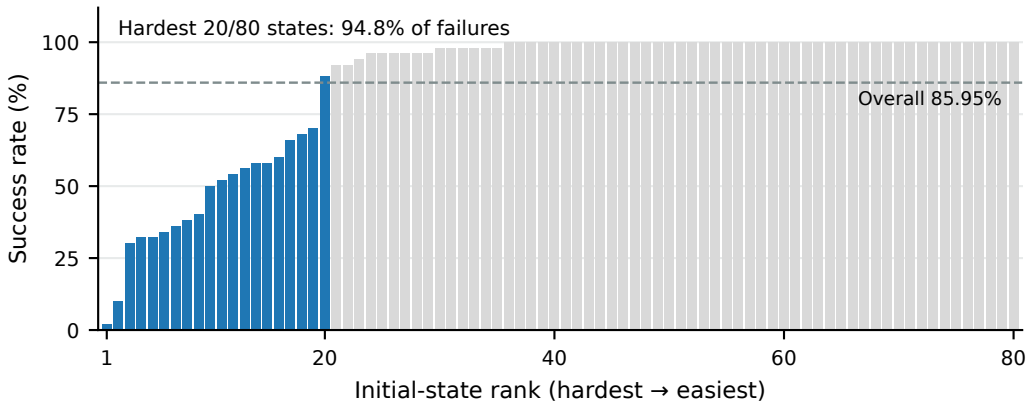


Figure 24: Success rate by initial state on SQUARE-NARROW (80 lattice initial states over the full reset range, 50 rollouts each, sorted) for a round-2 policy trained with coverage-only (Sobol) sampling, seed 1 of three. Overall success is 85.95% (dashed line), and the hardest 25% of the states account for 94.8% of failures. We recovered the per-state success counts exactly from the original evaluation plot, whose bars fall on the 2-point grid that 50 rollouts allow.

position and each clip's x, y, and yaw for ROUTE CABLE), snapped to the physical dot grid where a task uses discrete placements.

A seed ledger keeps the initial states used for collection and for evaluation disjoint. In simulation, the headline evaluation sets are fixed and reused across rounds, and the diagnostic rollouts whose failures are promoted into collection (Sec. 3.1) draw a fresh disjoint Sobol range each round. On INSERT MARKER and THREAD NUT, when a failed evaluation state is promoted into collection, we retire its seed, and the next round's evaluation reserves a new disjoint range (ROUTE CABLE reuses one block, App. D).

F.4 The shared sampler

Both domains run SELECTINITIALSTATES (Alg. 1), with the coverage fill of the main text extended by an optional hardness tilt. The fill maximizes $d(c, \mathcal{P} \cup \mathcal{G} \cup \mathcal{X})\,(1 + \beta\, h(c))$, where $d(c, \cdot)$ is the nearest-neighbor distance from candidate c to the set of previously collected initial states $\mathcal{P}$, promoted failures $\mathcal{G}$, and fills selected so far $\mathcal{X}$, and $h(c) \in [0, 1]$ is a task-specific hardness estimate (Apps. F.5 and F.6) with weight $\beta \geq 0$. With $\beta = 0$, the fill is farthest-point coverage [58, 59]. For small β the distance factor dominates, so coverage is preserved while supervision tilts toward hard regions.

The farthest-point and nearest-neighbor computations are identical across tasks. Only the hardness signal h, its weight β, and the promotion cap m_f vary by domain. The subsections below give each. Fig. 26 shows how the set of initial states evolves over rounds.

F.5 Hardness signal: simulation

The simulation hardness signal comes from the DAgger episodes and diagnostic rollouts of the current policy, two streams that real-world deployment also provides. After each round, we inspect where diagnostic failures and DAgger interventions concentrate in the initial-state space and summarize this as a geometric heuristic $h(c) \in [0, 1]$ over the initial-state parameters (on SQUARE-NARROW, a weighted sum of nut-y and yaw terms in $[0, 1]$ with weights summing to one, and on SQUARE-BROAD, a ridge regression of diagnostic episode length on seven features of the nut and peg placement, min-max normalized over the candidate pool). We pick the weight β during the

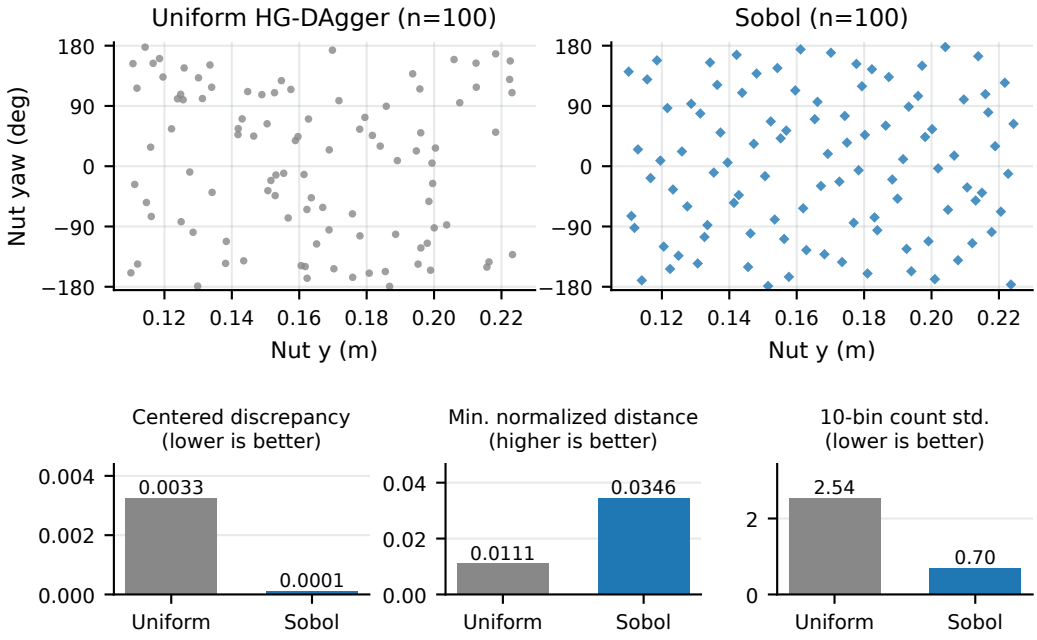


Figure 25: SQUARE-NARROW R0 initial-state coverage of uniform HG-DAgger (left) and Sobol (right), $n = 100$ each, over nut y and yaw. Bottom: centered discrepancy (distinct from the star discrepancy defined above), minimum Euclidean nearest-neighbor distance on the normalized unit square, and the mean of the two marginal count standard deviations over ten equal-width bins.

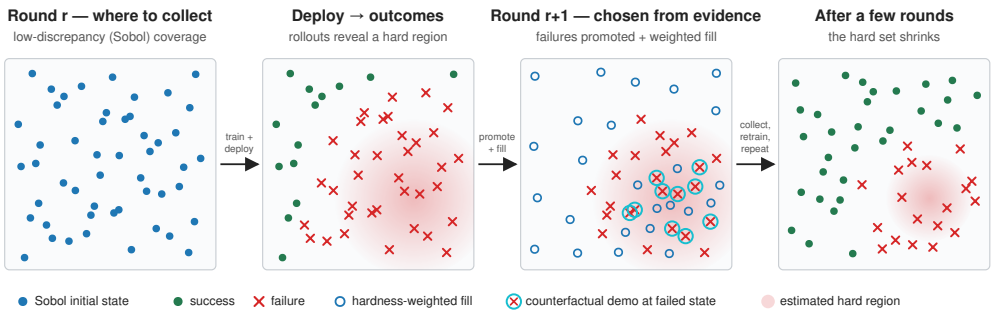


Figure 26: **Performance-guided initial-state design across rounds** (schematic, points are scrambled-Sobol draws). A collection round covers the joint initial-state space with a Sobol set. Deploying the trained policy reveals a hard region (red). The next round's collection set promotes the observed failures verbatim and spends the remaining budget on the coverage fill, optionally hardness-weighted. During collection, initial states where the policy fails again can receive a counterfactual demonstration. Over rounds, the hard set shrinks.

same inspection, typically in $[0.5, 1.0]$. It has to tilt the fill visibly toward the failure-dense regions and still pass the coverage guardrails of App. F.7.

In three rounds (SQUARE-NARROW R3 and SQUARE-BROAD R2–R3), the **Mulligan** fill also used a term derived from evaluation. This term is not part of the method in Sec. 3.1 and has no real-world counterpart. It gave each candidate initial state a hardness bonus based on the previous round's **Mulligan** success in that state's cell of the fixed headline evaluation set (80 cells on SQUARE-NARROW, 720 on SQUARE-BROAD). On SQUARE-NARROW R3 the bonus carried weight 1.0

next to the geometric term at 0.5. On SQUARE-BROAD ($\beta = 1.0$) it replaced the fitted term at R2 and was averaged with it at R3. The initial states used for collection stay disjoint from those used for evaluation. The bonus still steers collection toward the evaluation cells where the previous policy failed, information the uniform baseline never sees. We checked whether this inflated the gap between each learner with and without **Mulligan** (Tab. 4). The gaps at these rounds are not systematically larger. On SQUARE-NARROW, R3 has the smallest gap of the four rounds, $+1.8$ and $+1.5$ points against $+3.0$ to $+8.5$ at R0–R2. The SQUARE-BROAD R3 gap ($+7.6$, $+7.8$) sits below R1's $+8.5$ for both learners. Only SQUARE-BROAD R2 ($+11.1$, $+11.8$) exceeds the rounds without the term, and its intervals overlap those of R1.

Diagnostic failures also feed the next round directly through verbatim failure promotion (Alg. 1). Once promoted, an initial state is retired, and the next round's diagnostic rollouts draw from a fresh, disjoint Sobol range (App. F.3). Because the simulation budget covers every observed failure, we do not order promotions by severity (cf. App. F.6). We report simulation success rates on the fixed Sobol evaluation sets of Sec. 4. A separate fixed set of 400 initial states only monitors training and never enters the sampler.

F.6 Hardness signal: real world

The real world gives $n \approx 50$ evaluation episodes per round, too few rollouts per state to estimate per-cell success rates. We define hardness by *failure stage* instead. We label each evaluation failure by the stage at which the rollout broke down (e.g. grasp $\rightarrow$ transport $\rightarrow$ insert $\rightarrow$ seat $\rightarrow$ release) with the vision-language stage labeler of App. C.3. A state whose rollout broke down earlier is harder, because success requires passing every stage in order. Verbatim promotion (up to the cap m_f) takes failures *earliest-breakdown-first*. ROUTE CABLE collection before the second half of round 4 used a different order, by clip score and then time to failure. We keep $\beta = 0$ in the coverage fill unless a geometric hardness term predicts failure on that task's earlier rollouts. At round 5 this gave $\beta = 0.5$ on THREAD NUT and $\beta = 0$ on INSERT MARKER, where rounds 3 and 4 had run at 0.05 before we dropped the term. ROUTE CABLE used $\beta = 0$ through round 4 and $\beta = 1.0$ at round 5. In the farthest-point distance, we down-weight dimensions that show no hardness signal. The fill then spreads mainly over the axes that do, such as the manipulated object's pose rather than a fixed fixture.

On ROUTE CABLE, we tested eight pre-registered hardness terms on the rollouts before the round-4 design. None predicted failure consistently across rounds and arms, and the fills through round 4 were pure coverage under a stratified quota. At round 5, a descriptive analysis of every earlier ROUTE CABLE rollout with a recorded initial state found that held-out success, first-clip seating, operator burden, and teleoperation time all agreed on one axis, the bend of the rope path at the first clip. Success drops from 17.6% in the straightest bend bucket to 2.5% in the sharpest. We adopted that term at $\beta = 1.0$ for the round-5 coverage fill and left promotion unchanged.

F.7 Budget split and coverage guardrails

Each round's budget M goes first to verbatim failure promotion, one episode per observed failure up to the cap m_f. The hardness-weighted coverage fill takes the rest. The real-world tasks use $M = 50$ and $m_f = 25$ on INSERT MARKER and THREAD NUT, and $M = 100$ and $m_f = 50$ on ROUTE CABLE. Simulation uses $M = 100$ and promotes every diagnostic failure of the round.

At round 5, THREAD NUT split its budget into 21 promotions and 29 fills, and ROUTE CABLE into 50 and 50. On ROUTE CABLE, 21 promotions are initial states from the previous round's held-out evaluation on which both the actor and the reranked actor failed, and 29 are earlier evaluation failures that had not been promoted. These per-round evaluation blocks are separate from the single block on which we report ROUTE CABLE results, which we drew after the last collection round and kept disjoint from every collection and earlier evaluation state. INSERT MARKER at round 5 is the exception, with 30 promotions and 20 fills. Twenty of those promotions are failures of the actor-only arm, and the cap $m_f = 25$ applies to them. The other 10 are states on which the reranked

arm regressed, a separate tier for critic supervision. A floor of 15 coverage fills bounds the total, so promotion can reach at most 35. Two guardrails keep the hardness weighting from collapsing coverage:

- **Coverage ceiling.** Hardness weighting may shift where coverage is spent but may not leave large regions unsampled. We enforce this by keeping the cumulative fill's p_{95} nearest-neighbor coverage radius at or below that of a fresh pure-Sobol block of the same size.
- **No reuse.** We deduplicate candidate fills against all previously collected initial states and all evaluation states. The fitted hardness term learns only from collection episodes and diagnostic rollouts. The exception is the evaluation-grid term of SQUARE-NARROW R3 and SQUARE-BROAD R2–R3 (App. F.5).

G Actor and Critic Implementation

Referenced from Secs. 3.2 and 4.0.

The code is at `https://github.com/ankile/mulligan`.

G.1 Actor

In both settings, the actor is an action-chunked diffusion policy [1] executing 6-step chunks (prediction horizon 8 in simulation, 12 in the real world), trained with a behavior-cloning loss on the curated human pool $\mathcal{D}_h$ (Sec. 3.2). No critic gradients reach the actor in any arm. Tab. 16 lists the actor hyperparameters.

G.2 Critic

Sec. 3.2 names four departures from IQL. First, a transition spans the h executed steps of a chunk, so the critic bootstraps h steps ahead [53],

$$y = \sum_{i=0}^{h-1} \gamma^i r_{t+i} + \gamma^h (1 - \texttt{done})\, V_\xi(s_{t+h}), \tag{2}$$

where `done` marks the end of an episode. Second, over long horizons this one-chunk target propagates reward slowly, so the critic instead regresses onto the λ-return [69], an exponentially weighted average of chunked n-step targets in the spirit of GAE [70],

$$y^\lambda = (1-\lambda) \sum_{n \geq 1} \lambda^{n-1} y^{(n)}, \qquad y^{(n)} = \sum_{i=0}^{nh-1} \gamma^i r_{t+i} + \gamma^{nh}\, V_\xi(s_{t+nh}), \tag{3}$$

truncated at the end of the episode, with λ annealed from near 1 to 0 over early training so that the target reduces to eq. (2) and its bias vanishes (App. H.2). Third, its value head reimplements distributional implicit value learning (DIVL) [15], replacing the scalar value with a categorical return distribution (below). Finally, since these tasks alternate between large transport motions and small insertion corrections, the critic rescales its action input by a state-conditioned affine transform, $\tilde{\mathbf{a}} = (\mathbf{a} - \mu(s)) \odot \exp(-\ell(s))$, with offsets $\mu(s)$ and log-scales $\ell(s)$ produced by a small network, so that action differences are represented at a scale that depends on the current state (App. G.3).

The critic uses discount $\gamma \in [0.99, 0.997]$ depending on task horizon, Polyak-averaged target networks (rate $\rho = 0.005$), and base expectile/quantile level $\tau_{\text{base}} = 0.7$. Rewards are the sparse task outcome. In simulation, we add the intervention penalty of Luo et al. [20], -1 on intervention frames (Tab. 17). The deployed real-world critics set that penalty to zero. We take the minimum over an ensemble of two Q-networks and clip TD targets. In simulation, the clip range is the empirical return range, $\text{clip}(\hat{y}, G_{\min}, G_{\max})$, and in the real world it is a fixed per-task range, $[0, 1]$ on INSERT MARKER and THREAD NUT and $[0, 2]$ on ROUTE CABLE. A larger, REDQ-style ensemble [71] gave no benefit (App. J.5). Tabs. 17 and 18 list the critic hyperparameters.

λ-return target. The real-world critics regress onto the λ-return of eq. (3), the exponentially weighted average of chunked n-step targets $y^{(n)}$ with weights $(1-\lambda)\lambda^{n-1}$, truncated at the episode horizon so that the largest n spans the full episode (798 steps on INSERT MARKER, 480 on THREAD NUT, 846 on ROUTE CABLE, each a multiple of the 6-step chunk). λ starts at 0.995, is held for the first 10k updates (12.6k on ROUTE CABLE), and follows a cosine schedule to 0 by 150k updates (188k on ROUTE CABLE), after which the target is the plain chunked target of eq. (2) for the rest of training. We anneal λ so that early training sees low-bias long-horizon returns while the value estimate is poor, and the deployed checkpoints spend their final updates on the low-variance one-chunk target.

Distributional value head. The human-in-the-loop simulation results and the real-world critics use DIVL [15], a categorical value head over 101 fixed atoms with an HL-Gauss target. The quantile

level is $\tau = \mathrm{clip}(\tau_{\text{base}} - \alpha\,\mathcal{H}, \tau_{\min}, \tau_{\max})$, with $\alpha = 0.4$, $\tau_{\min} = 0.5$, and $\tau_{\max} = 0.95$, where $\mathcal{H}$ is the entropy normalized by the logarithm of the number of atoms. The simulation comparison in App. J.4 supports using the same formulation in both domains but does not show an improvement over scalar IQL.

G.3 Action-magnitude modulation in the critic

The round-5 critics for INSERT MARKER and THREAD NUT and all ROUTE CABLE critics use an observation-conditioned affine transformation of the action input to Q (Tab. 18). With it, the critic compares action differences at a scale set by the state. That scale matters when a task alternates between large transport motions and small insertion corrections, and Parés-Morlans et al. [72] report a sharp drop in action magnitude during contact for contact-rich VLAs.

Transformation. Let $x(s)$ concatenate the frozen visual features and normalized proprioception, and let $\bar{a}_{j,d}$ be the critic's globally z-scored action at chunk step j, dimension d. A two-layer MLP with a 256-unit ReLU hidden layer maps $x(s)$ to six offsets $\mu_d(s)$ and six log-scales $\ell_d(s)$. Before either Q head scores the chunk, we apply

$$\widetilde{a}_{j,d} = \begin{cases} \big(\bar{a}_{j,d} - \mu_d(s)\big)\exp\big(-\mathrm{clip}(\ell_d(s), -2, 2)\big), & d = 1, \ldots, 6, \\ \bar{a}_{j,d}, & d = 7. \end{cases} \tag{4}$$

We transform only the six arm dimensions and pass the gripper through unchanged, with one set of state-dependent parameters shared by every chunk step and candidate action. We implement it as FiLM-style feature-wise affine modulation [73] of the critic's action input.

Training and deployment. The transformation starts as the identity because we initialize the MLP's final weights and biases to zero. The Q loss trains its parameters, and the target Q heads that construct the V labels use a Polyak-averaged copy. At deployment, the transform acts only when the critic scores proposals. FiLM can change which proposal the robot executes but not the proposals themselves.

Evidence and scope. Offline, the evidence for FiLM was mixed. On THREAD NUT, FiLM candidates improved the point estimate of the candidate-ranking diagnostic in both tested training seeds, but its confidence intervals included no improvement, and FQE ranked the FiLM candidate below the other tested critics. On INSERT MARKER, frozen-state probes showed changes in selected action magnitude and progress along a human-annotated direction, with results depending on checkpoint and training seed. The later INSERT MARKER robot screen compared critics that all used FiLM, while the THREAD NUT screen compared FiLM with changes to batch size or the value quantile. The reported round-5 critics also combine FiLM with other training changes. These comparisons do not isolate FiLM's contribution to the reported robot gains.

G.4 Best-of-N reranking and inference cost

At deployment, the frozen actor draws N action chunks in a single batched forward pass, the critic scores all N, and the argmax chunk is executed (eq. (1)). The N samples share one denoising pass along a batch dimension, and the critic is a small MLP head on already-computed features. Reranking widens the batch instead of adding N sequential policy evaluations. Tab. 19 lists the deployed N and the selection settings.

We timed this on the robot for THREAD NUT R3 at $N = 16$ over 797 action chunks of 20 held-out episodes. From the start of candidate generation to the argmax selection, a chunk takes 39.1 ms at the median, 45.1 ms at the 99th percentile, and 48.8 ms at worst, with stage medians of 4.0 ms for image encoding, 24.0 ms for batched diffusion sampling with 8 DDIM steps, and 4.2 ms for critic scoring. A chunk of 6 steps at 15 Hz is a 400 ms budget, so this uses at most 12.2% of it.

G.5 Simulation

The actor is a state-based diffusion U-Net [1], paired with an MLP critic with batch size 1024. We train for a fixed number of steps (150k for SQUARE-NARROW, 250k for SQUARE-BROAD) across five replicate runs per arm, with $\gamma = 0.99$. The critic uses the DIVL value head (App. J.4), with base quantile $\tau_{\text{base}} = 0.7$ adjusted by the predicted entropy with sensitivity $\alpha = 0.4$ and clipped to $[0.5, 0.95]$. We deploy reranked simulation arms with $N = 32$. The autonomous baselines and REDQ ablations use a scalar IQL critic. The deployed SQUARE-NARROW Sobol sequence draws only two of the three initial-state axes (App. C.1), and App. F.3 gives the exact Sobol configuration. The simulation columns of Tabs. 16 to 18 and 21 give the full configuration.

G.6 Real world

The actor is an image-based diffusion policy [1] with a ResNet18 [74] encoder per camera (ImageNet [75] initialization and normalization, 224×224 inputs, batch size 64), paired with a Vision-IQL critic that takes its encoder from the deployed actor and keeps it frozen. The critic fits only the Q and V heads on features precomputed over 10 augmented views per frame. We evaluate value-based reranking with $N = 16$ on INSERT MARKER at R2–R4 and THREAD NUT at R3–R4, and with $N = 32$ on both tasks at R5 and on ROUTE CABLE at every reranked round (R3–R5). At INSERT MARKER R2 the reranked policy did not improve on its actor (10 rescues, 11 regressions), so the headline plots the actor alone at that round. The real-world columns of Tabs. 16 to 18 and 21 give the full configuration.

G.7 Compute and training budgets

Tab. 15 lists the logged runtime of the 386 training runs behind the reported results. These are the actor with its scalar critic and the DIVL critic for each of the 34 human-in-the-loop simulation cells and five seeds, and the 36 actors and 10 critics of the released real-world curves. Every run used a single GPU, over 90% of the hours on H100, L40S, RTX 4500 Ada, A40, and RTX A6000 cards. The runs total 801.5 GPU-hours, including setup and in-run evaluation. The total excludes separate evaluation jobs, feature caching, failed runs, autonomous baselines, and development sweeps. The per-run step budgets are in Tabs. 16 and 18.

Table 15: Logged runtime of the training runs behind the reported results, in hours on one GPU. Simulation actors and their scalar critics were trained jointly.

Task	Fit	Runs	Median h	Range h	Sum GPU-h
SQUARE-NARROW	Actor + scalar critic	80	1.90	1.33–5.17	166.9
SQUARE-NARROW	DIVL critic	80	0.65	0.53–1.17	56.9
SQUARE-BROAD	Actor + scalar critic	90	3.01	1.74–6.53	298.8
SQUARE-BROAD	DIVL critic	90	0.97	0.83–1.90	91.3
INSERT MARKER	Actor	12	3.16	1.82–7.85	46.3
INSERT MARKER	Critic	4	5.29	1.61–10.30	22.5
THREAD NUT	Actor	12	4.34	2.73–7.80	56.1
THREAD NUT	Critic	3	6.55	1.22–10.21	18.0
ROUTE CABLE	Actor	12	3.12	1.72–3.27	33.6
ROUTE CABLE	Critic	3	3.69	3.17–4.42	11.3
Total		386			801.5

H Hyperparameters

Referenced from Sec. 3.2.

We read the values below off the configurations that produced the reported results. When simulation and the real world share a value, it spans both columns. The reasoning behind these choices is in App. G. App. F derives the sampler, and the ablations in App. J settled the critic objective.

ROUTE CABLE entries. On ROUTE CABLE, a round is one 100-episode increment of App. E, and the table names ROUTE CABLE wherever its value differs from INSERT MARKER and THREAD NUT. Its three deployed critics (rounds 3–5) all use FiLM DIVL heads, $\gamma = 0.997$, the annealed TD(λ) target through horizon 846, four independent target views per transition, a warmup-cosine learning rate from 10^{-4} to a 10^{-5} floor, and $N = 32$ at deployment.

H.1 Actor

In both domains, the actor is an action-chunked diffusion policy [1] with a conditional U-Net over the action chunk, one conditioning observation step, and DDIM sampling with 8 denoising steps at inference. Simulation encodes a state vector and the real world uses per-camera ResNet18 features. As Tab. 16 shows, the prediction horizon and optimizer preset also differ.

H.2 Critic

We use the categorical value objective for the real-world critics and the human-in-the-loop simulation results (Tabs. 17 and 18), and scalar IQL for the simulation autonomous baselines and REDQ ablations. In simulation, the Q and V heads train while the actor and normalizer stay frozen. On the robot, we freeze the deployed actor's vision encoder and fit the heads on cached features.

H.3 Best-of-N reranking

At each policy query, the frozen actor draws N action chunks in one batched denoising pass and the critic scores each with $Q(s, a_{t:t+h})$ over the executed $h = 6$ steps. The robot executes the highest-scoring chunk. We take a plain argmax, with no temperature, top-k softening, or stage gate. App. G.4 reports the inference cost.

H.4 Initial-state sampler

Tab. 20 instantiates the procedure of App. F.4. Both domains maximize $d(c, \cdot)\,(1 + \beta\, h(c))$ with the same greedy farthest-point loop, although SQUARE-NARROW ranks by d^2. The domains draw fill candidates from different pools and differ in how they gate failures before promotion and how much weight the hardness term carries. The deployed β also changes with task and round. At $\beta = 0$, as on INSERT MARKER at round 5 and on ROUTE CABLE through round 4, the fill reduces to pure coverage.

H.5 Environment, control, and the improvement loop

The real-world episode caps in Tab. 21 follow the rule of App. C.3. Tab. 9 lists the per-task initial-state ranges, and App. D gives the evaluation protocol.

Table 16: Diffusion-policy actor. Simulation uses five replicate runs per arm, and the real world one run per arm per round. We fixed the training step count before seeing any result and deployed the final checkpoint in both domains.

Hyperparameter	Simulation	Real world
Architecture		
Backbone	1-D conditional U-Net over the action chunk	
U-Net channel widths	$[512, 1024]$	
Kernel size / group-norm groups	5 / 8	
Diffusion-step conditioning	128-D embedding, FiLM scale modulation	
Observation encoder	none (state vector)	ResNet18, one per camera
Encoder initialization	—	ImageNet, fully fine-tuned
Encoder normalization layers	—	batch norm, retained
Feature pooling	—	spatial softmax, 32 keypoints
Horizons		
Observation steps	1	
Prediction horizon	8	12
Executed steps per query h	6	
Diffusion		
Training schedule	squared-cosine, 100 timesteps, $\beta \in [10^{-4}, 2 \times 10^{-2}]$	
Parameterization	ϵ-prediction, samples clipped to $[-1, 1]$	
Inference sampler	DDIM, 8 steps	
Observations and actions		
Proprioception	eef. position, quaternion, gripper (9-D)	eef. pose, gripper (7-D)
Other observations	simulator object state	—
Cameras consumed	—	2 (INSERT MARKER, THREAD NUT), 3 (ROUTE CABLE)
Image resolution	—	224×224 after a fixed per-camera crop
Image augmentation	—	brightness, contrast, saturation, hue, sharpness, small affine; at most 3 per sample
Image normalization	—	ImageNet mean and standard deviation
Action space	operational-space delta pose, gripper (7-D)	Cartesian velocity, gripper (7-D) on INSERT MARKER and THREAD NUT; relative pose anchored at the measured end-effector pose, gripper (10-D) on ROUTE CABLE
State/action normalization	min–max	
Optimization		
Optimizer	AdamW	Adam
Learning rate	3×10^{-4}	10^{-4}
Betas	$(0.9, 0.999)$	$(0.95, 0.999)$
Weight decay	10^{-4}	10^{-6}
Schedule	constant	cosine, 500 warmup steps
Gradient clipping (global norm)	1.0	10.0
Weight EMA	not used	
Batch size	1024	64
Training steps	150k (SQUARE-NARROW), 250k (SQUARE-BROAD)	100k
Replicate runs per arm	5	1
Precision	bfloat16 autocast, TF32 matmuls	fp32
Actor training data	$\mathcal{D}_h$: human frames of successful episodes	

Table 17: Critic objective and reward. Both domains fit a categorical value distribution, with a quantile level that adapts to the predicted entropy. Simulation entries are for the reported human-in-the-loop cells and real-world entries for the deployed round-5 critics, with the task named wherever the three tasks differ.

Hyperparameter	Simulation	Real world
Objective		
Value head	categorical (DIVL)	categorical (DIVL)
Expectile / base quantile	$\tau_{\text{base}} = 0.7$	
Discount γ	0.99	0.99 (THREAD NUT), 0.997 (INSERT MARKER, ROUTE CABLE)
Target-network rate ρ	0.005 (Polyak)	
Q-ensemble	2 networks, min aggregation for both the TD label and reranking	
TD target	h-step chunked with a V-bootstrap, $h = 6$	
TD(λ) curriculum	none	λ-return over chunked n-step targets up to the episode horizon (798 steps on INSERT MARKER, 480 on THREAD NUT, 846 on ROUTE CABLE), λ held at 0.995 then cosine-annealed to 0 (hold 10k, zero at 150k on INSERT MARKER and THREAD NUT; hold 12.6k, zero at 188k on ROUTE CABLE)
Target clipping	empirical per-episode return range	$[0, 1]$ (INSERT MARKER, THREAD NUT), $[0, 2]$ (ROUTE CABLE)
Reward		
Task reward	sparse: 1 on success frames	sparse: 1 held over the terminal tail of a successful episode (2 for ROUTE CABLE, one per seated clip)
Intervention penalty	-1 at intervention frames	none
Reward shift	0	
Terminal handling	terminal chunks kept in the loss; padded actions drawn uniformly	terminal tails supervised without bootstrapping; timed-out episodes get no terminal anchor

Table 18: Critic network, optimization, and distributional value head. Real-world values are those of the deployed round-5 critics. Where the three tasks differ, the task is named.

Hyperparameter	Simulation	Real world
Network and optimization		
Q and V trunk	MLP $[1024, 1024, 1024]$ with layer norm	
Q input	observation features concatenated with the flattened 6-step chunk	
Visual features	—	the deployed actor's encoder, frozen; features precomputed over 10 augmented views per frame
Proprioception dropout	0	0.5
Extra critic head	none	state-conditioned FiLM rescaling of the critic's action input (App. G.3), used in the round-5 INSERT MARKER and THREAD NUT critics and in all ROUTE CABLE critics
Learning rate (Q and V)	3×10^{-4}	10^{-4}
Learning-rate schedule	constant	cosine with warmup to a 0.1 floor; 10k warmup steps (12.6k on ROUTE CABLE)
Weight decay	10^{-4}	
Gradient clipping	1.0	
Batch size	1024	256
Training steps	150k (SQUARE-NARROW), 250k (SQUARE-BROAD)	250k (INSERT MARKER, THREAD NUT), 314k (ROUTE CABLE; 270k at R3–R4)
Deployed checkpoint	final step	200k (INSERT MARKER, THREAD NUT), final (ROUTE CABLE)
Batch composition	50% human success, 50% DAgger	uniform over all cached frames
Critic training data	$\mathcal{D}_{\text{all}}$: every transition, regardless of source or outcome	
Relation to the actor	actor and normalizer frozen	frozen actor encoder
Distributional value head		
Atoms	101	101
Support	empirical return range widened by 5%	$[-0.05, 1.05]$ (INSERT MARKER, THREAD NUT); empirical return range widened by 5% (ROUTE CABLE)
HL-Gauss σ	$0.75\times$ atom spacing	$0.75\times$ atom spacing
Entropy sensitivity α	0.4	0.4
Quantile clip	$[0.5, 0.95]$	$[\tau_{\text{min}}, \tau_{\text{max}}] = [0.5, 0.95]$

Table 19: Reranking at deployment. Real-world rows list the reranked rounds, and actor-only evaluations use $N = 1$. App. J.6 discusses the choice of N.

Setting	Value
Candidates N, simulation	32
Candidates N, INSERT MARKER	16 at R2–R4, 32 at R5
Candidates N, THREAD NUT	16 at R3–R4, 32 at R5
Candidates N, ROUTE CABLE	32 at R3–R5
Scored quantity	$Q(s, a_{t:t+6})$, min over the 2-network ensemble
Selection rule	argmax
Candidate draw	one batched denoising pass over N copies of the observation
Rescoring cadence	once per executed chunk, that is every 6 steps
Reranking during collection	simulation: the DAgger collector is itself deployed at $N = 32$

Table 20: Initial-state sampler. All budgets are per arm per round. In simulation, "uncapped" promotion promotes every failure among the round's 100 diagnostic rollouts (the first 100 of 200 at SQUARE-BROAD R1), which came to 1–21 promotions on SQUARE-NARROW and 14–51 on SQUARE-BROAD.

Setting	Simulation	Real world
Sobol draw		
Sampled dimensions	2 (SQUARE-NARROW: nut y, yaw), 5 (SQUARE-BROAD)	5 (INSERT MARKER, THREAD NUT), 7 (ROUTE CABLE)
Remaining dimensions	SQUARE-NARROW nut x: drawn independently uniform	—
Scrambling	linear matrix scrambling with a digital shift (SciPy), always on	
Pool size	1024 (SQUARE-NARROW), 2048 before rejection (SQUARE-BROAD)	32768 (INSERT MARKER, THREAD NUT), 4096 (ROUTE CABLE)
Rejection	SQUARE-BROAD: nut–peg center distance above 13.3 cm	ROUTE CABLE: clip separation at least 5.1 cm
Placement snapping	none	1-inch grid on fixture placements
Index bookkeeping	contiguous disjoint blocks per round; a consumed block is retired	
Budget		
R0 demonstrations	100 (200 on SQUARE-BROAD)	100
Budget M (R1 on)	100	50 (INSERT MARKER, THREAD NUT), 100 (ROUTE CABLE)
Promotion cap m_f	uncapped	25 on the failure pool (50 on ROUTE CABLE); a floor of 15 fills bounds the total
Realized promotions	1–21 (SQUARE-NARROW), 14–51 (SQUARE-BROAD)	21 (THREAD NUT R5), 50 (ROUTE CABLE R5), 30 (INSERT MARKER R5)
Fill count	M minus promotions and perturbations	29 (THREAD NUT R5), 50 (ROUTE CABLE R5), 20 (INSERT MARKER R5)
Failure promotion		
Failure source	the round's diagnostic rollouts	earlier rounds' held-out evaluation blocks, never the reported ROUTE CABLE block
Eligibility	every failed initial state	failures of the **Mulligan** arm, of both actor and reranked actor once both ran
Ordering	none	earliest failure stage, then episode length
Replay	verbatim: the recorded initial state is reproduced exactly	
Deduplication	none	exact match against every previously collected initial state
Local perturbations	SQUARE-NARROW: one state around each of the 20 longest diagnostic episodes, in $\mathcal{G}$ (nut x redrawn uniformly, Gaussian noise of std. 0.025 m on nut y and 0.05 rad on yaw)	none
Hardness-weighted farthest-point fill		
Objective	$\arg\max_c \; d(c,\, \mathcal{P} \cup \mathcal{G} \cup \mathcal{X})\,(1 + \beta\, h(c))$, greedy and deterministic; SQUARE-NARROW uses d^2, and at SQUARE-BROAD R3 $\mathcal{P}$ omits the R2 initial states	
Distance	normalized to the unit cube, yaw treated as periodic (SQUARE-BROAD: not in the initial distance to $\mathcal{P} \cup \mathcal{G}$)	
Axis weights	none	0.70 on the holder (INSERT MARKER), 0.30 on the peg (THREAD NUT), none (ROUTE CABLE)
Fill candidate pool	fresh uniform draws: 50k (SQUARE-NARROW); 9k at R1, 7k at R2–R3 (SQUARE-BROAD)	the scrambled Sobol pool above
Hardness signal h	an episode-length fit over the initial-state parameters, combined at R3 with an evaluation-grid cell term that replaces it on SQUARE-BROAD R2 (App. F.5)	per-task geometric term, screened each round
Deployed β	SQUARE-NARROW: 0.5 on the fitted term, plus 1.0 on the cell term at R3; SQUARE-BROAD: 1.0 (fitted term at R1, cell term at R2, their mean at R3)	0.5 (THREAD NUT R5), 0 (INSERT MARKER R5), 0 (ROUTE CABLE through R4) and 1.0 on a rope-bend prior (ROUTE CABLE R5); INSERT MARKER R3–R4 used 0.05
Coverage guardrail	the cumulative p_{95} nearest-neighbor radius is held at or below that of a fresh Sobol block of the same size (SQUARE-BROAD R2–R3: within 5% of the worse of that block and a uniform draw)	

Table 21: Environment, control loop, and improvement loop.

Setting	Simulation	Real world
Platform	Robosuite with MimicGen assets	Franka Panda on DROID
Control rate	20 Hz	15 Hz
Controller	operational-space pose, delta, base frame	Cartesian velocity
Per-step action limits	5 cm, 0.5 rad	—
Cameras	none; state observations	ZED stereo, HD720 captured at 60 fps
Teleoperation	SpaceMouse, with operator-triggered interventions	
Evaluation episode cap	400 steps	600 (THREAD NUT), 800 (INSERT MARKER), 850 (ROUTE CABLE)
Collection episode cap	none; the operator ends the episode	
Rounds run	R0–R3	R0–R5
Episodes per round per arm	100 (200 at R0 on SQUARE-BROAD)	50 (100 on ROUTE CABLE), 100 at R0
Held-out evaluation set	8000 (SQUARE-NARROW) or 30000 (SQUARE-BROAD) fixed Sobol initial states, reused across rounds	50 fresh Sobol initial states per round (one shared block on ROUTE CABLE)
Training-time evaluation	400 fixed Sobol initial states, at the final step only	held-out action-reconstruction loss every 5k steps
Replication unit	5 training runs	evaluation episodes

I Baseline Details

Each comparison below changes one design choice. The actor architecture, the critic architecture where there is a critic, and the per-round collection budget stay fixed across all arms.

I.1 HG-DAgger

Our HG-DAgger baseline [11] shares the budget, round structure, blinding, evaluation protocol, and actor training recipe of **Mulligan**. The two differ in where episodes start and what the operator collects there. Its initial states are i.i.d. uniform draws over $\mathcal{S}_0$, and the operator records corrections but no counterfactual demonstrations. Because we collect both arms in the same blinded, interleaved sessions, drift in operator skill or in the environment affects both equally.

I.2 Autonomous self-improvement arms

The autonomous arms start from the supervised arms' round-0 checkpoint and then improve with no human in the loop, attempting as many episodes per round as the supervised arms collect. They differ in how they use their own rollouts. *Autonomous IL* [61, 1] retrains on all of them, *filtered BC* [62] retrains on the successful ones only, and *batch-online IDQL* [51, 16] trains a critic on all of them and deploys with best-of-N reranking. Its + *success BC* variant also retrains the actor on its successful rollouts. All of them share the round-0 fixed-grid evaluation. From round 1 on, IDQL deploys with $N = 32$ and the two imitation arms with $N = 1$.

I.3 RECAP learner comparison

Our conclusions could depend on the HiL-IDQL learner (a BC actor, an IQL critic, and best-of-N reranking). To test this, we train RECAP on the same collected datasets and score it on the same fixed-grid evaluation sets. Its prediction horizon (8), execution horizon (6), and advantage horizon (6) match the actor configuration used throughout.

As Tab. 22 shows, RECAP is below the matched HiL-IDQL control in every cohort, and the gap is widest on SQUARE-BROAD. Both learners act from a single actor sample (N=1), and we take each difference seed by seed over the same five seeds.

I.4 HiL-SERL in simulation

We ported HiL-SERL [17] to SQUARE-NARROW and SQUARE-BROAD as a second human-in-the-loop baseline. For the learner we use the SAC form of RLPD [25] from the EXPO recipe (10 critics, UTD 20, 50/50 demo/online batches), starting from the round-0 demonstrations that every other arm uses (100 on SQUARE-NARROW, 200 on SQUARE-BROAD). When the operator takes over with a SpaceMouse, the intervened frames of successful episodes go into the demonstration half of the buffer. With no operator, the split learner/actor system matches the reference RLPD implementation (0.90 vs. 0.84 ± 0.03 eval success at 300k environment steps, one seed vs. five). The reference RLPD runs used the EXPO stack on Robosuite 1.4.1 and MuJoCo 3.4.0, while every other simulation result in this paper, including HiL-SERL, uses Robosuite 1.5.2 and MuJoCo 3.3.7. Eval success is the success rate of 50 policy-only episodes from fixed initial states, scored every 10k

Table 22: RECAP vs. the matched HiL-IDQL learner on identical data. Mean $\pm$ standard error over five seeds. Differences are computed per seed, with two-sided Student-t 95% confidence intervals on four degrees of freedom. Each seed is scored at its final checkpoint on the fixed held-out grid (8,000 episodes on SQUARE-NARROW, 30,000 on SQUARE-BROAD).

Cohort	DP+IQL	RECAP	Δ (pp)	Paired 95% CI
SQUARE-NARROW R2	95.39 ± 0.30	87.02 ± 0.45	-8.37	$[-9.43, -7.31]$
SQUARE-NARROW R3	95.68 ± 0.31	90.03 ± 0.42	-5.65	$[-6.64, -4.66]$
SQUARE-BROAD R3	86.03 ± 0.81	65.36 ± 0.67	-20.67	$[-23.75, -17.59]$

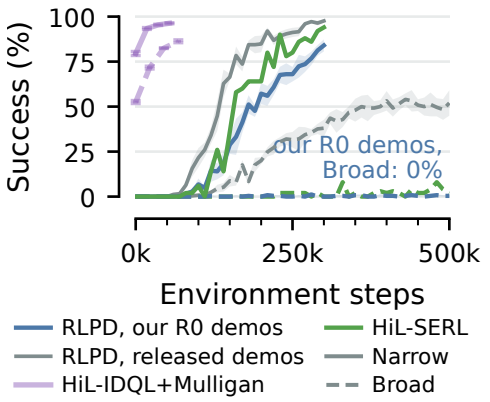


Figure 27: RLPD [25, 29] reaches 84% on SQUARE-NARROW but 0% on SQUARE-BROAD; HiL-SERL [17], with interventions, ends at 94% and 2%.

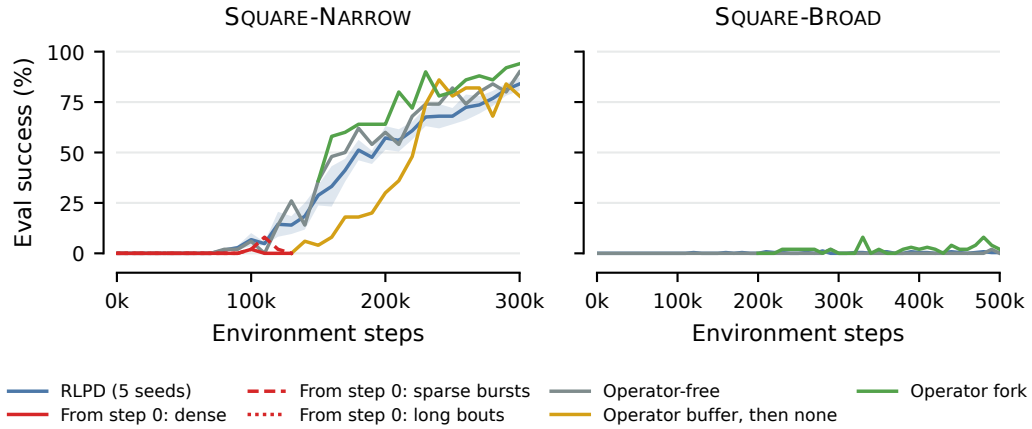


Figure 28: HiL-SERL eval success per session (one seed each) against RLPD (five-seed mean, shaded ± standard error) on SQUARE-NARROW (left) and SQUARE-BROAD (right). Operator forks branch from the operator-free run at 150k and 200k. The operator-buffer arm continues the sparse-burst session from 130k with no operator.

environment steps, and ± is the standard error over seeds. We run one seed per HiL-SERL arm. Figure 28 shows the eval curve of every session and Tab. 23 its operator cost. The HiL-SERL curves in Fig. 27 are the two operator forks: the operator-free run up to the fork, then the forked session with the operator.

SQUARE-NARROW sessions.

- **Operator from step 0**, three protocols: dense takeovers; 25 autonomous failed episodes (9,756 environment steps), then sparse takeover bursts with operator fail-termination; long takeover bouts with fail-termination. None exceeded 0.08 eval success by 91–131k environment steps, and 0 of 637 policy-only training episodes succeeded, over 12.0 operator hours. The operator-free run was at 0.26 by 130k.
- **Operator buffer, then no operator.** When the sparse-burst session continued from 130k without takeovers, the policy took off 50k steps later than the operator-free run (eval first reached 0.3 at 200k, not 150k) and reached 0.78 at 300k (0.90 without an operator).
- **Operator after takeoff.** When we forked the operator-free run at 150k (eval 0.36) and added the operator, the policy reached 0.94 at 300k (0.90 without) and first reached 0.8 at

Table 23: HiL-SERL operator cost per session, one seed each. A fork counts only its episodes after the fork. Steps: the environment-step range run under the protocol. Operator hours: summed wall time of the episodes. Human frames: share of environment steps under operator control. Training success: share of training episodes that succeeded, with operator help where given. Policy-only: successes among training episodes without a takeover. Final and peak eval: at the last checkpoint and the maximum after the fork. Total rows sum the sessions with an operator. Their human frames and takeover episodes are pooled shares.

Operator protocol	Env steps	Episodes	Operator hours	Human frames	Takeover episodes	Training success	Policy-only successes	Final eval	Peak eval
SQUARE-NARROW									
Step 0, dense	0–130k	558	6.0	47.5%	80%	0.37	0/112	0.00	0.02
Step 0, sparse bursts	0–130k	781	3.3	22.9%	54%	0.13	0/363	0.00	0.08
Step 0, long bouts	0–90k	471	2.7	51.3%	66%	0.24	0/162	0.00	0.00
Buffer, then none	130–300k	674	–	0.0%	0%	0.55	374/674	0.78	0.86
After takeoff	150–300k	1,310	3.1	15.6%	35%	0.93	819/856	0.94	0.94
Total	503k	3,120	15.1	32.3%	52%				
SQUARE-BROAD									
Fork at 200k	200–500k	1,660	7.9	44.9%	74%	0.56	1/436	0.02	0.08
Total, both tasks	803k	4,780	23.0	37.0%	60%				

210k (250k without), for 3.1 operator hours and 15.6% human frames. The gap at 300k is about one standard error.

- **RLPD ablations** (five seeds each) to test whether the ∼80k-step zero-success phase is set by failure data or by critic updates: early termination of doomed training episodes, and UTD 40 (actor updates unchanged). Neither changed learning speed (whole-curve mean eval within ±0.12 of the reference, 95% CI).

On SQUARE-NARROW, human takeovers from step 0 did not substitute for the RL phase and slowed it. Once the policy succeeded on its own, they sped up the rest of training, so we run HiL-SERL as a fork of an operator-free run after takeoff. This protocol can show that an operator speeds up training after first success, but not that it shortens the time to first success.

SQUARE-BROAD. The operator-free run, like RLPD, does not take off. Its best eval success over 10^6 environment steps is 1/50, so there is no takeoff point to fork from. We forked the operator-free run at 200k instead, where its policy succeeded in 0 of 50 eval episodes and lifted the nut in 13, and continued with the operator to 500k. The operator helped but did not solve the task. Over the 30 checkpoints from 210k to 500k, the forked policy's eval was above the operator-free run's at 19 and below it at none, averaged 0.020 against 0.001 (0.004 for the five-seed RLPD mean), and peaked at 0.08 against 0.02. Eval ended at 0.02 at 500k, and 1 of 436 policy-only training episodes succeeded, after 7.9 operator hours with 44.9% of frames under operator control. From 440k, the operator took over in 85–96% of episodes and assisted training success rose to 0.95, but the policy's own eval did not follow.

Takeover gripper defect. Until we fixed it partway through the HiL-SERL runs, the SpaceMouse gripper toggle carried over between takeover bouts and was copied into the operator's first action. In the sparse-burst session, 128 of 228 bout starts (56%) flipped the gripper. The defect affected the dense-takeover session and the first ∼62k environment steps of the sparse-burst session, where flipped bouts made up 0.7% of the demonstration half at the fix. The long-bout session, both forks, and the operator-buffer arm ran on the fixed code. The defect does not explain the operator-from-step-0 result, since the long-bout session ran entirely on the fixed code and also ended at 0.00 eval.

Summary. HiL-SERL is less sample-efficient than HiL-IDQL+**Mulligan**. On SQUARE-NARROW it needs over ten times the environment steps to reach 0.9 (Sec. 4), and on SQUARE-BROAD it stays below 0.1 over 500k steps, where HiL-IDQL+**Mulligan** reaches 0.86 with 68k collected frames (Fig. 27).

J Value-Learning Diagnostics and Ablations

Referenced from Sec. 4.2.

J.1 Real-world value learning

We train the real-world critic separately from the actor. It uses the actor's frozen visual encoder together with proprioception to predict state values and action-chunk values. Two Q heads score the actor's proposals, and a distributional V head supplies the value targets (Apps. G.2 and G.6). Its training pool grows across rounds to include demonstrations, corrections, counterfactual demonstrations, and autonomous rollouts, including failures. Completed evaluation rollouts from the **Mulligan** campaign can enter later critics' training pools. A block the critic trained on measures only how well it fit that block, so offline validation uses the blocks excluded from its training.

Offline critic selection. We asked whether diagnostics on recorded trajectories can pick the critic and training checkpoint under which best-of-N reranking performs best on the robot. We checked outcome discrimination, return calibration, value propagation, action sensitivity, and fitted-Q evaluation. A critic can recognize successful trajectories and still fail to choose better actions. For a fixed state, $V(s)$ is constant across proposals, so ranking by $Q(s, a) - V(s)$ is identical to ranking by $Q(s, a)$. Accurate state values alone do not show that the critic can tell the actor's candidate actions apart.

Outcome discrimination and calibration. We measured AUROC against recorded episode outcomes using $V(s_0)$, frame-level values, and the mean value over the last quarter of each episode. We also looked at value traces, at correlation with realized return and task progress, at return-prediction error, and at how far apart successful and failed episodes scored. When we compared training and excluded blocks, we found degradation that training fit hid. For example, late-value AUROC for an INSERT MARKER round-4 critic fell from 0.943 to 0.897 and from 0.910 to 0.862 on two excluded blocks between 125k and 150k updates, while its training-view AUROC rose to 1.000. We kept the earlier checkpoint even though the block-level confidence intervals overlapped. Different metrics also peaked at different checkpoints. As Fig. 29 shows, extending the original round-5 INSERT MARKER critic recipe from 150k to 475k updates increased mean initial value on the same 23 successful held-out episodes from 0.004 to 0.082, while candidate-maximum-Q outcome AUROC on the containing 50-episode block changed from 0.596 to 0.514. The paired bootstrap 95% interval for that change, $[-0.254,\ 0.085]$, is too wide for this block to resolve a decline.

Diagnostics on sampled actions. To look at the scores reranking actually uses, we had each critic score the same stored banks of actor proposals. We computed outcome AUROC of the maximum and mean candidate Q, the recorded action's score, score spread and top-score margins, and sensitivity to action perturbations. They showed weak or unstable action dependence. Because the robot never executed the alternative proposals, every label was still the outcome of a recorded trajectory. A critic could raise its candidate-score AUROC just by telling easy states from hard ones, without choosing better actions within a state. In the THREAD NUT round-5 screen, the apparent improvements over the first deployed round-5 critic had bootstrap intervals containing zero, and differences between training seeds were comparable to the differences between recipes. We examined the rollout blocks separately, since pooling blocks with different outcome frequencies can inflate discrimination. We used these screens to choose candidates for robot tests. They did not identify an offline winner.

Fitted-Q evaluation. We also tested fitted-Q evaluation (FQE), which fits separate policy-evaluation critics to estimate the return of each fixed actor or reranked policy from logged transitions. We backtested FQE on INSERT MARKER with $\gamma = 0.995$, bootstrapping through time-limit truncations and keeping the final converged checkpoint rather than the one with the smallest validation Bellman error. Pooled over 16 historical policies, the FQE estimates correlated with measured robot success at Spearman $\rho = 0.54$ (Fig. 30). The association depended on policy family, with $\rho = -0.37$ for HG-DAgger (6 policies), 0.43 for HG-DAgger+**Mulligan** (6 policies), and 1.00

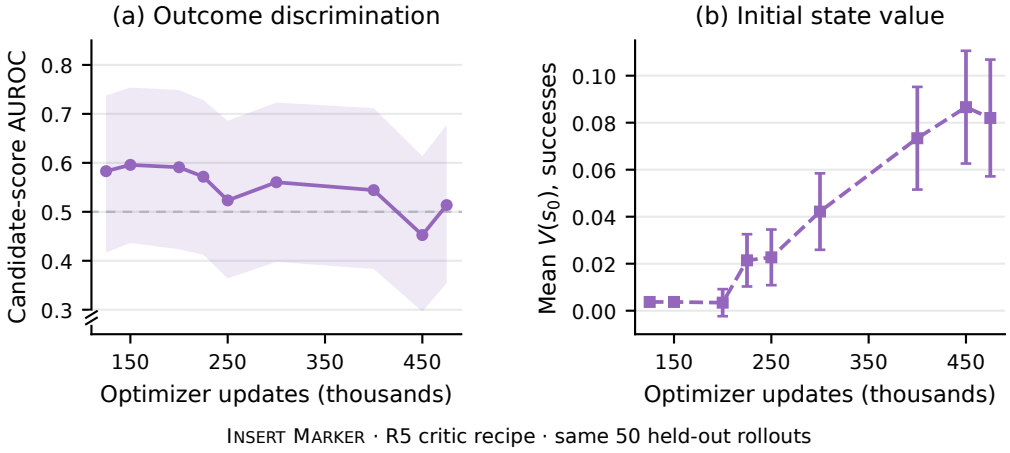


Figure 29: Offline diagnostics across training budgets for the round-5 INSERT MARKER critic recipe, using the same 50 excluded rollouts (23 successes, 27 failures). (a) How well the maximum Q over 32 sampled actions predicts the recorded episode outcome, as AUROC on an axis zoomed to $[0.25, 0.85]$. Shading gives pointwise 95% stratified episode-bootstrap intervals (5,000 resamples), and the dashed line marks chance. (b) Mean initial value on the 23 successful episodes $\pm$ one standard error across episodes. We took the 125k and 150k checkpoints from the original run, and the 200k–475k checkpoints from a longer run with the same recipe and seed.

for HiL-IDQL+**Mulligan** (only 4 policies). The nine-policy subset, excluding HG-DAgger and the round-0 DP actor, had $\rho = 0.78$. This retrospective result did not transfer to comparing a DP actor with its reranked version. FQE favored reranking in several comparisons where the robot measurements were near-ties, and adding baseline data to the fit did not change this. In the THREAD NUT round-5 candidate screen, FQE estimates across critics ranged only from 0.487 to 0.503, comparable to evaluator-seed variation. The FiLM candidate ranked highly by candidate-score AUROC but lowest among those critics by FQE. All three candidates we then tested on the robot scored 36/50. Offline, we could not settle the close policy comparisons that deployment needed.

Practical training-budget choice. With no reliable offline metric, we fell back on critics that had worked on the robot before. We used them as references and kept the update-to-data ratio (UTD) comparable as the training pool grew. Here UTD counts critic optimizer updates per loaded training transition and ignores batch size. At a fixed batch size, it is proportional to how often each transition is sampled. On THREAD NUT, keeping 150k updates while the pool grew from 80,300 to 133,953 and then 173,874 transitions let UTD fall from 1.87 to 1.12 and then 0.86. Later 200k–250k candidate checkpoints brought the step-based ratio back to 1.15–1.44, within the range of the earlier successful runs. The critic deployed for the headline THREAD NUT R5 evaluation trained on 201,788 transitions that add the rollouts of the superseded R5 evaluations and used its 200k-update checkpoint (UTD 0.99). The ratio is only a heuristic, and it misses batch-size changes, which alter data exposure without moving a step-based ratio. In the end, we used the offline diagnostics to flag degradation and robot evaluations to measure performance.

J.2 Critic training-data composition

We test whether reranking depends on the critic's non-human data. On the final R3 with-CF datasets of SQUARE-NARROW and SQUARE-BROAD, we train new DIVL Q/V heads behind the seed-matched frozen actor and normalizer, with the critic sampling only chunks from $\mathcal{D}_h$, those whose every executed action is human-controlled and whose episode succeeded (24,591 of 126,783 valid chunks on SQUARE-NARROW and 56,583 of 202,819 on SQUARE-BROAD). The control for each seed is the existing all-data DIVL critic of App. J.4, trained with the deployed sampler (50% human-

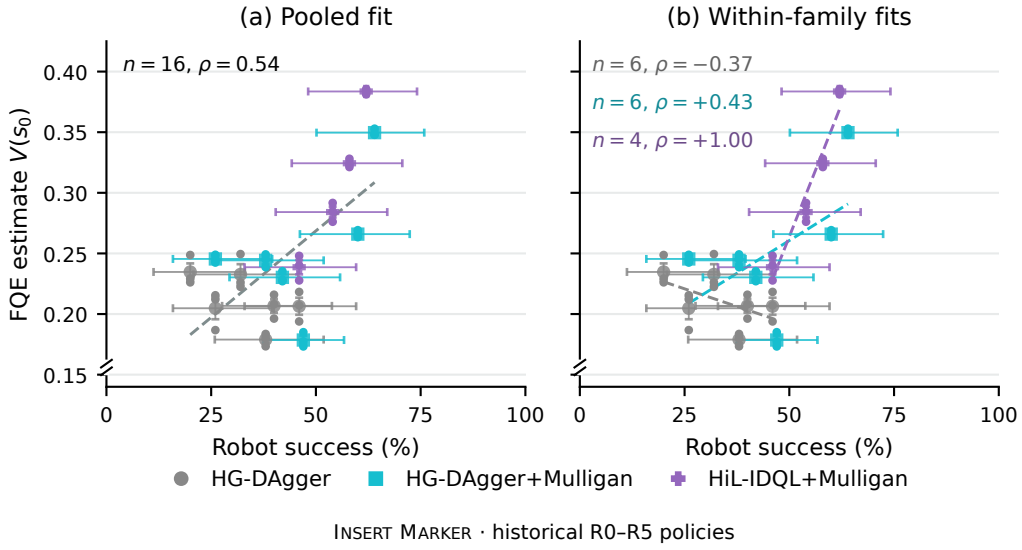


Figure 30: FQE backtest on 16 historical INSERT MARKER policies, shown with (a) a pooled fit and (b) separate fits by policy family. Dashed lines are descriptive least-squares fits, and ρ denotes Spearman correlation. The value axes are zoomed to $[0.14, 0.425]$. Markers average three evaluator seeds, small dots show individual seeds, and vertical whiskers give one standard error across seeds. Horizontal whiskers are Wilson 95% intervals from recorded robot counts (50 attempts per policy, except 100 for round-2 DP), without adjustment for repeated initial states. Evaluators use $\gamma = 0.995$, timeout bootstrapping, and their final 40k checkpoint. FQE scores every policy on the same 150 round-5 reset images (50 initial states), but each robot success rate comes from that policy's own historical round. The round-5 BoN point uses the original $31/50$ critic, not the critic selected later. Because the policies are related and the two axes use different initial-state distributions, the pooled fit is hard to interpret.

success anchors, 50% DAgger anchors). The two arms use the same datasets, normalizer, return bounds, architecture, optimizer, number of updates (150k on SQUARE-NARROW, 250k on SQUARE-BROAD), and seed. We evaluate each final checkpoint at $N{=}1$ and $N{=}32$ on 400 fixed Sobol initial states. $N{=}1$ uses only the actor, so we can check that both arms deploy the same policy.

Restricting the critic to successful human chunks does not improve reranking (Tab. 24). On SQUARE-NARROW the human-only critic is lower in all five seeds, and the 95% interval of the difference excludes zero. On SQUARE-BROAD, the interval covers zero. At $N{=}1$ both arms deploy the identical actor, so their differences are pure evaluation noise. That noise has a seed-level standard deviation of 1.6 and 1.2 points, as large as the $N{=}32$ effects. On these probes, even the all-data critic's gain over the actor is indistinguishable from zero. We cannot attribute a reranking benefit to the non-human data at this round. The ablation shows only that removing that data does not help, and that it costs about one point on SQUARE-NARROW, where success is near ceiling. All of this rests on final probes over 400 initial states, not the 8,000- and 30,000-state grids of App. J.4. Both R3 datasets come from rounds whose sampler used the evaluation-grid cell term (App. F.5), which affects both arms equally.

J.3 Critic-objective variations

We ablate several design choices for the critic objective. Their final-step means are similar (Tab. 25 and Fig. 31). These runs come from a superseded ten-round DAgger campaign on the same MimicGen task as SQUARE-NARROW. It shares no datasets with the R0–R3 campaign of Tab. 13, so absolute rates here are not comparable to any main-text SQUARE-NARROW number. Runs use an MLP critic (hidden dims $[1024, 1024, 1024]$ with layer norm), chunk size 5, 128 action samples,

Table 24: Human-only versus all-data critic behind the same frozen R3 actor, five paired seeds per task, final-step success on 400 fixed Sobol initial states. Δ subtracts all-data from human-only success in percentage points, and the rerank gain subtracts $N{=}1$ from $N{=}32$ success within an arm. We report means over seeds with two-sided Student-t 95% intervals over the five seed differences. The critic plays no role at $N{=}1$.

Task	All-data $N{=}32$	Human-only $N{=}32$	Δ at $N{=}32$	Δ at $N{=}1$ (same actor)	Rerank gain all-data	Rerank gain human-only
SQUARE-NARROW	96.25%	95.15%	-1.10	$+0.05$	$+0.40$	-0.75
			$[-1.91, -0.29]$	$[-1.88, +1.98]$	$[-0.89, +1.69]$	$[-1.83, +0.33]$
SQUARE-BROAD	85.80%	85.00%	-0.80	$+0.05$	-0.40	-1.25
			$[-2.96, +1.36]$	$[-1.38, +1.48]$	$[-1.93, +1.13]$	$[-2.80, +0.30]$

Table 25: Critic-objective variations on the SQUARE-NARROW task, final-step (300k) success rate, mean $\pm$ standard error over training seeds, on a superseded ten-round DAgger campaign rather than the R0–R3 campaign of Tab. 13. The reported critics use the unshifted reward and clipped targets of the baseline row (Tab. 17). "QC" denotes the no-V-network variant that takes a best-of-N max over Q as the TD target.

Condition	Seeds	Final SR (%)
Baseline, reward $[0, 1]$, clipped targets	5	91.60 ± 0.75
Shifted reward $[-1, 0]$, no target clipping	5	91.40 ± 1.91
Intervention frames treated as terminal	5	92.00 ± 1.70
Actor update cadence 4:1:1	5	91.60 ± 0.40
Actor update cadence 2:1:1	5	89.60 ± 1.57
Flat (uniform) critic sampling	3	90.00 ± 0.58
QC, no V-network	4	90.50 ± 0.87
QC + intervention-terminal + cadence 2:1	5	90.00 ± 1.00
QC + shifted reward $[-1, 0]$	4	91.25 ± 2.14

batch size 1024, 300k training steps, and 100 evaluation episodes every 50k steps. We report final-step success averaged over seeds. The deployed simulation critic uses chunk size 6 (App. H).

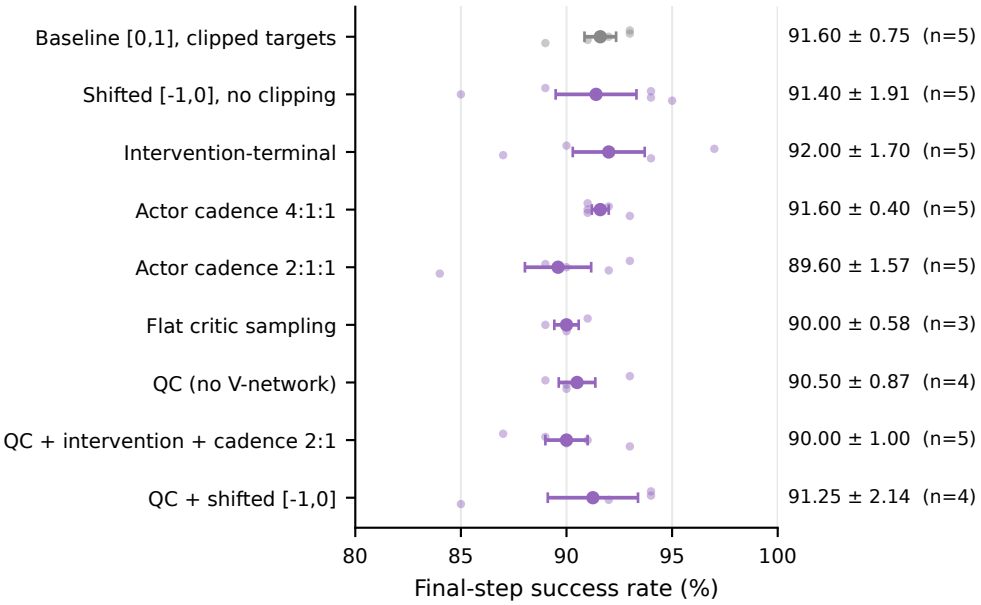


Figure 31: Final-step (300k) success rates for all nine critic-objective conditions in Tab. 25. Large dots and error bars show the mean and one standard error, and small dots show the completed seeds. The two QC conditions with four seeds exclude failed runs, and flat sampling has three seeds.

J.4 Distributional value head (DIVL) vs. scalar IQL critic

Adaptive DIVL in simulation. We compare adaptive DIVL against a scalar-IQL critic in the 34 human-in-the-loop simulation cells across rounds R0–R3, with five matched seeds per cell. Both critics train on the same data and rerank N=32 proposals from the same actor, and each seed is scored on the same grid of 8,000 initial states for SQUARE-NARROW and 30,000 for SQUARE-BROAD. Because adaptive DIVL changes two things at once, the distributional value head and the entropy-adaptive quantile ($\alpha = 0.4$), this comparison does not isolate the distributional form.

Tab. 26 summarizes the paired differences, computed as DIVL minus scalar-IQL success within each cell and seed, averaged over the five seeds of a cell, and weighted equally across cells in the task and combined rows. They are small, and their sign varies from cell to cell. We report the human-in-the-loop simulation N=32 results with these DIVL heads because they match the value formulation of the real-world critics.

Table 26: Adaptive DIVL versus scalar IQL on the 34 human-in-the-loop simulation cells, five paired seeds per cell. Δ subtracts scalar from DIVL success in percentage points and is averaged over seeds within a cell. The mean weights every cell equally, and the range and sign counts are over cell means.

Task	Cells	Mean Δ (pp)	Cell range (pp)	Cells favoring DIVL / scalar
SQUARE-NARROW	16	+0.09	[−0.47, +0.55]	9 / 7
SQUARE-BROAD	18	+0.00	[−0.32, +0.18]	11 / 7
Both tasks	34	+0.04	[−0.47, +0.55]	20 / 14

Earlier fixed-versus-adaptive screens. Tab. 27 separates the two components in three regimes. `divl-fixed` uses the distributional value network with a fixed quantile ($\alpha = 0$), and `divl-adaptive` also enables the entropy-adaptive quantile ($\alpha = 0.4$). Within a row the conditions are matched. Across rows the campaign, seed count, reranking budget, and evaluation set all change, which rules out a controlled comparison across success regimes. The table is also our only fixed-versus-adaptive comparison, since the comparison above evaluates adaptive DIVL alone.

Table 27: DIVL vs. scalar IQL, Δ in percentage points against the scalar control within each regime, matched within a row but not across rows. The low-SR and $\sim$94% rows are round-1 SQUARE-BROAD and SQUARE-NARROW cells, 5 seeds, final-step N=32 SR on 400 fixed-Sobol initial states. The $\sim$90% row is round 15 of a superseded SQUARE-BROAD campaign, 3 seeds, N=128 SR on 200 episodes. Scalar controls show mean $\pm$ SE. The distributional form alone (`divl-fixed`) gives no consistent gain, while the adaptive variant has a positive mean difference at low success and negative differences in both high-success regimes.

Regime (scalar-IQL base SR)	scalar IQL	`divl-fixed`	`divl-adaptive`
Low SR ($\sim$ 70%)	69.95 ± 0.74	$+1.45$	$+4.05$
High SR ($\sim$ 90%)	90.17 ± 0.73	-1.17	-0.83
High SR ($\sim$ 94%)	93.80 ± 0.65	$+0.35$	-0.35

We also tested whether the default success-biased critic sampler accounts for the small differences in the high-success regime. We re-ran the $\sim$94% SQUARE-NARROW round-1 cell under three critic samplers. The scalar-control means stay similar, and neither DIVL variant improves consistently across the three (Tab. 28).

Table 28: DIVL under three critic samplers in the $\sim$94% regime (5 seeds each, final-step N=32 SR on 400 fixed-Sobol initial states, Δ against the scalar control *within* each sampler). `straddled` is success-biased (50% human success / 50% DAgger), `balanced_outcome` is failure-biased (50% success / 50% fail-or-intervened), and `flat` is uniform over the natural distribution.

Critic sampler	scalar IQL (control)	`divl-fixed`	`divl-adaptive`
`straddled`	93.80 ± 0.65	94.15 ($+0.35$)	93.45 (-0.35)
`balanced_outcome`	93.95 ± 0.70	92.60 (-1.35)	93.25 (-0.70)
`flat`	93.80 ± 0.50	93.55 (-0.25)	92.70 (-1.10)

J.5 Critic ensemble size (REDQ)

We use a 2-Q min throughout. Tab. 29 compares it against a larger, REDQ-style [71] ensemble of 10 critics under two aggregations, holding the actor and critic architectures fixed. Neither larger ensemble improves the mean success rate, and REDQ-2 has a lower mean than the control. The late round is round 15 of the superseded SQUARE-BROAD campaign behind the $\sim$90% row of Tab. 27. The same ablation on the main campaign's SQUARE-BROAD round-1 cell is likewise flat.

J.6 Reranking budget N

Tab. 19 lists the deployed budgets, and the actor-only ($N = 1$) and reranked ($N = 32$) results in App. B.1 measure the effect of reranking with the deployed simulation critics. On the superseded campaigns with scalar critics, larger budgets give no further gain. SQUARE-BROAD at round 15 reaches 89.5 ± 0.8%, 92.0 ± 0.5%, and 91.6 ± 0.6% at $N = 1$, 32, and 128 (11 seeds, 200 episodes each), and SQUARE-NARROW at round 10 reaches 96.3, 97.6, and 97.6% (5 seeds).

On the robot, four THREAD NUT R5 critics rescoring fixed banks of 32 proposals changed the outcome AUROC of the top-scored candidate by less than 0.003 between $N = 16$ and $N = 32$.

Table 29: Critic ensemble size on SQUARE-BROAD at round 15 of a superseded DAgger campaign. 5 seeds each, 300k steps, final-step N=32 success rate (mean $\pm$ standard error). "REDQ-2" takes a min over a random pair of critics resampled at every Q evaluation. "Single" draws one random critic per evaluation.

Critic ensemble	|ensemble|	Final N=32 SR (%)	Δ
2-Q min	2	95.70 ± 0.86	—
REDQ-2	10	93.50 ± 0.57	-2.20
Single random critic	10	95.60 ± 0.62	-0.10

K Extended Related Work

Referenced from Sec. 2.

Here we extend Sec. 2 and compare the reset ranges and data budgets that prior real-world systems report with ours.

Real-world RL and autonomous self-improvement. Reinforcement learning has trained policies on physical robots since early policy search over motor skills [76, 77] and end-to-end visuomotor learning from pixels [78, 79], and recent systems make on-robot training sample efficient [18, 21, 22, 80, 81]. These systems report their best results under centimeter-scale initial-state randomization [17, 18, 19] (ranges in App. K.3), where a few episodes reach every initial state. We study ranges wide enough that a round's budget covers only part of the space, so the choice of where to start decides which states the operator supervises. A related line improves robots from their own experience in batches, without a human in the loop [82, 83, 84], and Dong et al. [16] show that a Q-function with best-of-N extraction improves policies in autonomous batch-online interaction. We adopt their value-learning and extraction design, since best-of-N improves a generative policy without backpropagating through its sampling chain (App. K.2), and add the operator's corrections and counterfactual demonstrations to the value function's training data. The subsections below extend this discussion to real-world RL systems and to RL finetuning of expressive policies.

K.1 Real-world RL for robot manipulation

Reinforcement learning on physical robots predates deep policies. Policy search over motor primitives learned motor skills on real arms [76, 77, 85], and model-based policy search learned manipulator control from a handful of trials [86, 87]. Guided policy search then trained visuomotor policies from pixels end to end on a real robot [78], followed by asynchronous off-policy training across several arms [88], large-scale self-supervised grasping [89, 90], and model-free and model-based training on low-cost hands and arms [91, 92, 93, 94, 95]. Later systems ran without human resets [96, 97, 98, 99] or ran iterated batch RL across a fleet [82, 100].

Sample-efficient real-world RL trains policies directly on the physical robot, without corrections from a human in the loop. QT-Opt [79] demonstrated large-scale grasping from pixels but required months of robot time across a fleet. SERL [18] and CQN [80] bring online training down to minutes on a single robot, under narrow initial-state distributions (Tab. 30). IBRL [81] bootstraps RL exploration with an imitation policy for faster convergence, and RL-100 [22] combines diffusion policies with online RL and action chunking. Both learn from autonomous exploration alone. VLAC [101] finetunes a VLA on the real robot with dense progress rewards from a vision-language critic. We instead keep the operator in the loop so that supervision reaches states the policy cannot yet solve, and update the policy offline between rounds. We compile the supervision each system spends per task in App. K.4.

K.2 RL finetuning of expressive policies

Applying RL to action-chunked diffusion, flow, or VLA policies is difficult because optimizing such a policy against a Q-function requires backpropagation through the multi-step denoising chain, which is unstable [102, 29].

Direct policy optimization updates the expressive policy's parameters with an RL objective. DPPO [103] and McAllister et al. [104] apply policy gradients through the full sampling chain. ConRFT [19] and flow Q-learning [105] route Q-function gradients through a single-step distilled policy, and QAM [102] uses adjoint matching to construct stable step-wise objectives for flow policies that use the critic's action gradient.

Auxiliary residual or edit networks keep the base policy fixed and train a small network to refine its outputs: residual action corrections [8, 23, 106], RL in the latent noise space of the sampler [107], residual RL combined with distillation [24], and EXPO [29], which trains a small Gaussian edit

policy alongside on-the-fly value-based action selection. These leave the base objective untouched at the cost of an additional network.

Sampling-based policy extraction draws several candidates from the base policy and executes the highest-valued one. IDQL [51] and V-GPS [108] rerank samples with values learned by IQL [52] and Cal-QL [109], respectively, without action chunking. PA-RL [110] uses Q-optimized actions as supervised targets to finetune OpenVLA [4]. Li et al. [53] extend temporal-difference learning to action chunks, which our chunked value targets build on (Sec. 3.2). Our best-of-N reranking comes from this family and treats the policy as a black box that proposes actions, which lets us apply it to any actor class without changing the base model.

K.3 Initial-state randomization: ours and prior real-world RL

Referenced from Secs. 1 and 4.0.

Tab. 30 records the initial-state randomization ranges that prior real-world RL and HiL-RL systems report, alongside ours. We take all numbers verbatim from the cited papers' setup descriptions or supplementary material. For each system we list the task with the widest published range together with its most precision-critical task, since randomization width and required precision trade off against each other.

We caution against reading this table as a ranking by area, for four reasons that cut in both directions.

What is randomized differs. SERL and the scripted-reset tasks of HiL-SERL vary the *arm's* initial pose while the object stays fixed. SERL expresses proprioception in the frame of the end-effector's initial pose, which its authors note is "equivalent to physically moving the target when viewed relatively from the frame attached to the end-effector" [18]. The observation absorbs the reset by design. A reset of this kind leaves the policy's relative problem unchanged, and its extent cannot be compared with variation in where the object is.

A stated region upper-bounds the variation exercised. RL-100's box-folding blank is described as randomized in translation and rotation over a 75×55 cm region, but the same paper reports that training demonstrations are concentrated near-aligned, at $\theta \in [-30°, 30°]$. The paper also states that it does "not maximize initial randomization" and instead emphasizes task-specific difficulty [22]. An entry in this column gives the region a paper says it samples from, not the spread its rollouts reach. RL-100 differs from the other rows in budget and sensing as well. It spends an order of magnitude more per task, $1{,}890$ episodes over 31.5 robot-hours for orange juicing and $1{,}950$ episodes over 24.5 hours for towel folding, where the human-in-the-loop systems typically use tens of demonstrations and one to two hours of online training. Its policies see calibrated point clouds. The system maps depth into the robot's root frame through Charuco intrinsics and an AprilTag extrinsic and then crops the cloud to the region that holds the task geometry. Ours act on raw RGB with no calibration or segmentation step, and an equal range does not mean an equal problem.

Center ranges versus object regions. We report the range of the object's *center*, and the envelope the whole object sweeps is larger by roughly its footprint. When a paper instead quotes a region the object must lie *within*, its center range is smaller by about the same amount, which for a towel or a paperboard blank is a large correction.

Evaluation protocols. With few exceptions, each paper reports success rates as totals from its own sessions, with the arm order, blinding, and per-episode reset procedure left to the reader. This matters because a policy can drop from $13/20$ to $0/8$ on identical initial conditions after an unrelated change to the lab [64]. We tabulate what each paper states. Our own evaluations pair the arms on the same held-out Sobol states, run blind and interleaved within a session, and place each reset from a per-episode schematic (App. D). Readers can check the pairing and the realized initial states in the released video of every evaluation episode behind the compared methods' reported results, which comes with its per-episode outcome (`https://huggingface.co/mulligan`).

In the prior grasp-and-insert tasks at millimeter clearance that randomize the object pose, the region spans a few centimeters and yaw at most $10°$. The tasks with large stated regions end in folding,

Table 30: Initial-state randomization as *reported* by prior real-world RL systems, and ours. "Varies" distinguishes randomization of the object pose from randomization of the arm's start pose with the object fixed, and marks the tasks in which the *target* the object must reach is also randomized. DoF counts the independent dimensions of the initial state that vary (three for an object's x, y, and yaw). Where a paper reports a range for one element only, we take the remaining elements to be fixed. We quote position ranges as published. Orientation is in-plane (yaw). Table numbers under "Source" refer to each paper's own numbering (appendix tables for HiL-SERL and ConRFT). ‡RL-100 states that the blank's and the half-orange's rotations are randomized but gives no range.

Method	Task	Varies	DoF	Position range	Yaw	Source
HiL-SERL [17]	IKEA side panel	arm	3	8 cm in x, y	1°	Tab. 6
HiL-SERL [17]	Cable clipping	object	3	4 cm in x, y	10°	Tab. 5
ConRFT [19]	Pick banana	object	2	3 cm in x, y	none	Tab. V
ConRFT [19]	Insert wheel	object	2	2 cm in x, y	none	Tab. XI
RL-100 [22]	Box folding	object	3	75×55 cm	unstated‡	Suppl.
RL-100 [22]	Orange juicing	object	3	29×20 cm tray	unstated‡	Suppl.
RL-100 [22]	Pouring	object + target	4	cup 39×16, plate 28×20 cm	none	Suppl.
RL-100 [22]	Unscrewing	object	2	12.5 cm-diam. circle	none	Suppl.
SERL [18]	Cable routing	arm	2	20×20 cm	none	Tab. 2
SERL [18]	PCB insertion	arm	2	10×10 cm	none	Tab. 2
Ours	INSERT MARKER	object + target	5	pen 15.2×30.5, holder 5.1×10.2 cm	360°	—
Ours	THREAD NUT	object + target	5	nut 25.4×45.7, peg 5.1×10.2 cm	360°	—
Ours	ROUTE CABLE	object + target	7	rope 40.6 cm in x, each clip 40.6×22.9 cm	clips $\pm45^\circ$	—
Ours (sim)	SQUARE-NARROW	object	3	nut 0.5×11.5 cm, peg fixed	360°	—
Ours (sim)	SQUARE-BROAD	object + target	5	nut 23.0×51.0, peg 40.0×40.0 cm	360°	—

pouring, or seating a compliant object. On INSERT MARKER and THREAD NUT, we place the object anywhere in a region tens of centimeters across, at any yaw, and the task still ends in a millimeter-clearance insertion.

K.4 Data and interaction budgets: ours and prior real-world RL

Referenced from App. K.1.

Tab. 30 records how wide each system's initial-state distribution is. Tab. 31 records how much supervision and on-robot experience each system spent covering it. Neither table compares cleanly across systems, since the robots, tasks, required precision, and units all differ. HiL-SERL and SERL count environment transitions in a replay buffer while ConRFT and RL-100 count episodes, and each paper folds scripted resets, intended stops and onboard computation into its "training time" in its own way. We keep each paper's unit and mark a cell *unstated* when the paper does not give the number.

Our rows come from Tab. 13 and App. E.3 and leave out the simulation tasks, whose episodes cost no robot time. We did not log session time, so our robot times are recorded trajectory time at 15 Hz (collection plus held-out evaluation) rather than the wall-clock hours these papers headline, and they exclude resets, object placement, and operator pauses. The one session we timed from save timestamps, a 200-episode ROUTE CABLE collection, took about 5.2 h of active session time for 104 min of recorded trajectory. At that ratio, a wall-clock figure comparable to the other rows would be roughly three times ours. Every frame carries a source tag, which makes our human-versus-autonomous split exact. The split covers only the arm's own training data, not the evaluation rollouts that the critic also trains on.

Table 31: Data and interaction budgets as *reported* by prior real-world RL systems, and ours. Rows match Tab. 30 task for task wherever the source paper reports per-task numbers. "Demos" are the offline human demonstrations that initialize learning, "Human corrections" are in-the-loop interventions, and "Autonomous experience" is the data the policy collected on the robot, in the paper's own unit. "Robot time" is the wall-clock number each paper headlines. Its definition varies, with scripted motion, intended stops and onboard computation included in HiL-SERL and ConRFT, and computation, resets and intended stops in SERL. RL-100 reports data-collection time per stage, and the entry here sums its three stages (demonstration, iterative offline RL, online RL). Table numbers under "Source" refer to each paper's own numbering. †Both human-in-the-loop systems report the intervention *rate*, the ratio of intervened to total timesteps, only as a training curve, for every task in HiL-SERL (Figs. 5 and 26) and for representative tasks in ConRFT (Figs. 3–4). Neither reports a count of interventions or a total intervened time for any task. ‡HiL-SERL's "total number of RL transitions" is the replay-buffer size at convergence and includes human-intervention timesteps, so it is not purely autonomous experience. The p1/p2 entries are the two side panels, which the paper trains and times separately. *ConRFT states only that the policy collects "approximately 80 to 120 successful and failed trajectories" during the online stage, for all tasks together. §These systems have no in-the-loop corrections and train from demonstrations plus autonomous rollouts. ¶Sum over the two Orange Juicing subtasks (Placing and Removal), which Tab. S3 reports separately. ‖Our rows count the HG-DAgger+**Mulligan** arm's own training data at the final round (Tab. 13), split into human-controlled frames (demonstrations, corrections, and counterfactual replays) and autonomous frames of the same episodes, at 15 Hz. For our rows, "robot time" adds up the recorded length of that arm's collection episodes and of the held-out evaluations of the three headline arms. It counts full episodes, including any tail that a later outcome review trimmed, and leaves out resets and operator setup. On ROUTE CABLE, we measured session wall-clock time from save timestamps at about three times the recorded trajectory time.

Method	Task	Demos	Human corrections	Autonomous experience	Robot time	Source
HiL-SERL [17]	IKEA side panel	20	rate curve only†	31,000 / 36,000 transitions‡ (p1/p2)	2 / 1.75 h	Tab. 1, 6
HiL-SERL [17]	Cable clipping	20	rate curve only†	28,000 transitions‡	1.25 h	Tab. 1, 5
ConRFT [19]	Pick banana	20	rate curve only†	80–120 episodes*	45 min	Tab. I, V
ConRFT [19]	Insert wheel	30	rate curve only†	80–120 episodes*	60 min	Tab. I, XI
RL-100 [22]	Box folding	200 (3 h)	none§	608 offline + 186 online epi.	15.3 h	Tab. S3
RL-100 [22]	Orange juicing	109 (2 h)¶	none§	791 offline + 990 online epi.¶	31.5 h¶	Tab. S3
RL-100 [22]	Pouring	64 (1 h)	none§	741 offline + 129 online epi.	9.3 h	Tab. S3
RL-100 [22]	Unscrewing	31 (0.5 h)	none§	467 offline + 288 online epi.	8 h	Tab. S3
SERL [18]	Cable routing	20	none§	unstated	31 min	Tab. 2
SERL [18]	PCB insertion	20	none§	unstated	20 min	Tab. 2
Ours	INSERT MARKER	100 / arm (R0)	R0–R5, 50 epi. / round / arm; 38,017 human frames‖	43,509 frames‖	91 min + 367 min eval‖	—
Ours	THREAD NUT	100 / arm (R0)	R0–R5, 50 epi. / round / arm; 34,536 human frames‖	31,818 frames‖	74 min + 277 min eval‖	—
Ours	ROUTE CABLE	100 / arm (R0)	R0–R5, 100 epi. / round / arm; 112,292 human frames‖	103,183 frames‖	244 min + 322 min eval‖	—